

Data Structure and Algorithm

yan zhao

Contents

1	Data structure	7
1.1	Array	7
1.1.1	Basic operation of array	7
1.1.2	Interview questions	10
1.1.2.1	Mono stack	13
1.1.2.2	Mono queue	15
1.1.2.3	Hints for array questions.	15
1.2	Linked list	17
1.2.1	Basic	17
1.2.2	Operations	18
1.2.3	Interview questions	19
1.3	Matrix	24
1.3.1	Sparse matrices	24
1.3.2	Matrices traversal methods and operation	25
1.3.3	Interview questions	30
1.4	Stack	30
1.4.1	Application of Stack	30
1.4.1.1	Offline Equivalence class	31
1.4.2	Interview questions	32
1.5	Queue	33
1.5.1	Circule Queue	33
1.5.2	Application of queue	33
1.6	Heap	34
1.6.1	Basic	34
1.6.2	Application	36
1.7	Tree	38
1.7.1	Basic knowledge	38
1.7.1.1	Basic conception	38
1.7.1.2	Other property of tree	39
1.7.2	Search Tree	40
1.7.3	Tree and recursion	41
1.7.3.1	Base case	41
1.7.3.2	Traversal Order	43
1.7.3.3	Interface	46
1.7.3.4	Span child tree	47
1.7.3.5	Two trees	48
1.7.3.6	Traps	49
1.7.3.7	Conclusion	50
1.7.4	Interview questions	50

1.7.4.1	Print part of tree	50
1.7.4.2	Build tree	52
1.7.4.3	Change tree	56
1.7.4.4	Other	57
1.8	Graph	58
1.8.1	Basic	58
1.8.2	DFS and BFS	60
1.8.2.1	DFS	60
1.8.2.2	BFS	61
1.8.3	Spanning tree	62
1.8.4	Application	62
1.9	Other customized data structure	64
1.9.1	union find	64
1.9.2	LRU and LFU	65
1.9.3	O(1) insert, delete and getRandom	66
2	Algorithm	67
2.0.1	Solution space	67
2.0.1.1	Big O	68
2.1	Recursive	69
2.1.1	Basic	69
2.1.2	Basic recursive pattern	70
2.1.2.1	When to Use Recursion (Despite the Overhead)	76
2.1.3	When to Avoid Recursion	77
2.1.4	Rule of Thumb	77
2.1.5	Conclusion	77
2.2	Math and geometry basic	78
2.2.1	Geometry	78
2.2.1.1	Convex hull	78
2.2.2	Permutation and combination	79
2.2.3	Bit	80
2.2.4	Usage of modulus %	81
2.3	Two pointers	81
2.3.1	Middle to both ends	82
2.3.2	slow and fast	83
2.3.3	slide windows	84
2.4	Sort	85
2.4.1	Kth element	85
2.4.2	Simple sort	87
2.4.3	quick sort	89
2.4.3.1	Quick sort	89
2.4.3.2	Merge sort	91
2.4.3.3	heap sort	93
2.4.3.4	introsort	94
2.4.4	Bucket and radix sort	94
2.4.4.1	Bucket sort	95
2.4.4.2	Radix sort	95
2.4.5	Summary	96
2.5	Greedy	97

2.5.0.1	Intervals questions	97
2.5.1	Application	100
2.6	Divide Conquer	101
2.6.1	Binary search	101
2.6.2	Other	104
2.7	Backtracking	104
2.7.1	Basic	104
2.7.2	Permutation, combination and subset	106
2.7.2.1	No duplicate, single select	106
2.7.2.2	No duplicate, multi select	108
2.7.2.3	Duplicate, single select	109
2.7.2.4	Parenthesis	110
2.7.2.5	Summary	110
2.7.3	Interview questions	111
2.8	Dynamic programming	112
2.8.1	Basic Idea	112
2.8.2	Three questions	113
2.8.3	one dimension	115
2.8.3.1	Conclusion	116
2.8.4	Two dimension	116
2.8.5	Three dimension	122
2.8.6	Summary	122
2.9	Branch and bound	125
2.10	Summary of algorithms	130
3	Interview preparation	133
3.1	Basic principle	133
3.2	Basic data structure and algorithms	133
3.2.0.1	Math	134
3.3	Code Style	134
3.3.1	STL container	134
3.3.1.1	construct and init	134
3.3.1.2	insert and erase	135
3.3.1.3	update	135
3.3.1.4	look up	135
3.3.1.5	operations	135
3.3.1.6	string	136
3.3.2	STL algorithms	136
3.3.3	Views	137
3.3.4	functor and lambda	139
3.3.5	Bit	139
3.3.6	Tips and Precautions	139
3.3.6.1	Tips	139
3.3.6.2	Precautions	143
3.4	Code template	143
3.5	Typical interview questions	146
3.5.1	data structure	146
3.5.1.1	array	146
3.5.1.2	list	147

3.5.1.3	tree	147
3.5.1.4	graph	147
3.5.1.5	interval	147
3.5.1.6	Two pointers-slow and fast	147
3.5.1.7	Two pointers-sliding windows	147
3.5.1.8	D&C	147
3.5.1.9	Binary search	147
3.5.1.10	Backtracking	148
3.5.2	Famous questions in leetcode	148
3.5.2.1	Math, permutation and combination.	148
3.5.2.2	String	149
3.5.2.3	Double pointer	151
3.5.2.4	STL container	151
3.5.2.5	STL Algorithm	152
3.5.2.6	fancy code	153
3.5.2.7	views	155
3.5.2.8	subset and subarray	156
3.5.2.9	tree	158
3.5.2.10	interval	160
3.5.2.11	bit	161
3.5.2.12	Good modeling	162
3.6	Project	165

Chapter 1

Data structure

Data structure is known as ADT (Abstract Date Type), which is independent of implementation. For example, stack can be implemented by array or linked list inside. Such as `std::stack` in STL library, It's implemented by `std::deque` default.

Linear and hierarchical structures. Array, linked lists, stacks and queue are linear structures. And tree, graph, heaps are hierarchical.

STL is a good reference when you learn data structure, especially (API) design and interface of STL container. such as `push`, `pop` and `top` for `stack` and `priority_queue`. For `queue`, besides `push`, `pop`, there are also `front()` and `back()`. Just remember five words: **push, pop, front, back and top.**

1.1 Array

1.1.1 Basic operation of array

All languages support array and it is a build-in data type in most of language.

```
1 //C/C++ language
2 int a[3]; or int a[4][5].
3 vector<int> vi(4,0); //STL
4 array<int, 4> a1 = {1, 2, 3, 4}; //new array in STL
5
6 //C# language
7 object[] a1 = {"string", 123, true};
```

Array support three basic operations: random access (index), insert, erase.

Dynamic two dimension array (ragged array) can be implemented below: `char ** p`; . Pay attention to `p[1][2]` and `pa[1][2]` in the below code. They are all correct forms.

All ragged array should include terminate flag, so you can know the length of each row. For C-style string ragged array, the 'NULL' in the end of string can be used as terminate flag directly, so ragged array is idea container for C-style string array.

```
1 char **p;
2 char* a[]={ "zhao", "yan", "hello" };
3 p = a; // use pointer of pointer to catch array of pointer.
4 printf( "%c\n", *((p+1)+2)); //p[1][2]
5
6 int (*pa)[3]; //pointer to array.
```

```

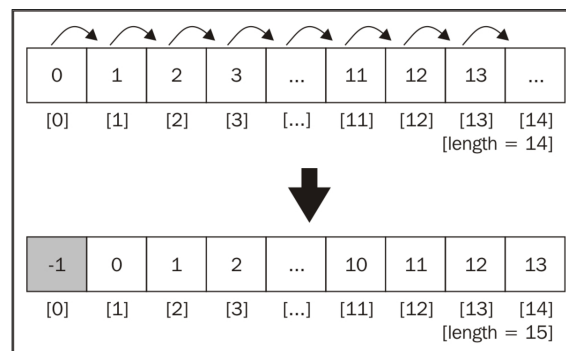
7 int aa[2][3] = {{1,2,3},{4,5,6}};
8 pa = aa;
9 int bb[][3]; //bb must have 3 as last dimension. c++ syntax.
10 bb = aa;
11 printf( "%d\n", pa[1][2] ); //*(*(pa+1)+2);

```

line 7-10 this is array of **array**. so you have to use a **pointer to array** to catch it. Pay attention to the syntax of a pointer to array in C++ language: `int (*pointer_to_array)[3];`

If you have an array of big object, and you need to sort it. You can build a array of pointer to point to each object in the array. When you sort the an array of big object, you just move pointer instead, It is called "**indirect addressing**", It can help you to avoid moving big chunk of data.

If you want to insert an element into an array, you **need to copy backward from end**. if erase an element from an array, **copy forward from erasing point**. Most of time, you can just call `insert(pos)` or `erase(pos)` in STL container. **It inserts the element before the pos and erase the element at th pos**, and pos is an iterator.



Reverse array can be implemented by **swap + double pointers**, But reversing list need different algorithm, so in STL , `std::list` has its own reverse member function. Reverse list is a typical interview question, and you should know it very well.

Boundary about array structure.

- Remember: if **n** is size of array, **n/2** is the middle element (odd) or the first element in the second half (even). This pattern is very helpful for you to write reverse function.
- Remember: if **end** is end of index, **end/2** is the middle element (odd) or the last element in the first half(even). Be careful with this, most of time, D&C like to use start and end index, not size of array.
- `std::distance(v.begin(), v.end());` is size of array/vector.

```

1 void reverseArray(int a [], int n){
2     int i, j, temp;
3     for ( i=0, j=n-1; i<n/2; ++i, --j ) {
4         swap(a[i], a[j]);
5     }
6 }
7
8 //another is based on iterator
9 auto bi = v.begin();
10 auto ei = v.end();
11 auto mi = next(bi, distance(bi, ei)/2 );
12
13 for (auto i = bi, j = prev(ei); i<mi; ++i, --j){

```

```

14  iter_swap(i,j);
15  }
16
17
18  for (last--; first < last; ++first, --last) {
19      iter_swap(first, last);
20  }
21
22  //pay attention next and distance usage, put being in front of end.
23  //distance(v.end(), v.begin()) is just size of array/vector.

```

Write a function `rotate(arr, d, n)` that rotates `arr` of size `n` by `d` elements.

- Single rotate method

```

1  while(d-- > 0) { //d-- > 0 is good trick
2      int temp = Array[0];
3      for (int i = 0; i < n-1; i++) { //left open, right close, when i < n-1, it means
4          //that we need to access array[i+1]
5          Array[i] = Array[i + 1];
6      }
7      Array[n-1] = temp;
8  }

```

- Reverse method. That is a very efficient implementation.

```

1  rotate(arr, d, n)
2      reverse(arr, 1, d);
3      reverse(arr, d + 1, n);
4      reverse(arr, 1, n);

```

Idea: middle part reverse twice, so keep origin value

- For `std::list`, you can use `std::rotate`, but the time complexity is $O(n)$, you can use splice function to reach $O(1)$.

```

1  std::list l{1, 2, 3, 4, 5};
2  auto i2 = std::next(i1, 2);
3
4  l.splice(l.end(), l, l.begin(), i2); //output 3, 4, 5, 1, 2
5  //this is a good pattern, need to remember, l.splice(l.end(), ...)

```

- You can use `std::rotate` in insert sort method. It also can be used in slide. Detail can be found in "Top 5 Beautiful C++ std Algorithms Examples"

```

1  for (auto i = start; i != end; ++i)
2      std::rotate(std::upper_bound(start, i, *i), i, std::next(i));
3  //all stl algorithms deal with left close, right open space.
4  //i and std::next(i) is right open point.
5
6  template <typename It>
7  auto slide(It f, It l, randIter p) -> std::pair<It, It> {
8      // p < [f...l] < p, two possibles
9      if (p < f) return { p, std::rotate(p, f, l) };
10     if (l < p) return { std::rotate(f, l, p), p };
11     return { f, l };
12 }
13
14 ForwardIt rotate( ForwardIt first, ForwardIt n_first, ForwardIt last );
15 // rotate returns Iterator to the new location of the element pointed by first

```

line 2: `rotate(a, b, c)` is `[a,b)` and `[b,c)` scope. they are all left close, right open scope. In this way, you can understand why we need `std::next(i)` here.

- You need to remember below code snippet. **reverse→rotate→ insertion order and slide.**

We can also use partition to implement gather. **Partition return: Iterator to the first element of the second group.**

Gather all eligible element around iterator p

```
template <typename BiIt, typename UnPred>
auto gather(BiIt f, BiIt l, BiIt p, UnPred s) -> std::pair <BiIt, BiIt>{
    return { stable_partition(f, p, not1(s)),
            stable_partition(p, l, s) };
}
```

1.1.2 Interview questions

- Two sum.
 1. Answer 1: First Sort $O(n \cdot \log(n))$, then use two pointers. **for the two pointers, from two ends to the middle, use while($l < r$) is a good strategy.** Two typical applications are reverse and the two sum.

```
1 bool hasArrayTwoCandidates(int A[], int arr_size, int sum) {
2     int l, r;
3     sort(A, A + arr_size);
4
5     /* Now look for the two candidates in the sorted array*/
6     l = 0;
7     r = arr_size - 1;
8     while (l < r) {
9         if (A[l] + A[r] == sum)
10            return 1;
11        else if (A[l] + A[r] < sum)
12            l++;
13        else // A[i] + A[j] > sum
14            r--;
15    }
16    return 0;
17 }
```

Idea: Don't need keep index information, so you can sort it.

2. Answer 2: Hash $O(n)$
- Count triplets with sum smaller than a given value.

```
1 1) Sort the input array in increasing order.
2 2) Initialize result as 0.
3 3) Run a loop from i = 0 to n-2. An iteration of this loop finds all
4 triplets with arr[i] as first element.
5     a) Initialize other two elements as corner elements of subarray
6        arr[i+1..n-1], i.e., j = i+1 and k = n-1
7     b) Move j and k toward each other until they meet, i.e., while (j < k)
8         (i) if (arr[i] + arr[j] + arr[k] >= sum), then do k--
9
10        // Else for current i and j, there can (k-j) possible third elements
11        // that satisfy the constraint.
12        (ii) Else Do ans += (k - j) followed by j++
```

Idea: Same Idea as previous question, Don't need keep index information, so you can sort it. brutal force, only sort can help here.

- Given an array of positive integers. All numbers occur even number of times except one number which occurs odd number of times. Find the number in $O(n)$ time and constant space. Example: I/P = [1, 2, 3, 2, 3, 1, 3] and O/P = 3. The Best Solution is to do bitwise XOR of all the elements. XOR of all elements gives us odd occurring element. Please note that XOR of two elements is 0 if both elements are same and XOR of a number x with 0 is x .

```
1 result = result ^ a[i];
```

Idea: Perform some calculation on all elements.

- You are given a list of $n-1$ integers and these integers are in the range of 1 to n . There are no duplicates in list. One of the integers is missing in the list. Write an efficient code to find the missing integer.

```
1 1) Get the sum of numbers
2   total = n*(n+1)/2
3 2 Subtract all the numbers from total and you will get the missing number.
```

Idea: Perform some calculation on all elements.

- Max product of the three numbers for a given array of size N . **Find the three largest numbers in the array (n_1, n_2, n_3) and the two smallest numbers (m_1, m_2). The answer is either $n_1 \times n_2 \times n_3$ or $n_1 \times m_1 \times m_2$. The tricky thing here is there are two potential answers.**

```
1 nth_element(...)
```

Idea: Not sort, but n th max element based on heap

- Given an unsorted array of non-negative integers, find a **continuous** subarray which adds to a given number. Examples: Input: arr[] = 1, 4, 20, 3, 10, 5, sum = 33; Ouptut: Sum found between indexes 2 and 4.

```
1 int arr[] = {1, 4, 20, 3, 10, 5};
2 int left = 0; int right = 0;
3 int sum = 0;
4 while(right < 6){
5     sum+=arr[right];
6     ++right;
7
8     while(sum>33){
9         sum-=arr[left];
10        ++left;
11        if (sum == 33){
12            cout<<left<<"_"<<right<<"_"<<endl;
13        }
14    }
15 }
```

Idea: 1) You must keep index information, so you can't sort. That is a typical sliding window problems. Keep left and right two index. Sliding window can be found in algorithm chapter.

- Given an array `arr[]` of integers, find out the difference between any two elements such that larger element appears after the smaller number in `arr[]`. Examples: If array is [2, 3, 10, 6, 4, 8, 1] then returned value should be 8 (Diff between 10 and 2). If array is [7, 9, 5, 6, 3, 2] then returned value should be 2 (Diff between 7 and 9). That is also a one time stock buy sell problem.

```
1 In this method, instead of taking difference of the picked element with every
  other element, we take the difference with the minimum element found so far.
  So we need to keep track of 2 things:
```

```

2 1) Maximum difference found so far (max\_diff).
3 2) Minimum number visited so far (min\_element).

```

Keep position, but this problem can be divided by sub-problem. Such as, [7, 9, 5, 6, 3, 2] can be seen as [7,9] [5, 6] [3] [2] Once current element is less than min_element, begin a new sub-problem.

- Inversion Count for an array indicates – how far (or close) the array is from being sorted. If array is already sorted then inversion count is 0. If array is sorted in reverse order that inversion count is the maximum. Formally speaking, two elements $a[i]$ and $a[j]$ form an inversion if $a[i] > a[j]$ and $i < j$. Example: The sequence 2, 4, 1, 3, 5 has three inversions (2, 1), (4, 1), (4, 3).

```

1 int main() {
2     vector<int> vi = {2, 4, 1, 3, 5};
3     set<int> si;
4     int sum = 0;
5     for(auto i : vi){
6         si.insert(i);
7         cout<<i<<endl;
8         auto it1 = si.upper_bound(i);
9         if (it1 != si.end())
10            cout<<"***"<<it1<<endl;
11            sum += distance(it1, si.end());
12    }
13
14    cout<<sum<<endl;
15    return 0;
16 }

```

Line 10: set also support iterator, it will output sorted element from beginning to end

Line 12 distance function, you have to input smaller iterator first. for set, if you input end, it will fall into dead loop.

For this questions, there are three points: 1) You need keep index(position) information. (No sort, no calculation). 2) you can't divid it to subj-problem. 3) You have to use auxiliary DS outside. such as set. which store how many element is greeter than current element.

- Convert array into Zig-Zag fashion ? The converted array should be in form $a < b > c < d > e < f$. Just like bubble sort, use swap to make it satisfy order. Pay attention here, no global order requirement, so just **divide task to atomic operation**.

```

1 void zigZag(int arr[], int n) {
2     // Flag true indicates relation "<" is expected,
3     // else ">" is expected. The first expected relation is "<"
4     bool flag = true;
5     for (int i = 0; i <= n - 2; i++) {
6         if (flag){ /* "<" relation expected */
7             /* If we have a situation like A > B > C,
8              we get A > C < B by swapping B and C */
9             if (arr[i] > arr[i + 1])
10                swap(arr[i], arr[i + 1]);
11        }
12        else{ /* ">" relation expected */
13            /* If we have a situation like A < B < C,
14             we get A < C > B by swapping B and C */
15            if (arr[i] < arr[i + 1])
16                swap(arr[i], arr[i + 1]);
17        }
18        flag = !flag; /* flip flag */
19    }

```



```

20 }
21 // use i%2 == 0 to instead flag is also OK here.

```

1.1.2.1 Mono stack

- Write a program to print all the LEADERS in the array. An element is leader if it is greater than all the elements to its right side. And the rightmost element is always a leader. For example in the array 16, 17, 4, 3, 5, 2, leaders are 17, 5 and 2.

```

1 Scan all the elements from right to left in array and keep track of maximum till
  now. When maximum changes it's_value, print it.

```

```

1 can also use mono stack, from right to left, when stack is empty, it's_LEADERS_
  number.

```

- Next Greater Element. Given an array, print the Next Greater Element (NGE) for every element. The Next greater Element for an element x is the first greater element on the right side of x in array. Elements for which no greater element exist, consider next greater element as -1. For the input array [4, 5, 2, 25], the next greater elements for each element are as follow.

4 --> 5 | 5 --> 25 | 2 --> 25 | 25 --> -1

```

1 void printNGE(int arr[], int n){
2     stack<int> s;
3     /* push the first element to stack */
4     s.push(arr[0]);
5
6     // iterate for rest of the elements
7     for (int i = 1; i < n; i++) {
8
9         if (s.empty()) {
10             s.push(arr[i]);
11             continue;
12         }
13         /* if stack is not empty, then pop an element from stack.
14         If the popped element is smaller than next, then
15         a) print the pair
16         b) keep popping while elements are
17         smaller and stack is not empty */
18         while (s.empty() == false && s.top() < arr[i]) {
19             cout << s.top()
20             << "—>" << arr[i] << endl;
21             s.pop();
22         }
23         /* push next to stack so that we can find
24         next greater for it */
25         s.push(arr[i]);
26     }
27     /* After iterating over the loop, the remaining
28     elements in stack do not have the next greater
29     element, so print -1 for them */
30     while (s.empty() == false) {
31         cout << s.top() << "—>" << -1 << endl;
32         s.pop();
33     }
34 }

```

- One of the main ideas of a monotonic stack is to discard elements that do not meet certain conditions and avoid pushing them onto the stack, as they are irrelevant to the solution.
- Given a string *s*, remove duplicate letters so that every letter appears once and only once. You must make sure your result is the smallest in lexicographical order among all possible results.

Our algorithm should pop elements from the stack when `stk.peek() > c`. However, there are two cases to consider:

1. Case 1: If the character at `stk.peek()` appears later in the string, it can be safely popped. Since it will reappear later, it can be pushed back onto the stack, satisfying lexicographical order requirements.
2. Case 2: If the character at `stk.peek()` does not appear again later, it cannot be popped. As mentioned earlier, the stack will not contain duplicate elements. Popping it means losing this character permanently.

Returning to the example *s* = "bcac", when inserting the character 'a', we see that the lexicographical order of 'c' is larger than 'a', and 'c' also appears later in the string. Hence, 'c' is popped from the stack.

The while loop continues, and we find that the lexicographical order of 'b' is still larger than 'a'. However, since 'b' does not appear again after 'a', it should not be popped. The key question is: how can the algorithm determine how many 'b's and 'c's exist after the current 'a'?

```

1 String removeDuplicateLetters(String s) {
2     Stack<Character> stk = new Stack<>();
3
4     // Maintain a counter to record the frequency of each character in the string
5     // Since the input consists of ASCII characters, size 256 is sufficient
6     int[] count = new int[256];
7     for (int i = 0; i < s.length(); i++) {
8         count[s.charAt(i)]++;
9     }
10
11     boolean[] inStack = new boolean[256];
12     for (char c : s.toCharArray()) {
13         // Decrease the count for each character as it is traversed
14         count[c]--;
15
16         // Skip if the character is already in the stack
17         if (inStack[c]) continue;
18
19         while (!stk.isEmpty() && stk.peek() > c) {
20             // Stop popping if the stack top character will not appear again
21             if (count[stk.peek()] == 0) {
22                 break;
23             }
24             // If it will appear again, it can be safely popped
25             inStack[stk.pop()] = false;
26         }
27         stk.push(c);
28         inStack[c] = true;
29     }
30
31     // Build the result string from the stack
32     StringBuilder sb = new StringBuilder();
33     while (!stk.isEmpty()) {
34         sb.append(stk.pop());
35     }
36     return sb.reverse().toString();
37 }

```

1.1.2.2 Mono queue

- Given an array and an integer K, find the maximum for each and every contiguous subarray of size k. It looks like two sum problem. there are three different levels time complexity.
 - The first brute force, $O(n*k)$.
 - The second, use balance Search Tree(std::set), $O(n*\log(k))$.
 - the third, use queue $O(n)$

```

1 std::deque<int> Qi(k);
2 /* Process first k (or first window) elements of array */
3 int i;
4 for (i = 0; i < k; ++i) {
5     // For every element, the previous smaller elements are useless so
6     // remove them from Qi
7     while ((!Qi.empty()) && arr[i] >= arr[Qi.back()])
8
9     Qi.pop_back();    // Remove from rear
10    Qi.push_back(i);  // Add new element at rear of queue
11 }
12 // Process rest of the elements, i.e., from arr[k] to arr[n-1]
13 for (; i < n; ++i) {
14     // The element at the front of the queue is the largest element of
15     // previous window, so print it
16     cout << arr[Qi.front()] << "_";
17     // Remove the elements which are out of this window
18     while ((!Qi.empty()) && Qi.front() <= i - k)
19     Qi.pop_front();  // Remove from front of queue
20     // Remove all elements smaller than the currently
21     // being added element (remove useless elements)
22     while ((!Qi.empty()) && arr[i] >= arr[Qi.back()])
23     Qi.pop_back();
24     // Add current element at the rear of Qi
25     Qi.push_back(i);
26 }
27 // Print the maximum element of last window
28 cout << arr[Qi.front()];

```

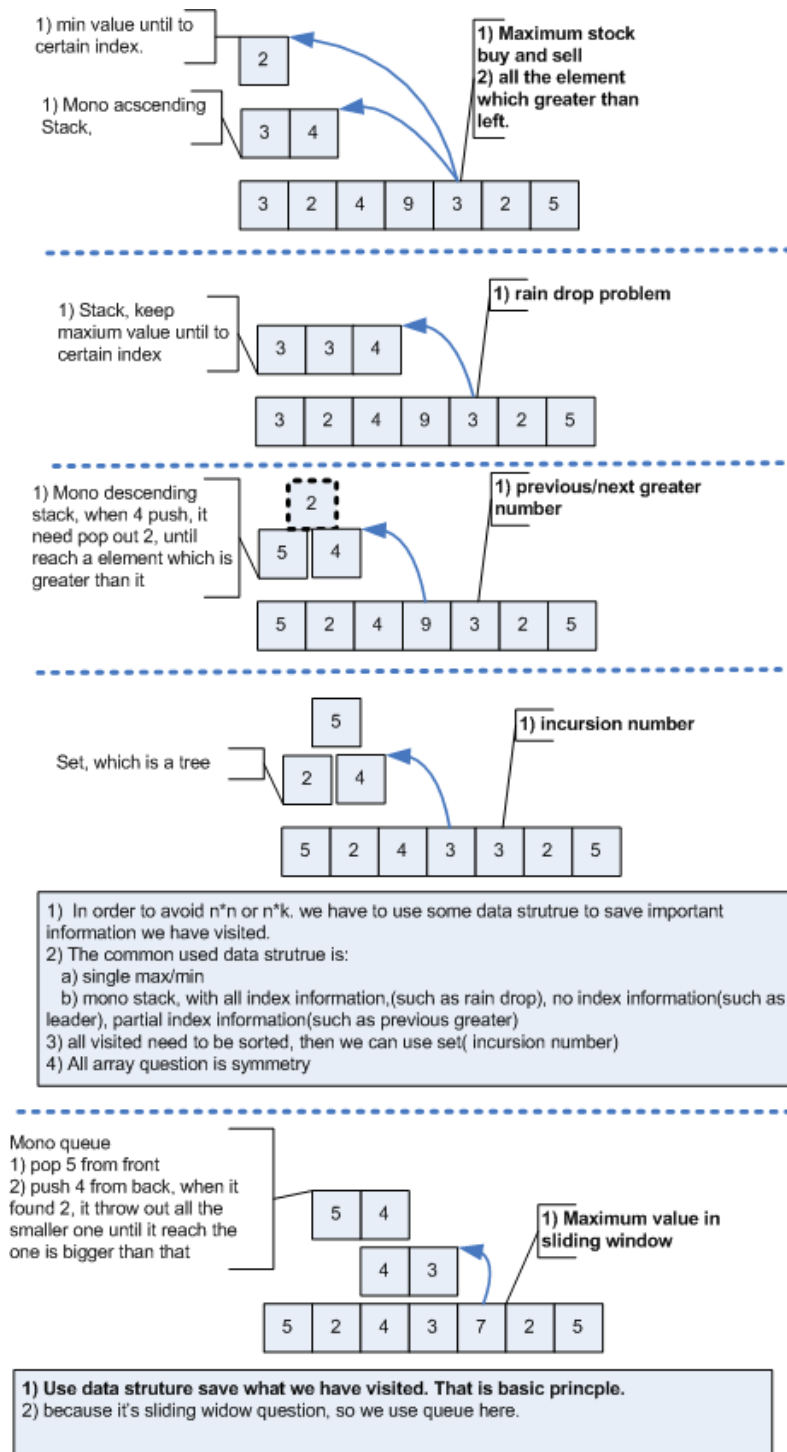
1.1.2.3 Hints for array questions.

There are a few common technology when you work on array related interview questions:

- Basic rule:** prefer to use STL algorithms and views, such as `views::split`, `views::chunk_by`, `views::slide`. Given any question, First try these existing algorithms and views.
- Sort, when you don't need index information, you can use sort + double pointers to avoid brute force. A common examples are two sum and three sum less than value.
- Get max and min value, based on heap, init a heap need $O(n)$, when you get the second largest or third largest one, you only need $O(\log(n))$. It's better than sort $O(n*\log(n))$.
- Two pointers algorithms
 - Two-Pointer Technique (from both ends towards the center): This involves starting two pointers from the two ends of a sequence and moving them towards the center. This technique is commonly used in palindrome checking or finding pairs that sum to a specific value. It's also used to implement reverse algorithms.
 - Fast and Slow Pointer Technique: This uses two pointers moving at different speeds, typically with the fast pointer moving twice as fast as the slow one. partition and remove can be implemented by this method.

3. slide windows: is similar with Fast and slow pointer, but it has some differences. The sliding window technique is often used for problems related to substring manipulation. Some leetcode example is leetcode 3, 76, 438 and 567.

- Calculation, example is missing one element, single non-duplicate element(XOR).
- index, Make use of index information, an example is leetcode 448: Given an array nums of n integers where nums[i] is in the range [1, n], return an array of all the integers in the range [1, n] that do not appear in nums. Example 1: Input: nums = [4,3,2,7,8,2,3,1] Output: [5,6]. Another famous example to use index information is <https://www.geeksforgeeks.org/counting-sort/>
- partial sum: The prefix sum technique, previously detailed, is a widely-used algorithmic approach. It is particularly suitable in scenarios where the original array remains unchanged, allowing for efficient repeated queries of cumulative sums over specific intervals. Another example is leetcode 528.
- adjacent difference: The primary use case for the difference array technique is when you need to frequently increment or decrement elements within a specific range of the original array.
- Mono stack: Leetcode 496, 503 and 739. Next Greater Element.
- Mono queue: Leetcode 239, Sliding window maximum
- Use other auxiliary DS, such as, map, set or hash_map. They can be used as count and sort. how many unordered? 0316.Remove Duplicate Letters, leetcode 1122



1.2 Linked list

1.2.1 Basic

List in STL is bidirectional list (double-linked), in C++11, you can use `std::forward_list` as single-linked list.

```

1 struct node{    //C implementation
2 int a;
3     struct node* link
4 } ;
5
6 Template <class T> //C++ implementation
7 Class ChainNode{
8     friend Chain<T>;
9     T data
10    ChainNode<T> *link;
11 }
12
13 template<class T>
14 class Chain{
15     .....
16     ChainNode<T> *head;
17 }

```

1.2.2 Operations

Insert after p node, **get an index to previous pointer**

```

1 //Consider if it's the first element.
2 Insert(int k, const T &x)
3 chainNode<T> *i = new chainNode<T> (x);
4
5 //leftside is pointer pointed direction
6 //rightside is node, but in list, all nodes only can be accessed by a pointer.
7 //insert after p node
8 i->link = p->link; //p->link is at right
9 p->link = i;       //p->link is at left now.
10 // i is inserted element, p is position.
11 //You can remember i=p p=i and fore three links.

```

Insert before p node, what should I do? **Just swap two node, then deal with next node. Don't touch next node in one atomic action. That is rule.**

```

1 1) create temp auto node, new i node.
2
3 while(p->link)
4 2) copy value from p to temp node
5 3) copy value i to p.
6 4) copy temp value to i value
7 p = p->link.

```

Delete the node after a given node.

```

1 Delete(int k, const T&x)
2 d = p->link;
3 p->link = d->link ;
4 delete d;
5 //d=p p=d and last three links

```

Delete the given node. That is a typical question. you need to remember it.

```

1 void deleteNode(Node* node_ptr){
2     // If the node to be deleted is the last node of linked list
3     if (node_ptr->next == NULL){

```

```

4   free(node_ptr); // this will simply make the node_ptr NULL.
5   return;
6 }
7
8 // if node to be deleted is the first or any node in between the linked list.
9 Node* temp = node_ptr->next;
10 node_ptr->data = temp->data;
11 node_ptr->next = temp->next;
12 free(temp);
13 }

```

Source code Psychically, still delete the next node, logically, we use next value to overwrite current node value, so we simulate "delete" in this way. remember, **We don't need while loop to reach the last node in the list.**

Three different operations:

- Normal single link list: insert and delete given a node, always insert after the given node, or erase the next node of the given node.
- Normal single link list: If you want to insert before or delete the given node, You still need to do it physically, but copy value to simulate the "insert" and "delete".
- Double link list, such as `std::list`, insert value before the given node, and delete the given node.

How to rotate a List? This question is interesting, you need to understand splice usage. splice perform 1) move like question, 2) before node of next(i,3) points to null, 3) pointer of end(li) points to i. All these things perform automatically.

```

1 list<int> li = {1, 2, 3, 4, 5};
2 auto i = begin(li);
3 //rotate(i, std::next(i, 3), end(li)); //must use std::next complexity is O(n).
4 li.splice(i, li, next(i, 3), end(li)); //better, use splice, O(1)

```

1.2.3 Interview questions

- Use dumb head for easy deleting node.

```

1 while (head != NULL && head->val == val) { // pay attention while here,
2     ListNode* tmp = head;           //5-->5-->5--> for this condition.
3     head = head->next;
4     delete tmp;
5 }
6
7 // delete non head
8 ListNode* cur = head;
9 while (cur != NULL && cur->next != NULL) {
10     if (cur->next->val == val) {
11         ListNode* tmp = cur->next;
12         cur->next = cur->next->next;
13         delete tmp;
14     } else {
15         cur = cur->next;
16     }
17 }
18 return head;
19 }

```

If we build dumb node, code is much easier now

```

1 ListNode* dummyHead = new ListNode(0); // dumb node
2 dummyHead->next = head;
3 // Set the virtual (dummy) head node to point to the head, which makes it easier
  to perform deletion operations later.
4 ListNode* cur = dummyHead;
5
6 while (cur->next != NULL) {
7     if (cur->next->val == val) {
8         ListNode* tmp = cur->next;
9         cur->next = cur->next->next;
10        delete tmp;
11    } else {
12        cur = cur->next;
13    }
14 }
15 return dummyHead->next;

```

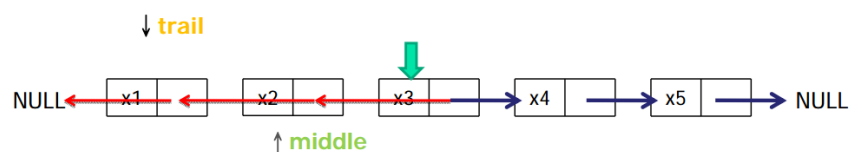
- Print the middle element of link list. When exit while, even node, fast_ptr will be NULL, odd node, fast_ptr->next will be null. Just like $n/2$, odd number will be in the middle, even number will be the first element in the second half.

```

1 void printMiddle(struct Node *head) {
2     struct Node *slow_ptr = head;
3     struct Node *fast_ptr = head;
4
5     if (head!=NULL) {
6         while (fast_ptr != NULL && fast_ptr->next != NULL) { //this statement is very
            important.
7             fast_ptr = fast_ptr->next->next;
8             slow_ptr = slow_ptr->next;
9         }
10        printf("The middle element is [%d]\n\n", slow_ptr->data);
11    }
12 }

```

- How to reverse a list There are two points: 1)while(p), 2) inside loop, do $p = p->next$, and necessary action(print out value and change pointer direction...)



```

middle = NULL;
while (lead) {
    trail = middle;
    middle = lead;
    lead = lead->link;
    middle->link = trail;
}
return middle;

```

First, initial the middle = NULL
And continue loop until lead == NULL

```

1 node* lead, middle, *tail;
2 middle = nullptr;
3 tail = head;
4 while(tail){
5     lead = middle;
6     middle = tail;

```

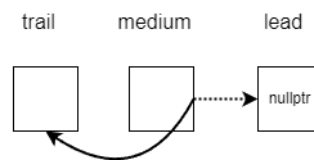


```

7   tail = tail->next;
8   middle->next = lead;
9   }
10  head = middle;

```

- Three steps to resolve this problem. this pattern can be applied on all link list problem.
 1. The end status is like below figure.
 2. so normal status is also like this(previous atomic operation end statu and next atomic operation start status are all like below), write while main logic.
 3. when you finish while main logic, according to this logic customize your beginning status. in this question, only tail = head and middle = nullptr;



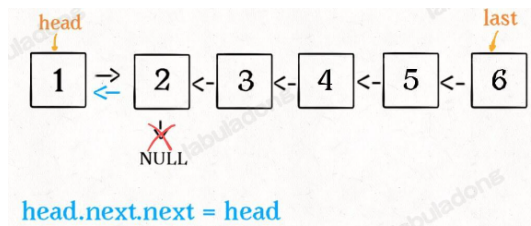
- Reverse linklist

```

1  // Definition: Given the head node of a singly linked list, reverse the list and
   // return the new head node.
2  ListNode reverse(ListNode head) {
3      if (head == null || head.next == null) {
4          return head;
5      }
6      ListNode last = reverse(head.next);
7      head.next.next = head;
8      head.next = null;
9      return last;
10 }
11 //1) in reverse, we don't use last node
12 //2) in each reverse, we just take action, don't need return something.
13 //3) just like reverse output a linklist.
14
15
16 //below is my implementation, Pay attention to the ListNode* &, we need to modify
17 //head->next, so we have to return reference. just return pointer doesn't work
   // here.
18 ListNode* ghead;
19 ListNode * & dfs(ListNode* head){
20     if(head == nullptr){
21         ghead = head;
22         return ghead;
23     }
24     if(head->next == nullptr){
25         ghead = head;
26         return ghead->next;
27     }
28     ListNode* & p = dfs(head->next);
29     cout<<head->val<<endl;
30     p=head;
31     head->next = nullptr;
32     return head->next;
33 }
34
35 ListNode* reverseList(ListNode* head) {
36     dfs(head);
37     return ghead;

```

38 }



- Reverse the first k element

```

1 ListNode successor = null; // Successor node
2 // Reverse the first n nodes starting from head and return the new head node
3 ListNode reverseN(ListNode head, int n) {
4     if (n == 1) { // Record the (n + 1)th node
5         successor = head.next;
6         return head;
7     }
8     // Starting from head.next, reverse the first n - 1 nodes
9     ListNode last = reverseN(head.next, n - 1);
10
11     head.next.next = head;
12     // Connect the reversed head node to the successor node
13     head.next = successor;
14     return last;
15 }

```

- Reverse m to n

```

1 ListNode reverseBetween(ListNode head, int m, int n) {
2     // base case
3     if (m == 1) {
4         return reverseN(head, n);
5     }
6
7     // advance to revert beginning point to trigger base case.
8     head.next = reverseBetween(head.next, m - 1, n - 1);
9     return head;
10 }

```

- Per k group to reverse

```

1 /** Reverse elements in the interval [a, b), note that it's left-closed and right-
2     open */
3 ListNode reverse(ListNode a, ListNode b) {
4     ListNode pre = null, cur = a, nxt = a;
5     // Just change the while loop's termination condition
6     while (cur != b) {
7         nxt = cur.next;
8         cur.next = pre;
9         pre = cur;
10        cur = nxt;
11    }
12    // Return the new head node after reversal
13    return pre;
14 }
15
16 ListNode reverseKGroup(ListNode head, int k) {
17     if (head == null) return null;

```

```

17 // Interval [a, b) contains k elements to be reversed
18 ListNode a = head, b = head;
19 for (int i = 0; i < k; i++) {
20     // If fewer than k nodes, no need to reverse; base case
21     if (b == null) return head;
22     b = b.next;
23 }
24 // Reverse the first k elements
25 ListNode newHead = reverse(a, b);
26 // Recursively reverse the remaining list and connect them
27 a.next = reverseKGroup(b, k);
28 return newHead;
29 }

```

- Given a singly linked list, determine if its a palindrome. This method takes $O(n)$ time and $O(1)$ extra space.
 1. Get the middle of the linked list.
 2. Reverse the second half of the linked list.
 3. Check if the first half and second half are identical.
 4. **Idea: 1) find the middle 2)reverse.** If it's odd number, you need to move the middle pointer to next position.
- Swap nodes in a linked list without swapping data. The idea it to first search x and y in given linked list. If any of them is not present, then return. While searching for x and y, keep track of current and previous pointers.

```

1 /* Split the nodes of the given list into front and back halves ,
2    and return the two lists using the reference parameters.
3    If the length is odd, the extra node should go in the front list.
4    Uses the fast/slow pointer strategy. */
5 void FrontBackSplit(struct node* source ,
6                     struct node** frontRef, struct node** backRef)
7 {
8     struct node* fast;
9     struct node* slow;
10    if (source==NULL || source->next==NULL){
11        /* length < 2 cases */
12        *frontRef = source;
13        *backRef = NULL;
14    }
15    else{
16        slow = source;
17        fast = source->next;
18
19        /* Advance 'fast' two nodes, and advance 'slow' one node */
20        while (fast != NULL){
21            fast = fast->next;
22            if (fast != NULL){
23                slow = slow->next;
24                fast = fast->next;
25            }
26        }
27
28        /* 'slow' is before the midpoint in the list , so split it in two at that point.*/
29        *frontRef = source;
30        *backRef = slow->next;
31        slow->next = NULL;
32    }
33 }

```

- Prints the given Linked List in reverse manner. (recursive)

```

1 void fun1(struct node* head){
2     if(head == NULL)
3         return;
4
5     fun1(head->next);
6     printf("%d_", head->data);
7 }

```

- Write a program function to detect loop in a linked list

```

1 This is the fastest method. Traverse linked list using two pointers. Move one
  pointer by one and other pointer by two. If these pointers meet at some node
  then there is a loop. If pointers do not meet then linked list doesn't have
  loop.

```

a circle has at last three nodes, so fast pointer each time step 2, So it will always go around inside the circle

Summary of link list:

- Just like tree, most link-list problems can be resolved by recursive.
- Most of time, In single link list, you need to keep previous node and current node.

	array	list
insert erase	O(n)	O(1)
sort	quicksort	mergesort
reverse	swap element	just manipulate pointer
rotate	easy pointer manipulate	swap element
merge		
other		

`std::list` has its own sort member function, but `std::vector` doesn't have. you can use `std::sort` algorithm. `std::list` has its own reverse, `std::vector` doesn't have, just use `std::reverse` algorithm. `std::list` and `vector` don't have their own rotate function. `std::list` has its own merge, `vector` use `std::merge` algorithm.

1.3 Matrix

1.3.1 Sparse matrices

Sparse matrix can implemented by on dimension array or linked list.

```

1 template<typename T>
2 class term{
3     int row, int col;
4     T value;
5 }
6
7 term<T> *sparseMatrix; // using one linked list

```

Define sparse matrix by using `std::list`.

```

1 using Row = list< pair<int, float> > ;
2 using Matrix = list< int, Row>;
3
4 list< pair<int, float> > row0;

```

```

5 row0.push_back({0, 1.0});
6 row0.push_back({3, 3.0});
7
8 Matrix matrix;
9 matrix.push_back({0, row0});

```

You can build a matrix class. use `m(2,3)` to access a element. And support `+`, `-`, `*`. You also can define index begin from 1. It will more reasonable for you when you use a matrix.

```

1 template<typename T>
2 class Matrix{
3     T& operator()(int row, int col){
4         ..
5     }
6 }

```

Some special square matrices

- Diagonal $i \neq j, M(i, j) = 0$ Can use one dimension array to describe it.
- Low triangular $i < j, M(i, j) = 0$ We can use one dimension array to implement lower triangular because it will save many spaces. Index can be calculated by $i * (i - 1) / 2 + j - 1$, and save it into a one dimension array.

```

1 [a11  0   0   0 ]
2 [a21 a22  0   0 ]
3 [a31 a32 a33  0 ]
4 [a41 a42 a43 a44]
5
6 a11: (1 - 1) * (1) / 2 + 1 - 1 = 0
7 a21: (2 - 1) * (2) / 2 + 1 - 1 = 1
8 a22: (2 - 1) * (2) / 2 + 2 - 1 = 2
9 a31: (3 - 1) * (3) / 2 + 1 - 1 = 3
10 a32: (3 - 1) * (3) / 2 + 2 - 1 = 4
11 .....

```

- Symmetric. Just like a low triangular matrix, can be described by one dimension array.

1.3.2 Matrices traversal methods and operation

- Row and column traversal, just use two for statement. Pay attention to the cache missing problem when you perform column traversal.
- For square matrix ($n \times n$), you can perform in place transpose. There are two points:
 1. Swap `i` and `j`.
 2. If bottom-left to top-right, then use `n-1` to subtract.

```

1 for (int i = 0; i < n; i++) {
2     for (int j = i; j < n; j++) {
3         swap(matrix[i][j], matrix[j][i]);
4     }
5 }
6
7 // Mirror-symmetric 2D matrix along the bottom-left to top-right diagonal.
8 for (int i = 0; i < n; i++) {
9     for (int j = 0; j < n - i; j++) {
10        swap(matrix[i][j], matrix[n-1-j][n-1-i]);
11    }
12 }

```

- Rotate clockwise or counter clockwise matrix, first transpose, then reverse each row, just remember this trick.
- Spiral print a matrix. Four boundary, then shrink step by step.

```

1 int m = matrix.length, n = matrix[0].length;
2 int upper_bound = 0, lower_bound = m - 1;
3 int left_bound = 0, right_bound = n - 1;
4 List<Integer> res = new LinkedList<>();
5 // When res.size() == m * n, the entire array has been traversed
6 while (res.size() < m * n) {
7     if (upper_bound <= lower_bound) {
8         // Traverse from left to right on the top
9         for (int j = left_bound; j <= right_bound; j++) {
10             res.add(matrix[upper_bound][j]);
11         }
12         upper_bound++; // Move the upper boundary down
13     }
14
15     if (left_bound <= right_bound) {
16         // Traverse from top to bottom on the right side
17         for (int i = upper_bound; i <= lower_bound; i++) {
18             res.add(matrix[i][right_bound]);
19         }
20         right_bound--; // Move the right boundary left
21     }
22
23     if (upper_bound <= lower_bound) {
24         // Traverse from right to left on the bottom
25         for (int j = right_bound; j >= left_bound; j--) {
26             res.add(matrix[lower_bound][j]);
27         }
28         lower_bound--; // Move the lower boundary up
29     }
30
31     if (left_bound <= right_bound) {
32         // Traverse from bottom to top on the left side
33         for (int i = lower_bound; i >= upper_bound; i--) {
34             res.add(matrix[i][left_bound]);
35         }
36         left_bound++; // Move the left boundary right
37     }
38 }

```

- DFS, recursive and iterator implementation.

```

1 //leetcode 200, Main function to count the number of islands
2 int numIslands(char [][] grid) {
3     int res = 0;
4     int m = grid.length, n = grid[0].length;
5     for (int i = 0; i < m; i++) { // Traverse the whole grid
6         for (int j = 0; j < n; j++) {
7             if (grid[i][j] == '1') {
8                 // Increase the island count when a new island is found.
9                 res++;
10                dfs(grid, i, j); // Then use DFS to sink the island.
11            }
12        }
13    }
14    return res;
15 }
16

```

```

17 // Starting from (i, j), turn all adjacent land into water
18 void dfs(char[][] grid, int i, int j) {
19     int m = grid.length, n = grid[0].length;
20     if (i < 0 || j < 0 || i >= m || j >= n) {
21         return; // Out of index bounds
22     }
23     if (grid[i][j] == '0') {
24         return; // Already water
25     }
26
27     grid[i][j] = '0'; // Turn (i, j) into water
28     // Sink the land above, below, left, and right
29     dfs(grid, i + 1, j);
30     dfs(grid, i, j + 1);
31     dfs(grid, i - 1, j);
32     dfs(grid, i, j - 1);
33 }

```

There are a few points you need to pay attention to when you implements dfs by iteration:

1. top and pop should stay together;
2. use pair to remember position,
3. use s!=empty();
4. check four boundary,
5. when pop, check if we visited, when push, check if we visited. (two positions checks)

```

1 void dfs(vector<vector<char>>& grid, int i, int j){
2     int row = grid.size();
3     int col = grid[0].size();
4     if(grid[i][j] == '0'){
5         return ;
6     }
7
8     stack<pair<int, int>> s;
9     s.push({i,j});
10    while(!s.empty()){
11        auto [i,j] = s.top(); //top and pop should be stay together
12        s.pop();
13        if(grid[i][j] == '1'){
14            grid[i][j] = '0';
15            if(i>0 && grid[i-1][j] == '1'){
16                s.push({i-1, j});
17            }
18            if(j<col-1 && grid[i][j+1] == '1'){
19                s.push({i, j+1});
20            }
21            if(i<row-1 && grid[i+1][j] == '1'){
22                s.push({i+1, j});
23            }
24            if(j>0 && grid[i][j-1] == '1'){
25                s.push({i, j-1});
26            }
27        }
28    }
29 }

```

- BFS. A few difference with DFS:

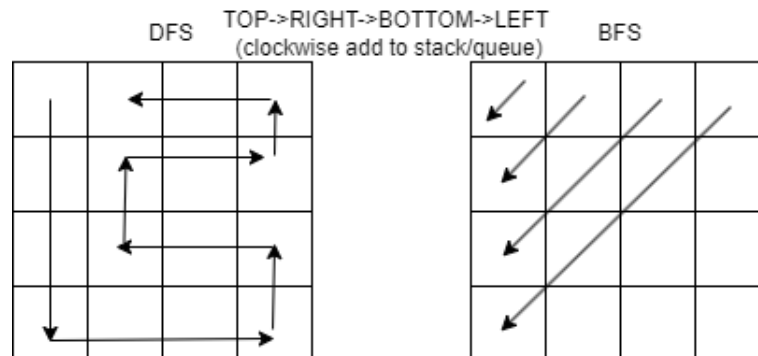
1. No need to check when pop, because when we push, we have visited, all the node in queue are visited.
2. Change stack to queue, change top to front.

3. When add the first element, mark it as visited.

```

1 void BFS(vector<vector<char>>& grid, int i, int j){
2     int row = grid.size();
3     int col = grid[0].size();
4     if(grid[i][j] == '0'){
5         return ;
6     }
7
8     queue<pair<int, int>> q;
9     q.push({i, j});
10    grid[i][j] = '0'; //the first element also need to marked as visited.
11    while(!q.empty()){
12        auto [i, j] = q.front();
13        q.pop();
14        cout<<i<<"_"<<j<<endl;
15        if(i>0 && grid[i-1][j] == '1'){
16            grid[i-1][j] = '0';
17            q.push({i-1, j});
18        }
19        if(j<col-1 && grid[i][j+1] == '1'){
20            grid[i][j+1] = '0';
21            q.push({i, j+1});
22        }
23        if(i<row-1 && grid[i+1][j] == '1'){
24            grid[i+1][j] = '0';
25            q.push({i+1, j});
26        }
27        if(j>0 && grid[i][j-1] == '1'){
28            grid[i][j-1] = '0';
29            q.push({i, j-1});
30        }
31    }
32 }

```



About the traversing direction

- Reverse traverse, from bottom to up, from left to right.

```

1 for (int i = n - 1; i >= 0; i--) {
2     for (int j = i + 1; j < n; j++) {
3         dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]); // current after left and right
                cell

```


1				
0	1			
0	0	1		
0	0	0	1	
0	0	0	0	1

- Diagonal traverse

```

1 for (int l = 2; l <= n; l++) { //l is length.
2   for (int i = 0; i <= n - l; i++) {
3     int j = l + i - 1;
4     // dp[i][j]
5   }
6 }
7   i:= 0 ..... n
8 l:=2   (0,1) (1,2) (2,3)
9 l:=3   (0,2) (1,3) (2,4)
10 l:=4   .....

```

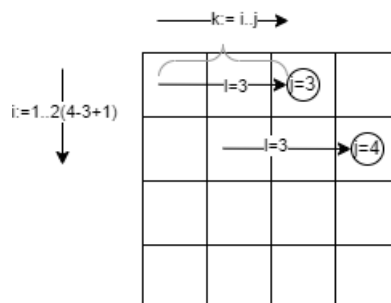
1				
0	1			
0	0	1		
0	0	0	1	
0	0	0	0	1

- The most implicated traversal, dp to calculate matrix multiplication. How to a kind of diagonal traversal matrix? this method is useful for matrix multiplication DP problem.

```

1 for (L = 2; L < n; L++)
2   for (i = 1; i < n - L + 1; i++) //n-L+1 remember n-L+1 and i+L-1.
3     j = i + L - 1 //i+L-1
4     for (k = i; k <= j - 1; k++){
5       ...

```



Fill and color difference. if you want to color fill, need to check if origin color and new color are the same, otherwise, it will lead dead loop.

matrix and spiral print all indicate a good hint:

- identify a sub task(function).

- then design this function API (what information this function need to finish it's task.)
- Once we have correct API, then loop call this function with correct parameter, this step will be easy now.

1.3.3 Interview questions

- leetcode 48. You are given an n x n 2D matrix representing an image, rotate the image by 90 degrees (clockwise). You have to rotate the image in-place, which means you have to modify the input 2D matrix directly. DO NOT allocate another 2D matrix and do the rotation.
- Two matrix multiply. `sum += v1[row][i]*v2[i][col];` is the key points to understand the whole problem.

```

1 int inner_product(int row, int col, int num, const vector<vector<int>> &v1,
2 const vector<vector<int>> &v2) {
3     int sum = 0;
4     for (int i = 0; i < num; i++) {
5         sum += v1[row][i] * v2[i][col];
6     }
7     return sum;
8 }
9 vector<vector<int>> m1 = {{1,2,3}, {4, 5, 6}}; //list initialization
10 vector<vector<int>> m2 = { {1,2}, {3,4}, {5,6}};
11
12 int m1_row = m1.size(); int m1_col = m1[0].size();
13 int m2_row = m2.size(); int m2_col = m2[0].size();
14 vector<vector<int>> result = {{0,0}, {0,0}};
15 for(int i = 0; i < m1_row; i++) {
16     for(int j = 0; j < m2_col; j++) {
17         int element = inner_product(i, j, 3, m1, m2); //call function here.
18         result[i][j] = element;
19     }
20 }

```

Source code In this questions, inner_product is key point to resolve this problem. First, get atomic operation and change it into a function. Then, design this function parameter and in the end, design outside environment to call this automatic function. This is very an important strategy.

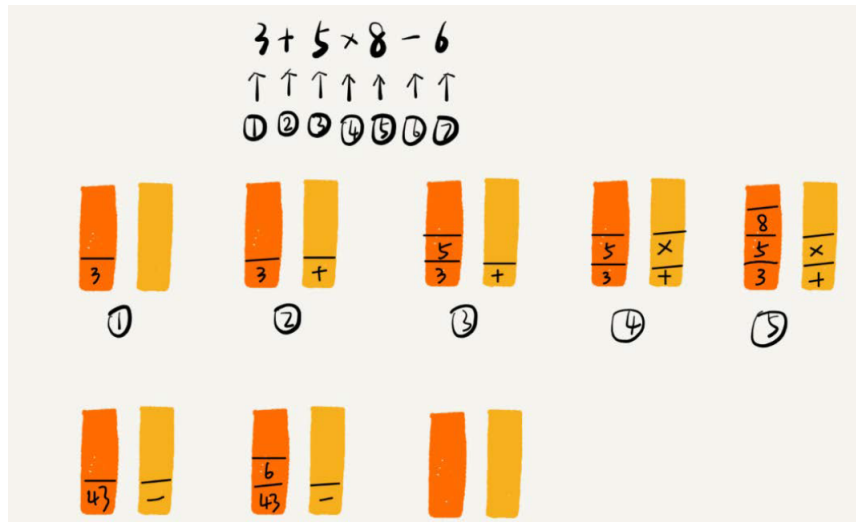
1.4 Stack

In C++ STL, `std::stack` is container adaptor, specifically designed to operate in a LIFO context. Stack is last-in first-out, and queue is first-in and first-out

1.4.1 Application of Stack

The most famous application about stack is **Recursive**. The most famous applications based on stack is: **Maze (depth first search)**. **Hanoi tower (Recursive)**. **brace Match**.

Some typical two stacks examples. One is expressions evaluation. we can build two stacks, one is for operands and the other for operators. When the operators precedence is higher than the stack top element, push the operators into the stack, otherwise, pop the operator and get two operands from operands stacks, get the result and push back to the operands stack.



Two stacks trick can also be used in browsers back and forward function. Another typical two stack usage are use two stacks to implement queue.



1.4.1.1 Offline Equivalence class

Offline equivalence class, known n and R , get all Equivalence class.

$n = 9$, $r = 11$ and relation pairs are: $(1,5), (1,6), (3,7), (1,5), (4,8), (5,2), (6,5), (4,9), (9,7), (7,8), (3,4), (6,2)$

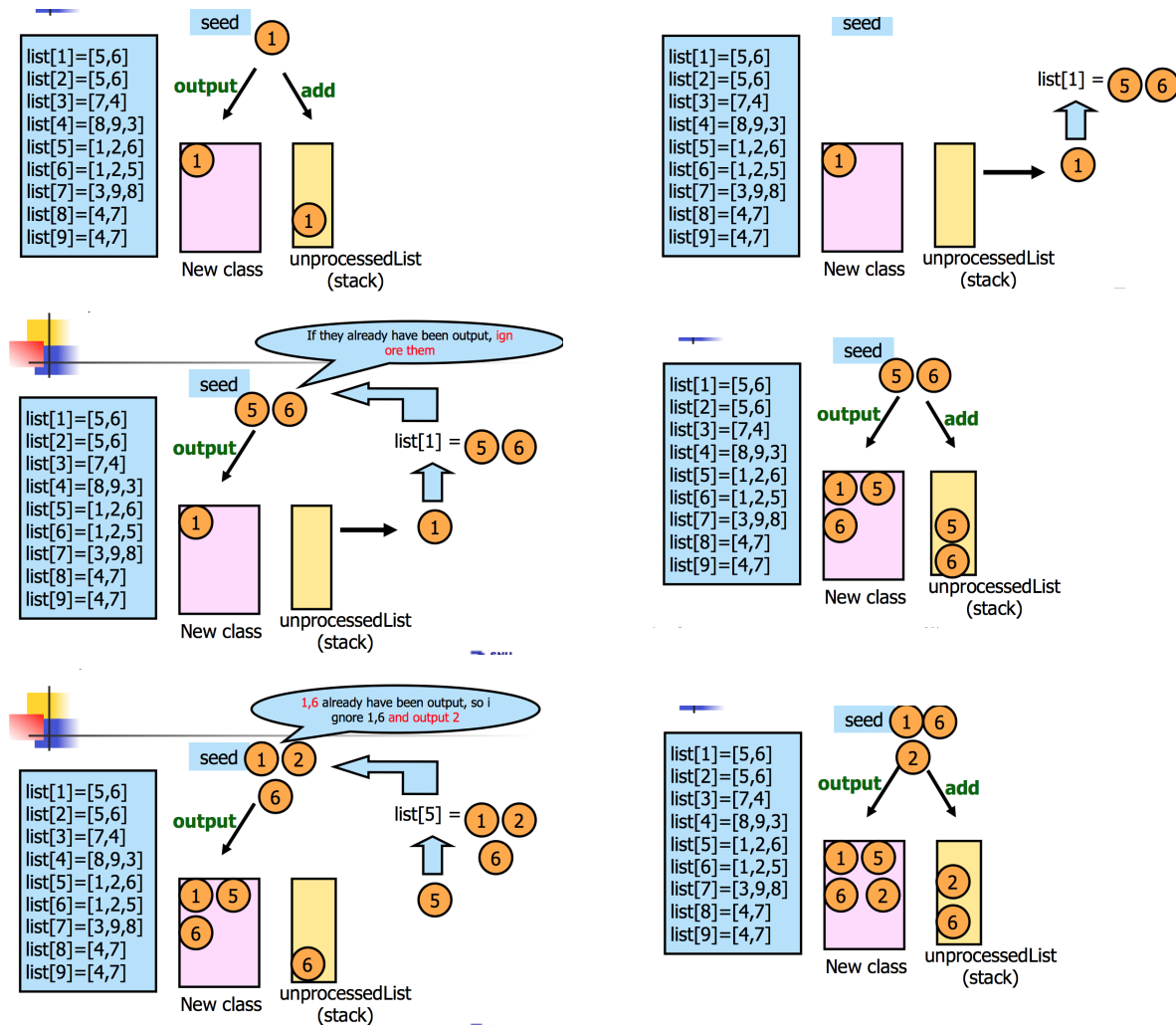
One word, **from relation pairs build a graphs(adjacency list)**, **Then use stack to perform DFS search, use equivalence class record if we have visited.**

Phase 1: Equivalence pairs (i, j) are read in and adjacency (linked) list of each object is built. Phase 2: Trace (output) the equivalence class containing object i with stack (depth-first search). Next find another object not yet output, and repeat.

```

1  for (int i = 1; i <= n; i++) // output equivalence classes
2  if (!out[i]) { // start of a new class
3      out[i] = true;
4      unprocessedList.push(new Integer(i));
5      while (!unprocessedList.empty()) {
6          // get rest of class from unprocessedList
7          int j = ((Integer) unprocessedList.pop()).intValue();
8          while (!list[j].empty()) {
9              // elements on list[j] are in the same class
10             int q = ((Integer) list[j].pop()).intValue();
11             if (!out[q]) { // q not yet output
12                 System.out.print(q + "_");
13                 out[q] = true;
14                 unprocessedList.push(new Integer(q));
15             } //end of while
16         } //end of while
17     } //end of if

```



1.4.2 Interview questions

- MinStack, all operation are constant time. Design a Data Structure SpecialStack that supports all the stack operations like push(), pop(), isEmpty(), isFull() and an additional operation getMin() which should return minimum element from the SpecialStack. All these operations of SpecialStack must have a time and space complexity of $O(1)$. Note: To implement SpecialStack, you should only use standard Stack data structure and no other data structure like arrays, lists, etc

```

1 class MinStack {
2 public:
3     void push(int val) {
4         if (stack.empty()) {
5             stack.push_back({val, val});
6         } else {
7             stack.push_back({val, min(stack.back().second, val)});
8         }
9     }
10    void pop() {
11        stack.pop_back();
12    }
13    int top() {
14        return stack.back().first;
15    }
16    int getMin() {
17        return stack.back().second;
18    }
19 }

```

```

18 }
19 private:
20 // first = val, second = min as of that insert
21 vector<pair<int, int>> stack;
22 };

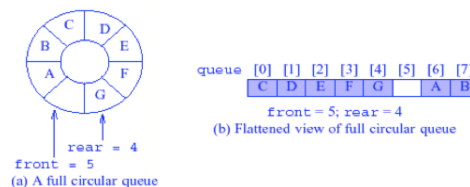
```

1.5 Queue

1.5.1 Circle Queue

You can use three different way to implement queue, array, linked list and circular array.

Circular array representation of a queue.



- Initial condition : $\text{front} = \text{rear} = 0$
- front is for pop, rear is for push (STL use back), so you can remember it easily.
- use $\text{rear}+1\% \text{length}$ and $\text{front}+1\% \text{length}$ to simulate $\text{rear}++$ and $\text{front}++$
- $\text{rear}+1\% \text{length} == \text{front}$ means full; $\text{front} == \text{rear}$ is empty.
- **front point to an empty cell. So at most it can save length-1 elements.** Otherwise, we can't distinguish empty and full.
- how to understand $(\text{rear}+1)\% \text{length}$. an array with 10 element, length is 10, the last index is 9. $(9+1)\%10 = 0$, it help use change 9 to 0 (from end array to begin array)

```

1 void push(T t){
2     if ((rear+1)%length == front){
3         throw "full" exception
4     }
5     rear = (rear+1)%length; //use (rear+1)%length simulate ++rear
6     array[rear] = t;
7 }

```

- Pop only increase front, so if $\text{front} == \text{rear}$, means front catch up rear, it means empty.

```

1 T pop(){
2     if (front == rear){
3         throw "empty" exption
4     }
5     front = (front+1)%length;
6     return array[front];
7 }

```

1.5.2 Application of queue

For BFS, path is not save in stack. So you need a new kind of data type.

```

1 struct position {
2     int x;
3     int y;
4     struct position* parent;
5 }

```

Just like DFS, before you add a position to queue, mark it first, and also marked it as visited.

1.6 Heap

1.6.1 Basic

A heap only supports three operations: initialization, insertion, and deletion of the maximum (or minimum) value. For insertion, the new element is placed at the last position and then repeatedly swapped with its parent to satisfy the heap condition. For deletion, the maximum (or minimum) value is removed, and the value from the last position is moved to the root. Then, it is compared with the left and right child nodes to decide which child to swap with in order to satisfy the heap condition. An important point during programming is the relationship between parent and child nodes, which is as follows:

```

1 Root is at index 0 in array. Relationship between child and parent are:
2 Left child of i-th node is at (2*i + 1)th index.
3 Right child of i-th node is at (2*i + 2)th index.
4 Parent of i-th node is at (i-1)/2 index.

```

Max(min) tree is each node is greater or equal to its children node. Heap just guarantees that elements on higher levels are greater (for max-heap) or smaller (for min-heap) than elements on lower levels, whereas BST guarantees order (from "left" to "right"). If you want sorted elements, go with BST.

Heap is **1)complete 2) binary 3)Max(min) tree**. Because Heap is complete binary tree. We can use an array to store it.

Insert is easier than you think, When you insert, put it on the end position, then exchange with parent.

```

1 insert(x){
2     int c = ++currentSize;
3     while(c!=1 && x>heap[c/2]){ //get its parent node and compare
4         heap[c] = heap[c/2]; //move down parent
5         // move parent down because it smaller
6         c/=2; //move up a level
7     }
8     heap[c] = x; //the right position of insert.
9     //at this time heap[c] has been moved away by previous
10    //statement heap[c] = heap[c/2]
11 }
12 //c is used to control level, c/=2 move up level
13 //heap[c/2] get its parent

```

Heap just support delete Max(Min) value. When delete, delete the first element, then put the end position element to the first position, then exchange it with either left child or right child.

```

1 T y = heap[CurrentSize--];
2 int i = 1, ci = 2;
3 //because ci will increase to get bigger one from two children.
4 //so I have to use another i to keep current level

```

```

5 while( ci<=CurrentSize){
6     if(heap[ci]<heap[ci+1]) ci++; // get the bigger child
7
8     if(y>=heap[ci]){
9         heap[i] = y; // i always empty for a new value.
10        break;
11    }
12    //i is current node
13    //ci is i biggest child node
14    heap[i] = heap[ci]; //move up child, insert is "move down parent"
15    i = ci; //move to next level
16    ci *=2;
17 }

```

Initialize a Heap: **For an array, from the middle point, loop backward until reach to beginning(0 index).** the time complexity is $O(n)$.

```

1 void heapify(int arr[], int n, int i){
2     int largest = i; // Initialize largest as root
3     int l = 2 * i + 1; // left = 2*i + 1
4     int r = 2 * i + 2; // right = 2*i + 2
5
6     if (l < n && arr[l] > arr[largest]) // If left child is larger than root
7         largest = l;
8
9     // If right child is larger than largest so far
10    if (r < n && arr[r] > arr[largest])
11        largest = r;
12
13    // If largest is not root
14    if (largest != i) {
15        swap(arr[i], arr[largest]);
16        // Recursively heapify the affected sub-tree
17        heapify(arr, n, largest);
18    }
19 }
20
21 // Function to build a Max-Heap from the given array
22 void buildHeap(int arr[], int n){
23     int startIdx = (n / 2) - 1; // Index of last non-leaf node
24
25     // Perform reverse level order traversal from last non-leaf node and heapify each
    node
26     for (int i = startIdx; i >= 0; i--) {
27         heapify(arr, n, i);
28     }
29 }

```

In STL, heap is called `priority_queue`, Default is maximum heap, if you want to build minimum heap, use `std::greater<int>` below, it's a little against intuitive.

```

1 //default compare function is less<T>
2 std::priority_queue<int, std::vector<int>, std::greater<int>> > q2;
3 for(int n : {1,8,5,6,3,4,0,9,7,2})
4     q2.push(n);
5 // 0, 1, 2, 3, 4, 5...

```

STL support a basic heap operation. just remember three functions: `std::make_heap`, `std::push_heap` and `std::pop_heap`.

```

1 vector<int> vi = {2, 4, 1, 3, 5,6,7};
2 cout<<"ddd"<<distance(end(vi),begin(vi))<<endl;
3 make_heap(begin(vi),end(vi)); // inplace modify
4
5 //after you pop_heap, you need to use v.pop_back()
6 //to make begin(vi), end(vi) is still heap.
7 v.pop_heap(begin(vi),end(vi));
8 v.pop_back();
9
10 //if you want to use push_heap, you have to
11 //call push_back first.
12 v.push_back(10);
13 v.push_heap(v.begin(vi),end(vi));
14
15 //still inplace modify
16 sort_heap(v.begin(vi),end(vi));

```

source code push_back and push_heap must appear together. At the same time, pop_heap and pop_back must appear together.

Why heap sort has $n\log(n)$? <https://stackoverflow.com/questions/54078858/why-is-the-time-complexity-of-heap-sort-onlogn>

1.6.2 Application

```

1 vector<int> v = {1, 1, 1, 1, 1, 2, 2, 3, 4, 1, 6, 6, 6, 7, 7, 7,7};
2 unordered_map<int, int> m;
3 for(auto e: v){
4     m[e]++;
5 }
6
7 auto l = [](pair<int, int> &e1, pair<int, int> &e2){
8     return e1.second > e2.second;
9 };
10
11 priority_queue< pair<int, int> , vector<pair<int, int> >, decltype(l) > pq;
12
13 for(auto &e : m){
14     if(pq.size() < 3){
15         pq.push(e);
16     }
17     else{
18         pq.push(e);
19         pq.pop();
20     }
21 }
22
23 while(!pq.empty()){
24     cout<< pq.top().first <<"_ "<< pq.top().second<<endl;
25     pq.pop();
26 }

```

You are given an integer array `nums`, an integer `k`, and an integer multiplier.

You need to perform `k` operations on `nums`. In each operation: Find the minimum value `x` in `nums`. If there are multiple occurrences of the minimum value, select the one that appears first. Replace the selected minimum value `x` with `x * multiplier`. Return an integer array denoting the final state of `nums` after performing all `k` operations.


```

1  std::vector v {2,1,3,5,6};
2      using pii = std::pair<int, int>;
3
4      int k = 5;
5      int m = 2;
6
7      std::priority_queue< pii, std::vector<pii>, std::greater<pii>> pq;
8      for(int i = 0; i < v.size(); ++i){
9          pq.push({v[i], i});
10     }
11     int i = 0;
12     while(i < k){
13         auto [f, e] = pq.top();
14         pq.pop();
15         pq.push({f*m, e});
16         ++i;
17     }
18
19     std::vector<int> result(v.size(), 0);
20     while(!pq.empty()){
21         auto [f, e] = pq.top();
22         result[e] = f;
23         pq.pop();
24     }
25
26     print("{} ", result);

```

Huffman tree use external nodes to represent a character, It assure no prefix is repeated. You can use minHeap to build huffman tree. In the minHeap, each element is child tree, and then pop twice, combine two child trees. then insert it back to minHeap. Until there is only one element in the Heap.

```

1  struct MinHeapNode { // A Huffman tree node
2      char data; // One of the input characters
3      unsigned freq; // Frequency of the character
4      struct MinHeapNode *left, *right; // Left and right child of this node
5  };
6
7  // Prints huffman codes from the root of Huffman Tree.
8  // It uses arr[] to store codes
9  void printCodes(struct MinHeapNode* root, int arr[],
10 int top) {
11     if (root->left) { // Assign 0 to left edge and recur
12         arr[top] = 0;
13         printCodes(root->left, arr, top + 1);
14     }
15
16     if (root->right) { // Assign 1 to right edge and recur
17         arr[top] = 1;
18         printCodes(root->right, arr, top + 1);
19     }
20
21     // If this is a leaf node, then it contains one of the input
22     // characters, print the character and its code from arr[]
23     if (isLeaf(root)) {
24         printf("%c: ", root->data);
25         printArr(arr, top);
26     }
27 }

```

Maximum k item in data steam. **use K minimum heap**. If you want Minimum k items (less than), use k maximum heap. A little anti intuitive, just remember it.

median number in data steam. **use one minimum heap and one maximum heap, and keep these two heaps has at least 1 number difference size**. What's difference with previous Maximum k item in data stream? median number or 10% top, answer depends on the whole size, so we need to keep all the data in the stream, Maximum k element does **NOT** depends on the size of data stream.

Another usage of heap is to use k-merge, you need to remember it.

1.7 Tree

1.7.1 Basic knowledge

1.7.1.1 Basic conception

List has a first node, every tree has a root node. Root node is a key component when you deal with most of tree problems. Pre-order, In-order and Post-order are based on middle node. **and they are all dfs algorithm**.

complete, full, perfect tree are different. Full Binary can be skewed shape.

- Full Binary Tree: A Binary Tree is full if every node has 0 or 2 children.
- Complete Binary Tree: A Binary Tree is complete Binary Tree if all levels are completely filled except possibly the last level and the last level has all keys as left as possible. (complete tree is heap, so complete tree is very important conception.)
- Perfect Binary Tree: A Binary tree is Perfect Binary Tree in which all internal nodes have two children and all leaves are at same level.

```

      18 //full binary tree
    /   \
  15     20
 /  \   /  \
40   50 30   50
/  \   /  \
30  50

```

```

      18 //complete (heap is complete tree)
    /   \
  15     30
 /  \   /  \
40   50 100  40
/  \   /
8   7  9

```

```

      18 //perfect binary tree.
    /   \
  15     30
 /  \   /  \
40   50 100  40

```

Heap is CBT(complete binary tree), not perfect binary tree.

Balance search tree includes AVL and RB tree, in order to keep it balance, you need to rotate child tree.

Tournament Tree: Tournament tree is a complete binary tree with some properties. Remove and Replay of Tournament tree gives you "sorting".

Binary Search Tree: BST is a binary tree with some properties (not necessarily CBT). In-order traversal of BST gives you "sorting" Can be skewed and unbalanced, so we need Balanced Tree.

1.7.1.2 Other property of tree

Horizontal, and diagonal traversal. horizontal distance is distance between root node. When you want to have bottom view or top view, you can use it. detail can be found here:

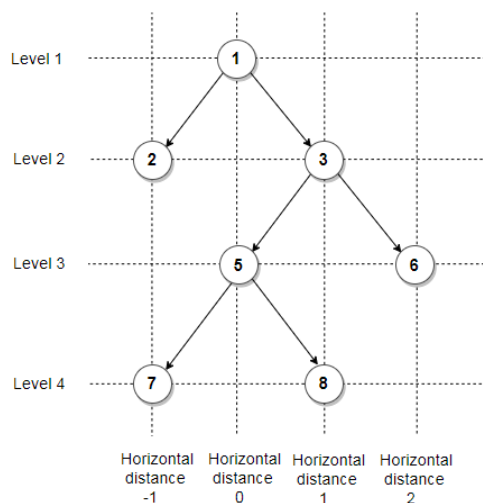
<https://www.techiedelight.com/print-bottom-view-of-binary-tree/>

```

1 map<int, pair<int, int>> map;
2 printBottom(root, 0, 0, map);
3 //-----
4 void printBottom(Node* node, int dist, int level, auto &map){
5     if (node == nullptr) { // base case: empty tree
6         return;
7     }
8
9     //if the current level is more than or equal to the maximum level seen so far
10    //for the same horizontal distance or horizontal distance is seen for the first time,
11    //update the map
12
13    if (level >= map[dist].second){
14        // update value and level for the current distance
15        map[dist] = { node->key, level };
16    }
17
18    // recur for the left subtree by decreasing horizontal distance and increasing level by
19    // 1
20    printBottom(node->left, dist - 1, level + 1, map);
21
22    //recur for the right subtree by increasing both level and horizontal distance by 1
23    printBottom(node->right, dist + 1, level + 1, map);
24 }

```

Source code Distance and level should be stack type value. Map is reference can be shared by all recursive functions. And this is very important data structure in this question.



Diagonal, Given a binary tree, calculate the sum of all nodes for each diagonal having negative slope. Assume that the left and right child of a node makes a 45-degree angle with the parent.

<https://www.techiedelight.com/find-diagonal-sum-given-binary-tree/>

```

1 // Recursive function to perform preorder traversal on the tree and
2 // fill the map with the diagonal sum of elements

```

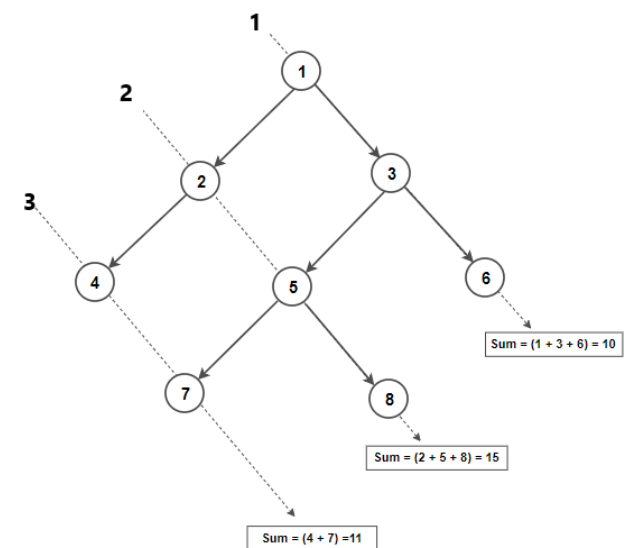
```

3 unordered_map<int, int> map;
4 void diagonalSum(Node* root, int diagonal, auto &map)
5 {
6     // base case: empty tree
7     if (root == nullptr) {
8         return;
9     }
10
11     // update the current diagonal with the node's value
12     map[diagonal] += root->data;
13
14     // recur for the left subtree by increasing diagonal by 1
15     diagonalSum(root->left, diagonal + 1, map);
16
17     // recur for the right subtree with the same diagonal
18     diagonalSum(root->right, diagonal, map);
19 }

```

line 15 diagonal is just add 1 when you go to the left tree. Don't change when you go right tree.

line 4 when you use sum of the whole tree, you must pass a reference type `auto&` map here.



1.7.2 Search Tree

Search tree is the key in each node must be **greater than all keys stored in the left sub-tree, and smaller than all keys in the right sub-tree.**

Difference between BST and hash

- So Hash Table seems to beating BST in all common operations. When should we prefer BST over Hash Tables? what are advantages? Following are some important points in favor of BSTs.
- We can get all keys in sorted order by just doing Inorder Traversal of BST. This is not a natural operation in Hash Tables and requires extra efforts.
- Doing order statistics, finding closest lower and greater elements, doing range queries are easy to do with BSTs. Like sorting, these operations are not a natural operation with Hash Tables.
- BSTs are easy to implement compared to hashing, we can easily implement our own customized BST. To implement Hashing, we generally rely on libraries provided by programming languages.

- With BSTs, all operations are guaranteed to work in $O(\log(n))$ time. But with Hashing, $O(1)$ is average time and some particular operations may be costly, especially when table resizing happens.

ascending or descending order of BST. Pay attention here, they are all in order traversal.

```

1 void ascending(BST* root){
2     if(root == NULL) return;
3     ascending(root->left);
4     std::cout<<root->data<<"_";
5     ascending(root->right);
6 }
7
8 void descending(BST* root){
9     if(root == NULL) return;
10    descending(root->right);
11    std::cout<<root->data<<"_";
12    descending(root->left);
13 }
```

1.7.3 Tree and recursion

1.7.3.1 Base case

Nearly all the tree problems can be resolved by recursive. Only focus on the current root logic. For example, kth element problem, or lowest common ancestor problem. The basic pattern is below:

```

1 TreeFun(root *...) {
2     BaseCase;
3     TreeFun(root->Left ...);
4     TreeFun(root->right ...);
5     Join logic with current node according to different problem.
6 }
```

Global variables, reference variables, local variables, and return variables: differences in their usage within tree recursion programs

95% Base Case is very simple. **Forgetting to check if the root is null. It's an important base case. In nearly all interview questions, the first statement inside recursive should be check root, then return nullptr, return 0 or return nothing.**

```

1 if(root == nullptr){
2     return ...
3 }
```

The easier part is recursive all with left child and right child. At this time, you should design the recursive interface according to specific question. For example, If it is required from you to write a function that returns a value (e.g. the number of nodes in a binary tree), you have to make sure that the function actually "returns". One of the common mistakes is just writing the recursive call without writing the word "return" before it.

The most difficult part is **Join**, You should write some logic here according to specific problem. For example, for the height of tree, we need to get max value of the left and right tree and increment it.

```

1 int height(Node t){
2     if(t == null) {
```

```

3      return -1;
4  }
5  else
6  {
7      int left = height(t->left);
8      int right = height(t->right);
9      //below are joint logic combine with current node
10     int height = 1 + max( left , right );
11     //put left and right recursive into assignment
12     return height;
13 }
14 }

```

There are two problems involving returning nodes: one is finding the lowest common ancestor, and the other is deleting a node with a certain value from a BST.

The inorder position is primarily used in BST (Binary Search Tree) scenarios, and you can consider the inorder traversal of a BST as traversing a sorted array.

The preorder position doesn't inherently have any special properties. The reason you find that many problems seem to involve writing code in the preorder position is simply because we tend to place code that isn't sensitive to preorder, inorder, or postorder in the preorder position.

You'll notice that code in the preorder position executes from top to bottom, whereas code in the postorder position executes from bottom to top. This is significant because code in the preorder position can only access data passed from the parent node via function parameters, whereas code in the postorder position can access both parameter data and data returned from the subtrees via the function's return value.

Can we obtain the solution by traversing the binary tree just once? If so, implement it using a traverse function along with external variables. This approach is called the "traversal" thinking pattern. Can we define a recursive function to derive the solution to the original problem using the answers from subproblems (subtrees)? If so, write the definition of this recursive function and fully utilize its return value. This is known as the "divide and conquer" thinking pattern. No matter which thinking pattern you use, you must consider:

- If you isolate a single node of the binary tree, what does it need to do? When should it do it (preorder/inorder/postorder)? You don't need to worry about other nodes—the recursive function will execute the same operation on all nodes.
- Differences between global variables, reference variables, local variables, and return variables when used in tree recursion programs.
 1. global variable is just reference, you need ++ and – to keep it. global can save stack size.
 2. local variable don't need ++ and –, it save in the local stack, if we want to get level, you can use it, if just int, it also has the same size as reference.
 3. **return value usually used in postorder traversal.** most of time return bool, int(size, lenh or), or tree node.
- Referring back to the previous discussion: when encountering subtree problems, the first thing to consider is setting a return value for the function and focusing on postorder traversal.
- Conversely, if you've written a recursive-in-recursive solution like in the beginning, it's likely worth considering whether it can be optimized using postorder traversal.

1.7.3.2 Traversal Order

Next big question is pre-order, in-order or post-order. **90% we use post-order.** But it also depends on practical requirement.

In order examples:

- inorder: two applications Find k'th smallest and k'th largest element in a BST:

```

1 int kthSmallest(Node* root, int *i, int k){
2     if (root == nullptr) { // base case
3         return INT_MAX;
4     }
5
6     int left = kthSmallest(root->left, i, k); // Left
7
8     if (left != INT_MAX) { // Join
9         return left;
10    }
11
12    // if the current element is k'th smallest, return its value
13    if (++*i == k) {
14        return root->data;
15    }
16
17    return kthSmallest(root->right, i, k); // right
18 }

```

source code Only In order traversal give ascending order, we have to use here. for example, kth element, you only think that the element in left tree, then return it, current root, then return it. if it's in right tree, then recursive call with root->right. return INT_MAX can be thought as a flag. if you want to look for a integer, you can use INT_MAX to denote a flag. I should be shared by all recursive call, so we use pointer or reference here.

- Print an expression tree

```

1      *
2     /\
3    +  c
4   /\
5  a  b
6
7 void inorder(Node* root)
8 {
9     if (root == nullptr) {
10         return;
11     }
12
13     // if the current token is an operator, print open parenthesis
14     if (isOperator(root->data)) {
15         cout << "(";
16     }
17
18     inorder(root->left);
19     cout << root->data;
20     inorder(root->right);
21
22     // if the current token is an operator, print close parenthesis
23     if (isOperator(root->data)) {
24         cout << ")";
25     }

```

```
26 }
```

Pre order examples:

- Print left view of a binary tree

```
1 // Recursive function to print the left view of a given binary tree
2 void leftView(Node* root, int level, int &last_level)
3 {
4     // base case: empty tree
5     if (root == nullptr) {
6         return;
7     }
8
9     // if the current node is the first node of the current level
10    if (last_level < level)
11    {
12        // print the node's data
13        cout << root->key << "_";
14
15        // update the last level to the current level
16        last_level = level;
17    }
18
19    // recur for the left and right subtree by increasing the level by 1
20    leftView(root->left, level + 1, last_level);
21    leftView(root->right, level + 1, last_level);
22 }
```

Post order examples:

- Find maximum sum root to leaf path in a binary tree

```
1 // Function to calculate the maximum root-to-leaf sum in a binary tree
2 int getRootToLeafSum(Node* root){
3     // base case: tree is empty
4     if (root == nullptr) {
5         return 0;
6     }
7
8     // calculate the maximum node-to-leaf sum for the left child
9     int left = getRootToLeafSum(root->left);
10
11    // calculate the maximum node-to-leaf sum for the right child
12    int right = getRootToLeafSum(root->right);
13
14    // consider the maximum sum child
15    return (left > right? left : right) + root->data;
16 }
```

- combined: find LCA

```
1 bool findLCA(Node* root, Node* &lca, Node* x, Node* y)
2 {
3     // base case 1: return false if the tree is empty
4     if (root == nullptr) {
5         return false;
6     }
7
8     // base case 2: return true if either 'x' or 'y' is found
9     if (root == x || root == y){ //
10        // set lca to the current node
```



```

11     lca = root;
12     return true;
13 }
14
15 // recursively check if 'x' or 'y' exists in the left subtree
16 bool left = findLCA(root->left, lca, x, y);
17
18 // recursively check if 'x' or 'y' exists in the right subtree
19 bool right = findLCA(root->right, lca, x, y);
20
21 // if 'x' is found in one subtree and 'y' is found in the other subtree,
22 // update lca to the current node
23 if (left && right) {
24     lca = root;
25 }
26
27 // return true if 'x' or 'y' is found in either left or right subtree
28 return left || right;
29 }

```

Only focus on the current root logic. If left and right, then return root. otherwise just return left or right. so simple!!

Level order examples:

```

1 // Function to print spiral order traversal of a given binary tree
2 void spiralOrderTraversal(Node* root){
3     if (root == nullptr) {
4         return;
5     }
6
7     // create an empty double-ended queue and enqueue the root node
8     list<Node*> deque; // or use deque
9     deque.push_front(root);
10
11     // 'flag' is used to differentiate between odd or even level
12     bool flag = false;
13
14     while (!deque.empty()) { // loop till deque is empty
15         // calculate the total number of nodes at the current level
16         int nodeCount = deque.size();
17
18         if (flag){ // print left to right
19             // process each node of the current level and enqueue their
20             // non-empty left and right child to deque
21             while (nodeCount) {
22                 Node* curr = deque.front(); // pop from the front if 'flag' is true
23                 deque.pop_front();
24
25                 cout << curr->key << "_";
26
27                 // it is important to push the left child into the back,
28                 // followed by the right child
29                 if (curr->left != nullptr) {
30                     deque.push_back(curr->left);
31                 }
32                 if (curr->right != nullptr) {
33                     deque.push_back(curr->right);
34                 }
35                 nodeCount--;
36             }

```

```

37 }
38 else { // print right to left
39     // process each node of the current level and enqueue their
40     // non-empty right and left child
41     while (nodeCount){
42         Node* curr = deque.back(); // it is important to pop from the back
43         deque.pop_back();
44
45         cout << curr->key << "_"; // print front node
46
47         // it is important to push the right child at the front,
48         // followed by the left child
49         if (curr->right != nullptr) {
50             deque.push_front(curr->right);
51         }
52         if (curr->left != nullptr) {
53             deque.push_front(curr->left);
54         }
55         nodeCount--;
56     }
57 }
58 flag = !flag; // flip the flag for the next level
59 cout << endl;
60 }
61 }

```

Source code use BFS, so use queue, but we also need spiral, so have to use deque which support pop_front and pop_back. In order to print each level, use below source code pattern.

```

1 while (!deque.empty()) {
2     // calculate the total number of nodes at the current level
3     int nodeCount = deque.size();
4     while (nodeCount) {
5         nodeCount--;

```

Traversal order summary

- Most of time, traversal order can be decided by the question itself very quickly.
- **Any time when you find Maximum or ancestor, this is a strong indication which we can use postorder.**
- You can combined use preorder visit and postorder logic combine together, such as LCA.

1.7.3.3 Interface

Next questions is interface design. (all auxiliary parameter) Find k'th smallest and k'th largest element in a BST: `int *i` or `int& i` is shared variable among all recursive. which record how many nodes we have visited so far. Return value is just your question answer. that is very easy to understand.

If you change inside function, sometimes, you need to change it back when you return.

```

1 // Recursive function to find paths from the root node to every leaf node
2 void printRootToLeafPaths(Node* node, vector<int> &path){
3     if (node == nullptr) { // base case
4         return;
5     }
6
7     // include the current node to the path
8     path.push_back(node->data);

```

```

9
10 if (isLeaf(node)){ // if a leaf node is found, print the path
11     for (int data: path) {
12         cout << data << "_";
13     }
14     cout << endl;
15 }
16
17 // recur for the left and right subtree
18 printRootToleafPaths(node->left , path);
19 printRootToleafPaths(node->right , path);
20
21 // backtrack: remove the current node after the left , and right subtree are done
22 path.pop_back();
23 }

```

Another simple is use local stack variable. If you don't use reference, you don't need to pop_back. That is not memory friendly solution, but you can avoid pop_back function call.

```

1 // Recursive function to find paths from the root node to every leaf node
2 void printRootToleafPaths(Node* node, vector<int> path){
3     if (node == nullptr) {
4         return;
5     }
6
7     path.push_back(node->data);
8
9     // if a leaf node is found, print the path
10    if (isLeaf(node)) {
11        for (int data: path) {
12            cout << data << "_";
13        }
14        cout << endl;
15    }
16
17    // recur for the left and right subtree
18    printRootToleafPaths(node->left , path);
19    printRootToleafPaths(node->right , path);
20
21 }

```

Information flow:

- return value from child to parent, (only return one value, so need some calculation inside from both left and right children)
- pass from parent to child, parameter to remember auto value(level) ,
- shared between children and parent. and pass a pointer parameter(max_level) to remember static, which all recursive

1.7.3.4 Span child tree

Determine whether the given binary tree nodes are cousins of each other. Two nodes of a binary tree are cousins if they have the same depth, but have different parents. **traversal all, use global variable to store the good result when you traversal all.**

```

1 // Perform inorder traversal on a given binary tree and update 'x' and 'y'
2 void inorder(Node* root, Node* parent, int level, NodeInfo &x, NodeInfo &y)
3 {

```

```

4 // base case: tree is empty
5 if (root == nullptr) {
6     return;
7 }
8
9 // traverse left subtree
10 inorder(root->left, root, level + 1, x, y);
11
12 // if the first element is found, save its level and parent node
13 if (root->key == x.key) {
14     x.level = level;
15     x.parent = parent;
16 }
17
18 // if the second element is found, save its level and parent node
19 if (root->key == y.key) {
20     y.level = level;
21     y.parent = parent;
22 }
23
24 // traverse right subtree
25 inorder(root->right, root, level + 1, x, y);
26 }

```

Source code Use two global valure as reference, then search the whole tree.

1.7.3.5 Two trees

Recursive can be used to deal with two trees at the same time.

```

1 int identicalTrees(struct node* a, struct node* b){
2     /*1. both empty */
3     if (a==NULL && b==NULL)
4         return true;
5
6     /* 2. both non-empty -> compare them */
7     if (a!=NULL && b!=NULL) {
8         return(
9             a->data == b->data &&
10            identicalTrees(a->left, b->left) &&
11            identicalTrees(a->right, b->right)
12        );
13    }
14
15    /* 3. one empty, one not -> false */
16    return false;
17 }

```

Some problems can be changed to two tree problems. such as if mirror.

```

1 // Function to check if subtree rooted at 'X' and 'Y' mirror each other
2 bool isSymmetric(Node* X, Node* Y)
3 {
4     // base case: if both trees are empty
5     if (X == nullptr && Y == nullptr) {
6         return true;
7     }
8
9     // return true if

```

```

10 // 1. Both trees are non-empty, and
11 // 2. The left subtree is the mirror of the right subtree, and
12 // 3. The right subtree is the mirror of the left subtree
13 return (X != nullptr && Y != nullptr) &&
14 isSymmetric(X->left, Y->right) &&
15 isSymmetric(X->right, Y->left);
16 }

```

Fill any nodes next pointer:

```

1 Node connect(Node root) {
2     if (root == null) return null;
3     connectTwoNode(root.left, root.right);
4     return root;
5 }
6
7 void connectTwoNode(Node node1, Node node2) {
8     if (node1 == null || node2 == null) {
9         return;
10    }
11
12    node1.next = node2;
13    // sibling
14    connectTwoNode(node1.left, node1.right);
15    connectTwoNode(node2.left, node2.right);
16    // cousin
17    connectTwoNode(node1.right, node2.left);
18 }

```

Source code Any time you need to access cousin, you should use two nodes. Three branch recursive, That is very different with others.

1.7.3.6 Traps

You can check if one node is leaf node. The basic idea is just like to find if a value exist in the tree. But you can't ignore base case.

```

1      1      //must have base case to deal with this tree
2     / \
3    null 2
4
5
6      1      //if (root->left == null && root->right == null) return;
7     / \    //can deal with this context. but can't deal with previous tree.
8    null null
9
10 int countLeaves(Node* t){ //an example of isLeaf
11     if(t == null)
12         return 0;
13
14     if(isLeaf(t)) {
15         return 1;
16     }
17     else
18         return countLeaves(t.left) + countLeaves(t.right);
19 }

```

- **Accessing the children values.** You **SHOULD NOT** do that unless if want to judge if it's leaf node. The child passed to the recursive call is treated as the new root.

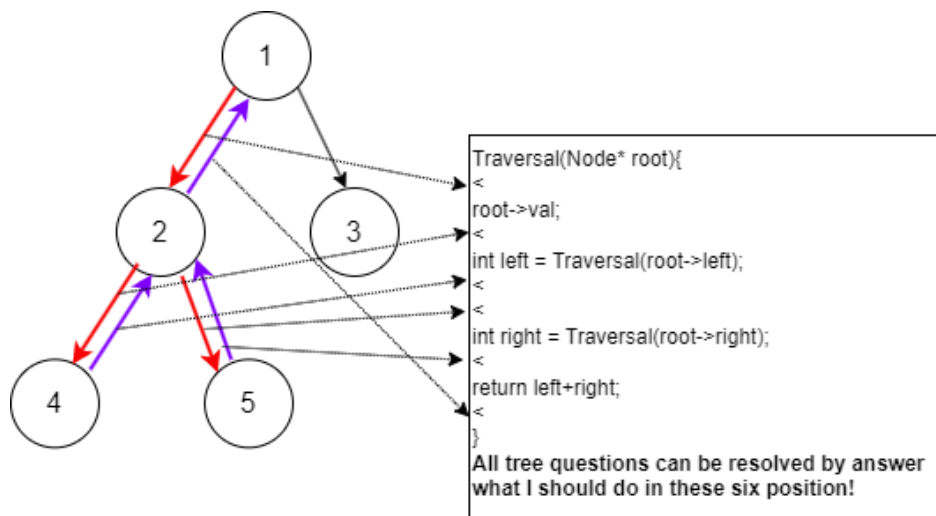
1.7.3.7 Conclusion

According to question, if it will return a value, such as leaf count, height. It has to have a function return value. What value should we return? Usually, this is related to question. For example, kth element should return integer, LCA should return a pointer. At the same time, integer and pointer also can act as flag value.

The first statement always should be base case.

```
1 if(root == nullptr)
2     return [value]
```

- decide pre-order, in-order, and post-order.
- For some easy problems, you have finished the problems. Then for some difficult problems, you need to some logic comparison or calculation. such as Maximum path sum in a binary tree. or max height tree.
- Logic inside the function, an good example is LCA: there are four different possible. 1) The root node is x or y, 2) the x or y only in left tree, 3) the x or y only in the right tree, 4) the x and y in the two separate tree. That is all the possibles, then write all logic to deal previous 4 possibles.
- That is ALL!, never try to recursive call in your head.



1.7.4 Interview questions

1.7.4.1 Print part of tree

Usually use pre-order, then use some condition to suppress cout. At the same time, also need to pass some condition as parameter, (left view, level) return some flag(path include a node)

Print left view When you return, the level will be local stack value, each value has stack scope.

```

1 // Recursive function to print the left view of a given binary tree
2 void leftView(Node* root, int level, int &last_level)
3 {
4     if (root == nullptr) { // base case: empty tree
5         return;
6     }
7
8     // if the current node is the first node of the current level
9     if (last_level < level){
10         cout << root->key << "_"; // print the node's data
11
12         // update the last level to the current level
13         last_level = level;
14     }
15
16     // recur for the left and right subtree by increasing the level by 1
17     leftView(root->left, level + 1, last_level);
18     leftView(root->right, level + 1, last_level);
19 }

```

Print the certain level

```

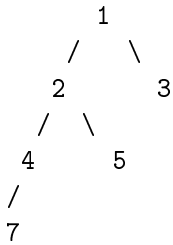
1 // Function to print all nodes of a given level from left to right
2 bool printLevelLeftToRight(Node* root, int level)
3 {
4     if (root == nullptr) {
5         return false;
6     }
7
8     if (level == 1)
9     {
10         cout << root->key << "_";
11         return true; //return, don't go deeper to save time complexity.
12     }
13
14     // process left child before the right child
15     bool left = printLevelLeftToRight(root->left, level - 1);
16     bool right = printLevelLeftToRight(root->right, level - 1);
17
18     return left || right; //that is why we need to return bool.
19 }
20
21 // Function to print level order traversal of a given binary tree
22 void levelOrderTraversal(Node* root)
23 {
24     // start from level 1 till the height of the tree
25     int level = 1;
26
27     // run till printLevel() returns false
28     while (printLevel(root, level)) {
29         level++;
30     }
31 }

```

Print path include a node. If target is present in tree, then prints all the ancestors and returns true, otherwise returns false.

- The trick in this question is that it doesn't tell us the function should return bool, you should guess it out.

- preorder and postorder should be combined. when search key, use preorder, when print all ancestor, use post order.
- this combined order model also used in LCA.



key is 7, then your function should print 4, 2 and 1.

```

1 bool printAncestors(struct node *root, int target){
2     /* base cases */
3     if (root == NULL)
4         return false;
5     if (root->data == target)
6         return true;
7
8     /* If target is present in either left or right subtree of this node, then print this
9        node */
9     if ( printAncestors(root->left , target) ||
10    printAncestors(root->right , target) ){
11         cout << root->data << "_";
12         return true;
13     }
14     /* Else return false */
15     return false;
16 }

```

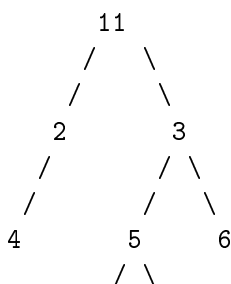
Source cde: Idea: just like max height example, return bool instead value, logic || instead max. the basic idea are quite same.

Source code: use return bool value to suppress cout

Source code: focus local root principal also can be used here. for local root, there are only three possibles. current == target, target in left tree, target in right tree.

1.7.4.2 Build tree

The problem of constructing a binary tree usually follows the "divide and conquer" approach: constructing the entire tree = root node + constructing the left subtree + constructing the right subtree. serialize and deserialize a tree




```

      /   \
     7     8
//11,2,4,...,3,5,7,...,8,...,6,...

```

```

1 string serialize(Node* root) { //serialize the root
2     if (root == NULL) {
3         return "."; //a '.' character represents a null node
4     }
5
6     string serialized = "";
7
8     serialized += to_string(root->key);
9     //add the current node to the serialized string then traverser its left and right
    children respectively
10
11    serialized += "," + serialize(root->left); //add the seperator after each node
12    serialized += "," + serialize(root->right);
13
14    return serialized;
15 }
16
17 Node* deserializeUtil(vector<string>& serialized, int& i) {
18
19     if (i == serialized.size()) { //if the whole array is exhausted
20         return NULL;
21     }
22
23     string val = serialized[i++];
24
25     if (val == ".") { //if we encounter the '.' character => a null pointer
26         return NULL;
27     }
28     //build the tree using the serialized array
29     Node* node = new Node{ stoi(val) };
30
31     node->left = deserializeUtil(serialized, i);
32     node->right = deserializeUtil(serialized, i);
33
34     return node;
35 }
36

```

Source code: with nullptr(.) in string, we can use in-order unique define a tree.

Construct a binary tree from inorder and preorder traversal.

```

1 Node* construct(int start, int end, vector<int> const& preorder,
2 int& pIndex, unordered_map<int, int>& map, int level){
3
4     if (start > end) { // base case
5         return nullptr;
6     }
7
8     // The next element in 'preorder[]' will be the root node of subtree
9     // formed by sequence represented by 'inorder[start, end]'
10    Node* root = newNode(preorder[pIndex++]);
11
12    // get the root node index in sequence 'inorder[]' to determine the
13    // left and right subtree boundary
14    int index = map[root->key];

```

```

15 // recursively construct the left subtree
16 for (int i = 0; i < level; ++i)
17     cout << "_";
18     cout << root->key << "_" << start << "_" << index << "_" << end << endl;
19
20
21     root->left = construct(start, index - 1, preorder, pIndex, map, level+1);
22
23 // recursively construct the right subtree
24     root->right = construct(index + 1, end, preorder, pIndex, map, level+1);
25
26 // return current node
27     return root;
28 }
29
30 // Construct a binary tree from inorder and preorder traversals.
31 // This function assumes that the input is valid
32 // i.e., given inorder and preorder sequence forms a binary tree
33 Node* construct(vector<int> const& inorder, vector<int> const& preorder){
34     // get the total number of nodes in the tree
35     int n = inorder.size();
36
37     // create a map to efficiently find the index of any element in a given inorder
38     // sequence
39     unordered_map<int, int> map;
40     for (int i = 0; i < n; i++) {
41         map[inorder[i]] = i;
42     }
43
44     // 'pIndex' stores the index of the next unprocessed node in preorder;
45     // start with the root node (present at 0th index)
46     int pIndex = 0;
47
48     return construct(0, n - 1, preorder, pIndex, map, 0);
49 }
50 //output
51
52      1
53     / \
54    2   3
55   / \ / \
56  4  5 6
57   / \
58  7  8
59
60
61
62 1 0 2 7
63 2 0 1 1
64 4 0 0 0
65 3 3 6 7
66 5 3 4 5
67 7 3 3 3
68 8 5 5 5
69 6 7 7 7

```

Source code: Iterate one time each time in pre-order. Build map to store in-order index, divide it into left tree and right tree. **just like quick_sort**, pick an element from pre-order, then divide left and right according to index value in map

How many BST?

```

1  /* Main function , how many different BST tree given a number */
2  int numTrees(int n) {
3      // Calculate the number of BSTs that can be formed from the closed interval [1, n]
4      return count(1, n);
5  }
6
7  /* Calculate the number of BSTs that can be formed from the closed interval [lo, hi] */
8  int count(int lo, int hi) {
9      if (lo > hi) return 1; // base case
10
11     int res = 0;
12     for (int i = lo; i <= hi; i++) {
13         // Use the value of i as the root node
14         int left = count(lo, i - 1);
15         int right = count(i + 1, hi);
16         // The total number of BSTs is the product of combinations of the left and right subtrees
17         res += left * right;
18     }
19     return res;
20 }

```

Build BST

```

1  /* Main function */
2  public List<TreeNode> generateTrees(int n) {
3      if (n == 0) return new LinkedList<>();
4      // Construct BSTs that can be formed from the closed interval [1, n]
5      return build(1, n);
6  }
7
8  /* Construct BSTs that can be formed from the closed interval [lo, hi] */
9  List<TreeNode> build(int lo, int hi) {
10     List<TreeNode> res = new LinkedList<>();
11     // base case
12     if (lo > hi) {
13         res.add(null);
14         return res;
15     }
16
17     // 1. Enumerate all possible root nodes.
18     for (int i = lo; i <= hi; i++) {
19         // 2. Recursively construct all valid BSTs for the left and right subtrees.
20         List<TreeNode> leftTree = build(lo, i - 1);
21         List<TreeNode> rightTree = build(i + 1, hi);
22         // 3. Enumerate all combinations of left and right subtrees for the root node.
23         for (TreeNode left : leftTree) {
24             for (TreeNode right : rightTree) {
25                 // Use i as the value of the root node
26                 TreeNode root = new TreeNode(i);
27                 root.left = left;
28                 root.right = right;
29                 res.add(root);
30             }
31         }
32     }
33
34     return res;
35 }

```

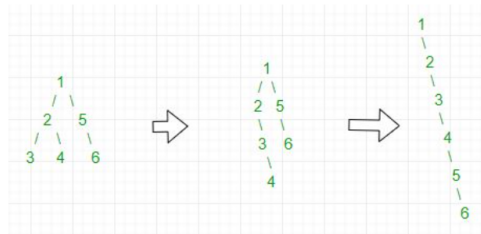
1.7.4.3 Change tree

Flatten tree. In order and post order are both OK, but preorder is not good.

```

1 void flatten(struct Node* root){
2     if (root == NULL || root->left == NULL &&
3         root->right == NULL) {
4         return;
5     }
6
7     // if root->left exists then we have to make it root->right
8     if (root->left != NULL) {
9         flatten(root->left); // move left recursively
10
11        // store the node root->right
12        struct Node* tmpRight = root->right;
13        root->right = root->left;
14        root->left = NULL;
15
16        // find the position to insert
17        // the stored value
18        struct Node* t = root->right;
19        while (t->right != NULL) {
20            t = t->right;
21        }
22
23        t->right = tmpRight; // insert the stored value
24    }
25
26    // now call the same function for root->right
27    flatten(root->right);
28 }

```



Truncate a binary tree to remove nodes that lie on a path having a sum less than k

- Only node on a path, should use postorder.
- Only print(or not print) or keep (or delete), should return bool
- calculate sum, should input a parameter sum
- **When delete leaf, parent will become leaf, That is why we have to use postorder**
- You have to use reference to pointer.

```

1 void trunc(Node* &curr, int k, int sum){
2     if (curr == nullptr) { // base case: empty tree
3         return;
4     }
5
6     // update sum of nodes in the path from the root node to the current node
7     sum = sum + (curr->data);
8 }

```

```

9  // Recursively truncate left and right subtrees
10 trunc(curr->left , k, sum);
11 trunc(curr->right , k, sum);
12
13 // Since we are doing postorder traversal, the subtree rooted at the current
14 // node may be already truncated, and the current node is a leaf
15 // if the current node is a leaf node and its path from the root node has a sum
16 // less than the required sum, remove it
17 if (sum < k && isLeaf(curr)){
18
19     delete(curr); // free the memory allocated to the current node
20
21     curr = nullptr; // set current node to null (node is passed by reference)
22 }
23 };

```

These two methods have common points: we both modify tree from bottom to top. When we modify current node, we assume that all the child tree has finished. One use in-order, other use post-order. (Why? think about later.)

1.7.4.4 Other

Given the root of a binary tree, return all root-to-leaf paths in any order. (you need root == nullptr, also need to judge if it's leaf node). That is an interesting example.

```

1  void pathTree(TreeNode* root , string path , vector<string> & result){
2      if(root == nullptr){
3          return;
4      }
5      if(path == ""){
6          path = to_string(root->val);
7      }
8      else{
9          path+="->";
10         path+=to_string(root->val);
11     }
12     if(root->left == nullptr && root->right == nullptr){
13         result.push_back(path);
14         return;
15     }
16
17     pathTree(root->left , path , result);
18     pathTree(root->right , path , result);
19 }
20
21 vector<string> binaryTreePaths(TreeNode* root) {
22     string s = "";
23     vector<string> result;
24     pathTree(root , s , result);
25     return result;
26 }

```

In order without recursive,

```

1  // An iterative process to print preorder traversal of Binary tree
2  void iterativePreorder(node *root){
3      if (root == NULL) // Base Case
4          return;
5

```

```

6 // Create an empty stack and push root to it
7 stack<node *> nodeStack;
8 nodeStack.push(root);
9
10 /* Pop all items one by one. Do following for every popped item
11    a) print it
12    b) push its right child
13    c) push its left child
14    Note that right child is pushed first so that left is processed first */
15 while (nodeStack.empty() == false) {
16     // Pop the top item from stack and print it
17     struct node *node = nodeStack.top();
18     printf ("%d_", node->data);
19     nodeStack.pop();
20
21     // Push right and left children of the popped node to stack
22     if (node->right)
23         nodeStack.push(node->right);
24     if (node->left)
25         nodeStack.push(node->left);
26 }
27 }

```

Idea: A common implementation of DFS, without check two unvisited condition. Because tree is a 1) minimally connected graph and having only one path between any two vertices. 2) no loop.

Determine whether a binary tree is a subtree of another binary tree. both inorder and preorder together identify a tree uniquely.

Can be changed into a binary tree problem!

- print all subsequence, select current or don't select
- stair case, walk one or two
- house robbery
- All possible combine in array.

```

1 Input:  digits[] = { 1, 2, 2, 1 }
2 {1, 2, 2, 1}
3 {1, 2, 21}
4 {1, 22, 1}
5 {12, 2, 1}
6 {12, 21}

```

1.8 Graph

1.8.1 Basic

traverse all nodes is different with path from source node to target node.

- traverse use visited[] to avoid loop.
- if it's DAG, you don't need to use visited. for example, we don't use visited in tree.
- For path, that should be DAG, in this way, we don't need use visited either. (if there is a loop, can't print a valid path)
- for path, if you want to use visited, it will select only one path pass a visited node, but skip others(maybe it's not what you want.)

- DFS is different with backtrack, one care about node(traverse), The bt care about different decision(select). Although they are looks like tree, but different, **in tree and graph, node are really exist, but in bt, node are not really exist, they generate by a selection, disappear by revoke**

```

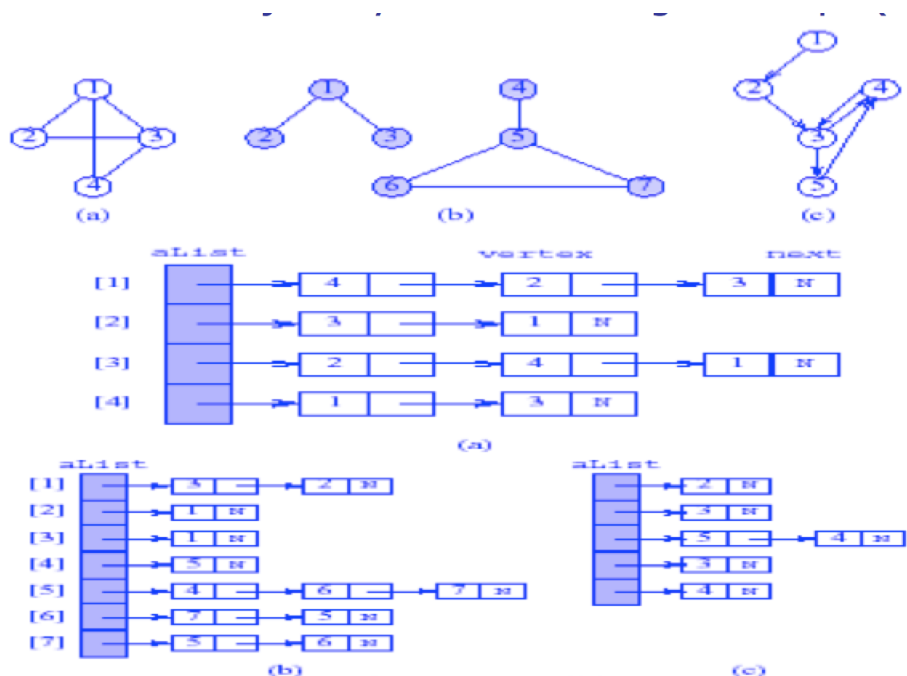
1  dfs () {
2      path.push
3      for (...) {
4          dfs
5      }
6      path.pop
7  }
8  }
9
10 bt () {
11
12     for (...) {
13         select.push() //select
14         bt();
15         select.pop() //revoke.
16     }
17 }
18 }

```

A Tree is just a restricted form of a Graph. They fit with in the category of Directed Acyclic Graphs (or a DAG). So Trees are DAGs with the restriction that a child can only have one parent.

Graphs are generally search breath first or depth first. The same applies to Tree.

Basic represent way: **adjacency list** and **adjacency matrix**:



You want to mirror the problem using a tree-like structure: For this we have boost graph library. The BGL currently provides two graph classes and an edge list adaptor: `adjacency_list`, `adjacency_matrix` and `edge_list`. The `adjacency_list` class is the general purpose "swiss army knife" of graph classes.

1.8.2 DFS and BFS

1.8.2.1 DFS

DFS has three versions 1) recursive 2) iterate. 3) uniform version.

- Recursive version: in this dfs, **you only need to check one place if unvisited.** . Also you should select dfs version first.

```

1 DFS(current){
2   visit current;
3   for all(next to current){
4     if(unvisited(next)){
5       DFS(next);
6     }
7   }
8 }
9
10
11 //Another better version is to put "is visited" into the dfs beginning.
12 //
13 DFS(current){
14   if(current is visited)
15     return;
16   visit current;
17   for all(next to current){
18
19     DFS(next);
20
21   }
22 }
```

- iterate

```

1 DFS(start){
2   S.push(start)
3   while(!S.empty()){
4     current = top();
5     visit current;
6     If (current has one unvisited next){
7       stack.push(next);
8     }
9     else{
10      stack.pop()
11    }
12  }
13 }
```

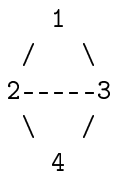
- Unified version:

```

1 DFS(start){
2   S.push(start)
3   while(!S.empty()){
4     current = S.pop
5     if(unvisited(current)){
6       visit current.
7       for all(next to current and unvisited)
8         S.push(next);
9     }
10  }
11 }
```


Why we need two "unvisited" words in DFS unified version.

- When we pop node from S, then we visit it, then we add all adjacent nodes to S, In this way, When we pop 1 from S, we add 2, 3 to S. at this time, 3 is unvisited, then when we pop 2 from S, we will add 4, 3 to S. so there are two 3 in the S.
- Based on, we need to first unvisit to avoid visit 3 twice
- When we reach 3; 1,2,4 have been visited, so when we want to add all 3 adjacent, we need second unvisited to avoid 1,2,4 to S.



DFS is an algorithm to search a hierarchical structure. But, pre-order traversal seems to be something similar also. So, what is the difference between the two?? DFS says:

1. If element found at root, return success.
2. If root has no descendants, return failure
3. Recursive DFS on left subtree: success if element found
4. If not, Recursive DFS on right subtree: success if element found

Pre-order Traversal says:

1. Visit the root
2. Recursive pre-order on left subtree
3. Recursive pre-order on right subtree

1.8.2.2 BFS

There are two version of BFS:

- Form DFS unify version, We can change S(stack) to Q(queue) and get unified version. **That is why we call it unified version**

```

1 BFS( start ) {
2   Q.push( start )
3   while ( !Q.empty() ) {
4     current = Q.pop
5     if ( unvisited( current ) ) {
6       visit current .
7       for all ( next to current and unvisited )
8         Q.push( next ) ;
9     }
10  }
11 }

```

- Common version: Unified version will have multi-copy vertex in the queue. We can improve it as below: 1) when you add it to queue, visit it. In this way, avoid add multi-copy vertex in queue 2) Because All the vertex in queue is visited, when you pop it, you don't need to judge if it's visisted.

```

1 DFS( start ) {
2   D.push( start )

```

```

3  while(!D.empty()){
4      current = D.pop //2) no if here
5      for all(next to current and unvisited){
6          visit current. //1) push and visit at the same time.
7          D.push(next);
8      }
9  }
10 }
```

- conclusion:

1. **Conclusion, just remember unified version.** 1) pop current, push neighbor, 2) check two unvisited
2. unified version will push two nodes into stack or queue, so for BFS, you can visit and push at the same time, in this way, you don't need to check current if it is unvisited.
3. For DFS, you can't use this way, because you can only visit only one neighbor, or it will become BFS. so each time, you just push one node into stack. It leads to two other versions. recursive version, and loop version.

Path is saved in the stack. when you quit while loop. just pop up each position from stack, then you get a path. But for BFS, you need extra information to save parent information.

1.8.3 Spanning tree

BFS and DFS will produce spanning tree.

The minimum spanning tree can be obtained in polynomial time: Kruskal's algorithm, Prim's algorithm and Sollin's algorithm.

Kruskal algorithm:

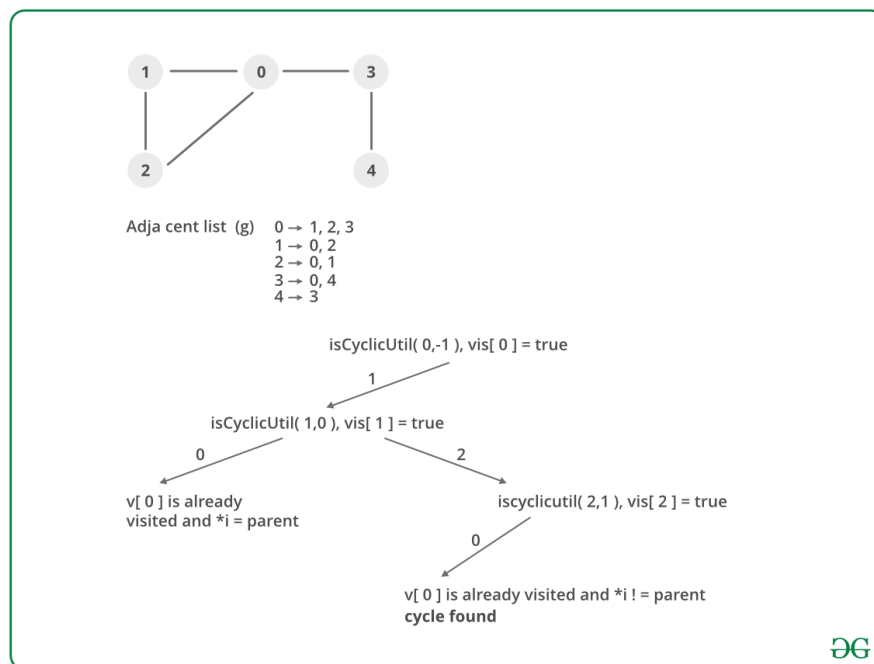
```

1 KRUSKAL(G) :
2   A = empty
3   foreach v in G.V:
4       MAKE-SET(v)
5   foreach (u, v) ordered by weight(u, v), increasing:
6       if FIND-SET(u) != FIND-SET(v):
7           A = A ∪ {(u, v)}
8           UNION(u, v)
9   return A
```

Source code: union find algorithm.

1.8.4 Application

detect a cycle in the graph. **Undirected graph, not directed graph.** The key point is just find visited[i], but if visited[i] is parent, then it's not a cycle. The whole logic is not complex.



```

1 // A recursive function that uses visited[] and parent to detect
2 // cycle in subgraph reachable from vertex v.
3 bool Graph::isCyclicUtil(int v, bool visited[], int parent){
4     visited[v] = true; // Mark the current node as visited
5
6     // Recur for all the vertices adjacent to this vertex
7     list<int>::iterator i;
8     for (i = adj[v].begin(); i != adj[v].end(); ++i){
9         // If an adjacent is not visited, then recur for that adjacent
10        if (!visited[*i]) {
11            if (isCyclicUtil(*i, visited, v))
12                return true;
13        }
14    }
15    // If an adjacent is visited and not parent of current vertex, then there is a cycle.
16    else if (*i != parent)
17        return true;
18    }
19    return false;
20 }

```

Direct graph detect cycle. **need to add a onPath to detect cycle, different with undirected graph.** When we want to push node to stack, don't judge if we have visited them or not? (undirected graph use visited[], directed graph use onPath[])

```

1 boolean[] onPath;
2 boolean[] visited;
3 boolean hasCycle = false;
4 void traverse(List<Integer>[] graph, int s) {
5     if (onPath[s]) {
6         // find cycle
7         hasCycle = true;
8     }
9     if (visited[s] || hasCycle) {
10        return;
11    }

```

```

12  visited[s] = true;
13
14  onPath[s] = true;
15  for (int t : graph[s]) {
16      traverse(graph, t);
17  }
18  onPath[s] = false;
19  }

```

1.9 Other customized data structure

1.9.1 union find

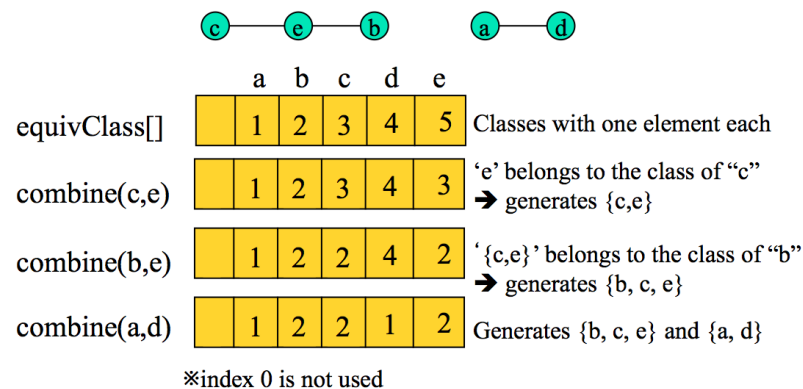
It is also called as "Online Equivalence class"

definition of Equivalence relationship

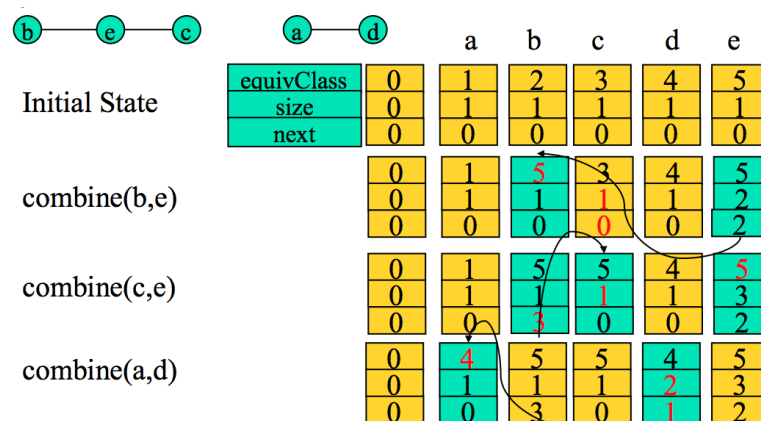
- $a \sim a$. (Reflexivity)
- $a \sim b$ if and only if $b \sim a$. (Symmetry)
- if $a \sim b$ and $b \sim c$ then $a \sim c$. (Transitivity)

online equivalence class, also known union-find question.

- method 1: use array



- method 2: stimulate pointer



```

1  class UF {
2      // Number of connected components
3      private int count;
4      // Stores the parent node of each node
5      private int[] parent;
6
7      // n is the number of nodes in the graph
8      public UF(int n) {
9          this.count = n;
10         parent = new int[n];
11         for (int i = 0; i < n; i++) {
12             parent[i] = i;
13         }
14     }
15
16     // Connects node p and node q
17     public void union(int p, int q) {
18         int rootP = find(p);
19         int rootQ = find(q);
20
21         if (rootP == rootQ)
22             return;
23
24         parent[rootQ] = rootP;
25         count--; // Merges two connected components into one
26     }
27
28     // Checks if node p and node q are connected
29     public boolean connected(int p, int q) {
30         int rootP = find(p);
31         int rootQ = find(q);
32         return rootP == rootQ;
33     }
34
35     public int find(int x) {
36         if (parent[x] != x) {
37             parent[x] = find(parent[x]);
38         }
39         return parent[x];
40     }
41
42     // Returns the number of connected components in the graph
43     public int count() {
44         return count;
45     }
46 }

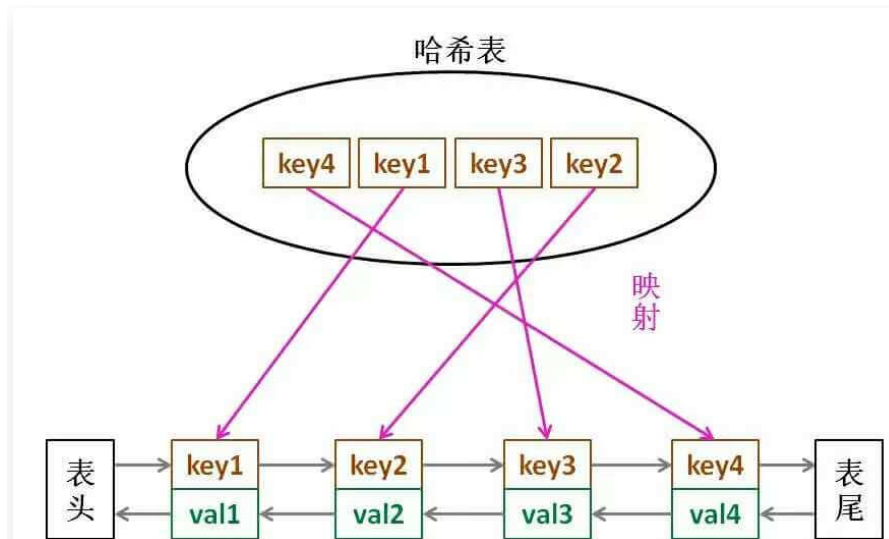
```

1.9.2 LRU and LFU

LRU is hash + list. (listhashmap)

LFU is hash + heap. hashMap<key, pairvalue, frequent>. we still need key data structure to manage the frequency hashMap<frequent, hashLinkSet> freqToKeys. freqToKeys is key to this question.

In LRU and LFU, The key data structure is LinkedHashMap, in LFU, it use LinkedSetMap, in fact, LinkedSetMap still use the LinkedHashMap behind the scene.



1.9.3 $O(1)$ insert, delete and getRandom

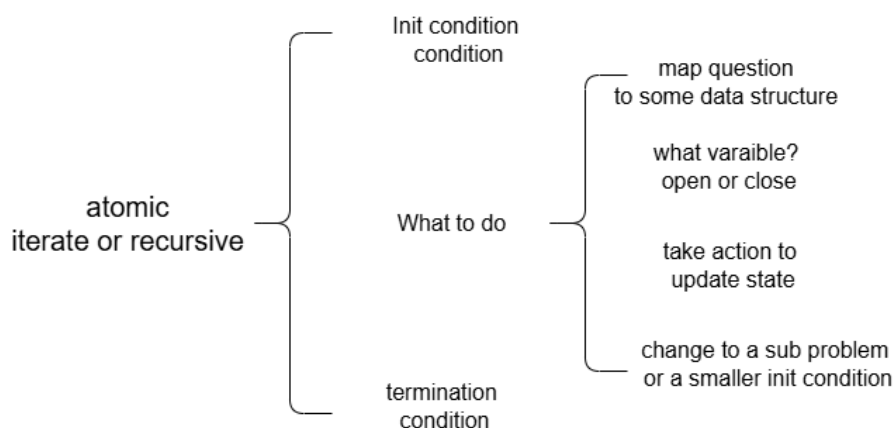
Insert, delete get random $O(1)$ hash + vector (no need to keep vector sort). When you want to delete, swap it with the last element in the vector, then delete it. **Most of complex data structure need hash + list/vector combine, it's a good clue**

- C++ does not offer a collection template with the behavior that would mimic Java's `LinkedHashMap<K,V>`, so you would need to maintain the order separately from the mapping. This can be achieved by keeping the data in a `std::list<std::pair<K,V>>`, and keeping a separate `std::unordered_map<k,std::list<std::pair<K,V>>::iterator>` map for quick look-up of the item by key:
 1. On adding an item, add the corresponding key/value pair to the end of the list, and map the key to the iterator `std::prev(list.end())`.
 2. On removing an item by key, look up its iterator, remove it from the list, and then remove the mapping.
 3. On replacing an item, look up list iterator from the unordered map first, and then replace its content with a new key-value pair.
 4. On iterating the values, simply iterate `std::list<std::pair<K,V>>`.

Chapter 2

Algorithm

2.0.1 Solution space



Why this basic conception is so important? Repeated permutation can be use in 0/1 backpack problem. LCS problem.

Nearest point pair's solution space is $n*(n-1)$. It's n^2 . A good hint is that it can be reduce $n * \log(n)$. So We should consider D&C (divide and conquer)

0/1 packback's solution's space is 2^n , A good hint is that I can be reduce to n^c . (c is constant) by dynamic programming.

If there is not D&C or DP, you have to use branch and bound or backtracking. Such as TSP traveling sales problem

If you need optimal answer, You need to traversal the whole solution space. (such as TSP and 0/1 backpacking). If you don't need optimal answer, you don't need to do that, In these two conditions, you all need to **backtracking**.

Why I can't use DP to TSP, but I can use DP to 0/1?

Recursion is a mathematical induction.

Modeling is mapping your problem to a abstract structure.

- Permutation: arrangement, tour, ordering sequence
- subset: cluster, collection, committee, group, packaging, selection
- tree:hieracrchy, dominance, ancestor/descendant, taxonomy

- graphs: network, circuit, web, relationship
- points
- polygon
- string

Some typical applications, you need to remember them.

- 5 cities, TSP, time complexity is $n!$ ($5! = 120$)
- 5 number, whole set partition. bell number, for 5, last result is 42. please see set partition number in recursive section.
- 5 object, 0/1 backpack, Subset, time complexity is 2^n ($2^5=32$)
- 5 icecream, 3 spoon, combination, ($c(5, 3) = 10$). combination is subset(number of element is 3) of subset, they are different.
- 5 repetitive combination, 5 icecream, 3 spoon, ($c(5+3-1, 3)$)
- distribute 10 candy to 3 kid, that is integer partition. result is ($c(10+3-1, 2)$). It's like above repetitive combination problem. thank there are 10 number, and two bar, put two bar in 12 position. The same question is: 10 same ball, put three bucket, that is same above quetions.

```

1 void distribute(int balls_left, int buckets_left, std::vector<int>& current) {
2     if (buckets_left == 1) {
3         // Only one bucket left: put all remaining balls here
4         current.push_back(balls_left);
5
6         // Print the result
7         for (int val : current) std::cout << val << "_";
8         std::cout << "\n";
9
10        current.pop_back(); // backtrack
11        return;
12    }
13
14    for (int i = 0; i <= balls_left; ++i) {
15        current.push_back(i);
16        distribute(balls_left - i, buckets_left - 1, current);
17        current.pop_back(); // backtrack
18    }
19 }
20
21 int main() {
22     int total_balls = 10;
23     int buckets = 3;
24     std::vector<int> current;
25     distribute(total_balls, buckets, current);
26 }

```

- 10 different ball, put 3 different bucket, There are two different perspectives. One is from ball perspective, the other is from bucket perspective. From ball perspective, the result is $\text{pow}(3, 10)$.
- 10 differnt ball, split 3 equals. A specal case of above question, you can use ball perspective way, but it's low effecient. A better way is to use bucket perspetivce, we don't need to get different combination, for one bucket, get all subset of 10. the process can be repeat 3 time. So the result is $3 * \text{pow}(2, n)$. much less than $\text{pow}(3, 10)$

2.0.1.1 Big O

You should understand Big O here. For example:

- $O(n)$ is found max, min or sum.
- $O(n^2)$ is bubble sort
- $O(n^3)$ is solutions for a multi-variable equation: $3x+4y+5z = 108$, then get all x, y, z . three kids split 10 candy.
- $O(n \log n)$ is quick sort. binary search. DC
- $O(2^n)$ print all subsequences. subsequence and subset are different concepts, even though they sound similar. subsequence must keep the original order, but in subset, order doesn't matter. subsequence mainly used in string or array, subset mainly used in set or list.
- catalan number. print all parentheses.
- $O(n!)$ print all permutation.

Big O notation. Remember below English words: constant, logarithmic ($\log n$), linear (n) linearithmic ($n \log n$), quadratic 2^n (2 to the power of n), polynomial (quadratic cubic), exponential c^n . factorial $n!$. $O(10^N)$: trying to break a password by testing every possible combination (assuming numerical password of length N). traveling salesman problem (TSP) is factorial $n!$. Both factorial and exponential are non-polynomial (NP).

verb|<https://webusers.imj-prg.fr/~jan.nekovar/co/en/en.pdf>| list some common mathematical English. space complexity is much simple, usually, we only use $O(1)$, $O(n)$ and $O(n^2)$, we don't use other forms very often.

2.1 Recursive

2.1.1 Basic

Consider an array/string: 1,2,3,4

```

1 Subarray: contiguous sequence in an array i.e. {1,2},{1,2,3}
2
3 Subsequence: Need not to be contiguous, but maintains order i.e. {1,2,4}
4
5 Subset: Same as subsequence except it has empty set i.e. {1,3},{ }
6
7 Given an array/sequence of size n, possible
8 Subarray =  $n*(n+1)/2$ 
9 Subsequence =  $(2^n) - 1$  (non-empty subsequences)
10 Subset =  $2^n$ 
```

recursive is not an algorithm, it's just a method. D&C, DP, backtrack, bound and boundary all use recursive in their implementation, but different ways.

First, the inline specification on a function is just a hint. The compiler can (and often does) completely ignore the presence or absence of an inline qualifier. With that said, a compiler can inline a recursive function, much as it can unroll an infinite loop. It simply has to place a limit on the level to which it will "unroll" the function.

One recursive (Factorial), Two recursive call (Hanoti) (Tree), Multi Recursive (Permutation)

result is single (Factorial), Result is many steps (Hanoti) (Maze), Result is a set (permutation). you can see that 1) any recursive function should at least one input parameter, and this parameter should to be pass sub-problem. 2) for return value, it has two different kind, if result is single value, you should return a value, such as Factorial. If result is set (permutation) or many steps (in-order traversal of tree or Hanoi tree), you can declare your function as void. 3) Some functions need input another

parameter, such as level information in the tree or position information in the permutation. 4) Base case can be understood differently, most of time base case is 0 or null, but for permutation, base case is $i=n$. most subproblem, i decrease, but for permutation, for each subproblem i increase.

- single result

```

1  int Factorial(int n){
2      if (n == 1)
3          return 1;
4      return n*Factorial(n-1);
5  }

```

- result is serial steps (Hanoti) DFS(Maze)

```

1  FUNCTION MoveTower(disk, source, dest, spare):
2      IF disk == 0, THEN:
3          move disk from source to dest
4      ELSE:
5          MoveTower(disk - 1, source, spare, dest)    // Step 1 above
6          move disk from source to dest                // Step 2 above
7          MoveTower(disk - 1, spare, dest, source)    // Step 3 above
8      END IF

```

- Result is a set(permutation)

It includes direct recursive and indirect recursive. direct recursive is $R()$ call $R()$ again in it's function body. It has two parts: **1)base and 2) recursive component**

2.1.2 Basic recursive pattern

Print all subsequences, why is $power(2,n)$?

```

1  void sub(string& s, int i, int j, string res) {
2      if (i == j) {
3          cout << res << endl;
4          return;
5      }
6      sub(s, i + 1, j, res); //don't select current character
7
8      res.push_back(s[i]); //select current character.
9      sub(s, i + 1, j, res);
10 }

```

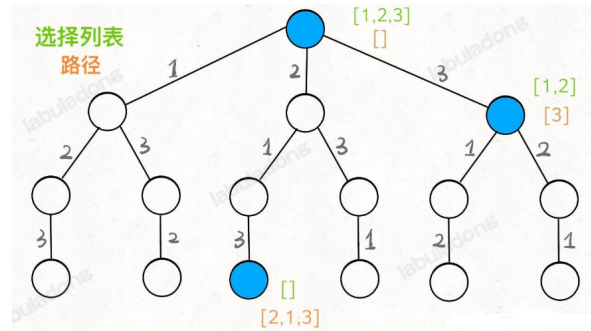
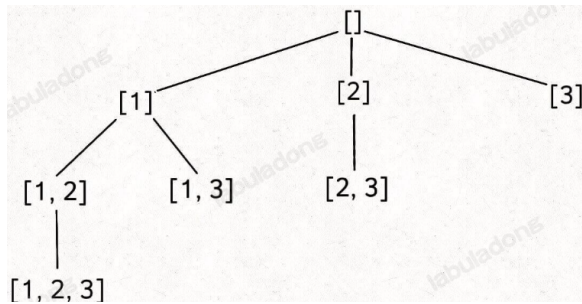
We can use another method to resolve this question. That is bucket perspective, above code is ball perspective.

```

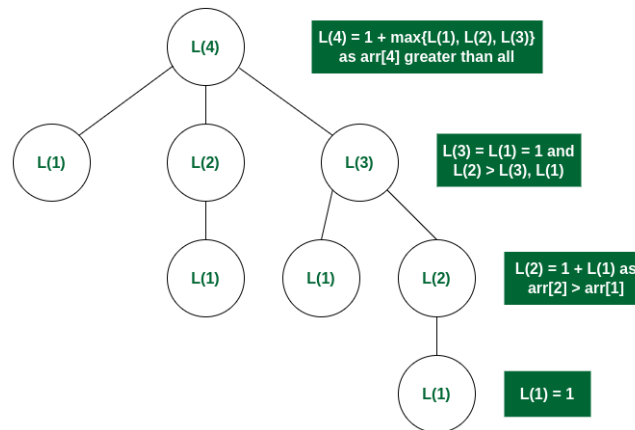
1  void backtrack(int[] nums, int start) {
2      // pre-order, every node is a subset. such as, empty subset[]
3      res.add(new LinkedList<>(track));
4
5      for (int i = start; i < nums.length; i++) {
6          track.addLast(nums[i]); // make a choice
7
8          // Control the traversal of branches using the start parameter to avoid
9          // generating duplicate subsets.
10         backtrack(nums, i + 1);
11
12         track.removeLast(); // revoke a choice
13     }
14 }

```

In this tree, each edge is choice, each node is an answer, for $n = 3$, there are 8 nodes in below graph. We call it back track tree.



A similar idea is to LIS problem. it's also 2^n you can think in this way, previous one is $2*f(n-1)$, below is $f(n-1) f(n-2) \dots f(1)$ but $f(n-2) \dots f(1)$ is just another $f(n-1)$, so equal $2*f(n-1)$. Detail is we have a lot of overlap problem. Time complexity is $\text{power}(2, n)$ The time complexity of this recursive approach is exponential as there is a case of overlapping subproblems as explained in the recursive tree diagram above.



```

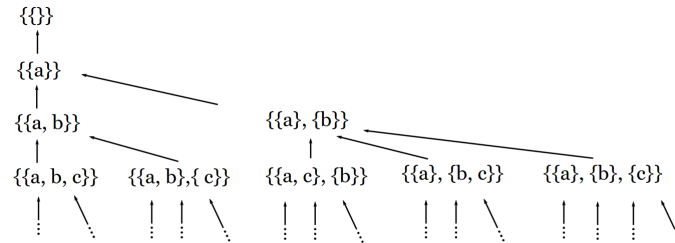
1  int _lis( int arr[], int n, int *max_ref){
2      if (n == 1) /* Base case */
3          return 1;
4
5      // 'max_ending_here' is length of LIS ending with arr[n-1]
6      int res, max_ending_here = 1;
7
8      /* Recursively get all LIS ending with arr[0], arr[1] ... arr[n-2]. If arr[i-1] is
9         smaller
10     than arr[n-1], and max ending with arr[n-1] needs to be updated, then update it */
11     for (int i = 1; i < n; i++){
12         res = _lis(arr, i, max_ref);
13         if (arr[i-1] < arr[n-1] && res + 1 > max_ending_here)
14             max_ending_here = res + 1;
15     }
16
17     // Compare max_ending_here with the overall max. And update the overall max if
18     // needed
19     if (*max_ref < max_ending_here)
20         *max_ref = max_ending_here;
21
22     // Return length of LIS ending with arr[n-1]
23     return max_ending_here;
24 }

```

If the answer is one, such as LIS, the result should be in the root node, we should use post order, If the answer are a lot , the result should be in leaf node, we should use base case. such as all possible subsequence.

Given a set $A = 1, 2, 3, \dots, n$. It is called a partition of the set A if the following conditions follow:

1) The union of all the sets is the set A . 2) The intersection of any two sets is an empty set.



```

1 // Function to print a partition
2 void printPartition(vector<vector<int>> ans)
3 {
4     for (auto i : ans) {
5         cout << "{_";
6         for (auto element : i) {
7             cout << element << "_";
8         }
9         cout << "}_" ;
10    }
11    cout << endl;
12 }
13
14 // Function to generate all partitions
15 void Partition(vector<int> set, int index, vector<vector<int>>& ans){
16
17     // If we have considered all elements in the set print the partition
18     if (index == set.size()) {
19         printPartition(ans);
20         return;
21     }
22
23     // For each subset in the partition add the current element to it and recall
24     for (int i = 0; i < ans.size(); i++) {
25         ans[i].push_back(set[index]);
26         Partition(set, index + 1, ans);
27         ans[i].pop_back();
28     }
29
30     // Add the current element as a singleton subset and recall
31     ans.push_back({ set[index] });
32     Partition(set, index + 1, ans);
33     ans.pop_back();
34 }
35
36 // Function to generate all
37 // partitions for a given set
38 void allPartitions(vector<int> set){
39     vector<vector<int>> v;
40     Partition(set, 0, v);
41 }
42

```

```

43
44 int main() {
45     int n = 5;
46     vector<int> set(n);
47     for (int i = 0; i < n; i++) {
48         set[i] = i + 1;
49     }
50
51     // Generate all partitions of the set
52     allPartitions(set);
53     return 0;
54 }

```

```

1 { 1 2 3 4 5 }
2 { 1 2 3 4 } { 5 }
3 .....
4 { 1 2 4 } { 3 5 }
5 { 1 } { 2 } { 3 } { 4 } { 5 }

```

Catalan number, all possible parentheses combination. (It's not good implementation, some child problems have been calculated many time. You should use dynamic programming to remember the child problems result. but child program result is very big. It's a question for ALL , not for ONE result. So backtrace algorithm is more good for this questions.

Re-interpreting the symbol X as an open parenthesis and Y as a close parenthesis, Cn counts the number of expressions containing n pairs of parentheses which are correctly matched:

```

1 void generate(int open, int close, string& current) {
2     if (open == 0 && close == 0) {
3         cout << current << endl;
4         return;
5     }
6
7     if (open > 0) {
8         current.push_back('(');
9         generate(open - 1, close, current);
10        current.pop_back(); // backtrack
11    }
12
13    if (close > open) {
14        current.push_back(')');
15        generate(open, close - 1, current);
16        current.pop_back(); // backtrack
17    }
18 }
19
20 int main() {
21     int n = 3;
22     string current;
23     generate(n, n, current);
24     return 0;
25 }

```

```

1 void generate(int open, int close, string current, vector<string>& result) {
2     if (open == 0 && close == 0) {
3         result.push_back(current);
4         return;
5     }
6

```

```

7   if (open > 0)
8       generate(open - 1, close, current + "(", result);
9
10  if (close > open)
11      generate(open, close - 1, current + ")", result);
12  }
13
14  int main() {
15      int n = 3;
16      vector<string> result;
17      generate(n, n, "", result);
18
19      for (const string& s : result)
20          cout << s << endl;
21
22      return 0;
23  }

```

```

1   ((( )))      (( ) ( )      ( ( ) ( )      ( ) ( ( )      ( ) ( ) ( )

```

C_n is the number of different ways $n + 1$ factors can be completely parenthesized (or the number of ways of associating n applications of a binary operator, as in the matrix chain multiplication problem). For $n = 3$, for example, we have the following five different parenthesizations of four factors:

```

1   ((ab)c)d      (a(bc))d      (ab)(cd)      a((bc)d)      a(b(cd))

```

```

1   vector<string> ap(string& s, int i, int j) {
2       vector<string> result;
3       if (i == j-1 && i < s.size()) {
4           result.push_back(s.substr(i,1));
5           return result;
6       }
7       for (int k = i; k < j-1; ++k) {
8           vector<string> v1 = ap(s, i, k+1);
9           vector<string> v2 = ap(s, k+1, j);
10
11           string rs;
12           for (auto e : v1) {
13               for (auto e1 : v2) {
14                   rs = "(";
15                   rs += e;
16                   rs += e1;
17                   rs += ")";
18                   result.push_back(rs);
19               }
20           }
21       }
22
23       return result;
24   }
25
26   string s = "abcd";
27   auto com = ap(s, 0, 4);
28
29   for (auto e : com) {
30       cout << e << endl;
31   }
32
33   (a(b(cd)))

```

```

34  (a((bc)d))
35  ((ab)(cd))
36  ((a(bc))d)
37  (((ab)c)d)

```

The same idea is to build binary search tree

```

1  // Recursive function to return a list of tree pointers of all possible
2  // binary trees having the same inorder sequence as 'in[start, end]'
3  vector<Node*> generateBinaryTrees(vector<int> &in, int start, int end)
4  {
5      // create an empty list to store the root of the constructed binary trees
6      vector<Node*> trees;
7
8      // base case
9      if (start > end)
10     {
11         trees.push_back(nullptr);
12         return trees;
13     }
14
15     // consider each element in the inorder sequence as the root
16     for (int i = start; i <= end; i++)
17     {
18         // recursively find all possible left subtrees for root 'i'
19         vector<Node*> left_subtrees = generateBinaryTrees(in, start, i - 1);
20
21         // recursively find all possible right subtrees for root 'i'
22         vector<Node*> right_subtrees = generateBinaryTrees(in, i + 1, end);
23
24         // do for each combination of left and right subtrees
25         for (Node* l: left_subtrees)
26         {
27             for (Node* r: right_subtrees)
28             {
29                 // construct a binary tree with i'th element as the root and whose
30                 // left and right children point to 'l' and 'r', respectively
31                 Node* tree = new Node(in[i], l, r);
32
33                 // add this tree to the output list
34                 trees.push_back(tree);
35             }
36         }
37     }
38
39     return trees;
40 }

```

```

1  void subset(int arr[], vector<int>& result, int k) {
2      if (k == 6) {
3          cout << "one_answer_is_(_";
4          for (int i = 0; i < result.size(); i++) {
5              cout << result[i] << "_";
6          }
7          cout << ")" << endl;
8          return;
9      }
10     result.push_back(arr[k]);
11     subset_bucket(arr, result, k + 1);
12     result.pop_back();

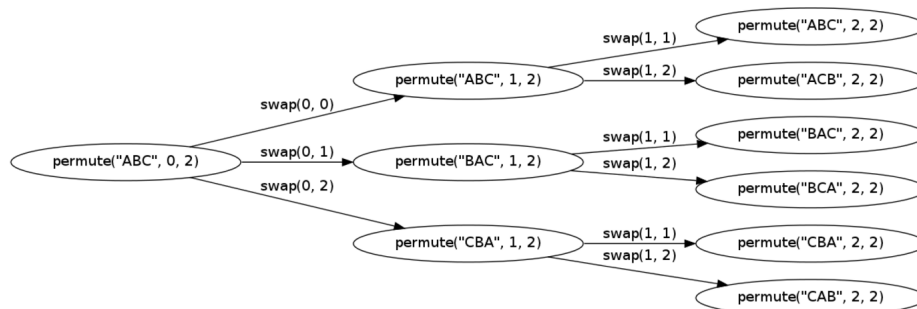
```

```

13 subset_bucket(arr, result, k + 1);
14 return;
15 }

```

permutation, It is also worth noting that when you make a recursive call, you advance down an individual branch of the tree, and an additional branch is added with every iteration of a 'for' or 'while' loop. One confusing thing about this problem is the second swap after the recursive call to permute. This can be interpreted as 'unswap,' and is required because the char array is passed by reference, not by value, and every time you swap elements in the array the change is visible down stream.



```

1 void permute(char a[], int i, int n){
2     int j;
3     if (i == n)
4         cout << a << endl;
5     else {
6         for (j = i; j <= n; j++) {
7             swap(a[i], a[j]);
8             permute(a, i+1, n);
9             swap(a[i], a[j]);
10        }
11    }
12 }

```

compared with subset code, You will find that they are all recursive. permutation is $n!$

```

1 permute(i)
2 for(j...) //calculate j times same size(i+1) sub problem.
3 permute(i+1) //similar with n*f(n-1)

```

2.1.2.1 When to Use Recursion (Despite the Overhead)

1. Clarity and Simplicity for Recursive Data Structures

Some problems map naturally to recursion:

- Linked lists
- Trees
- Graphs
- Recursive patterns (e.g., fractals, file systems)

Example:


```
void inorder(TreeNode* root) {
    if (!root) return;
    inorder(root->left);
    visit(root->val);
    inorder(root->right);
}
```

This recursive tree traversal is clean, readable, and directly mirrors the structure of the data.

2. Divide and Conquer Algorithms

Recursive thinking fits naturally in algorithms like:

- Merge Sort
- Quick Sort
- Binary Search
- Backtracking (e.g., solving Sudoku, N-Queens)

Even if they can be rewritten iteratively, the recursive form is often much easier to understand and implement.

3. Mathematical Recursion / Definitions

Some algorithms are defined recursively, and recursive implementations are more direct:

- Factorial, Fibonacci
- Combinations / permutations
- Recursive dynamic programming (with memoization)

2.1.3 When to Avoid Recursion

- Performance-critical code
- Large inputs (risk of stack overflow)
- Deep recursion without tail-call optimization
- Platforms with limited stack size (e.g., embedded systems)

2.1.4 Rule of Thumb

Situation	Use Recursion?	Reason
Tree traversal	Yes	Natural fit, clean syntax
Graph DFS/BFS	Maybe	DFS = recursion; BFS = queue-based
Deep iteration with no recursion	No	Stack-safe, faster
Divide & conquer (e.g., mergesort)	Yes	Matches conceptual model
Large input size ($> 10^4$ elements)	No (usually)	Stack depth issues

Table 2.1: When to use recursion: rule of thumb

2.1.5 Conclusion

While recursion has performance drawbacks, it is still highly useful for expressing problems cleanly and matching natural structures. Use it when it makes code clearer, but be mindful of stack depth and convert to iteration if performance or memory becomes an issue.

2.2 Math and geometry basic

2.2.1 Geometry

If three points make a triangle?

```

1 // Calculation the area of triangle. We have skipped
2 // multiplication with 0.5 to avoid floating point computations
3 // if a is 0, don't make a triangle.
4 int a = x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);

```

Judge three points are in the same line?

$$\frac{y_2 - y_1}{x_2 - x_1} = \frac{y_3 - y_1}{x_3 - x_1}$$

Get one points in the triangle.

$$x = \frac{\frac{x_1 + x_2}{2} + x_3}{2}$$

If two rectangles overlap? l1: Top Left coordinate of first rectangle. r1: Bottom Right coordinate of first rectangle. l2: Top Left coordinate of second rectangle. r2: Bottom Right coordinate of second rectangle. 1) One rectangle is above top edge of other rectangle. 2) One rectangle is on left side of left edge of other rectangle.

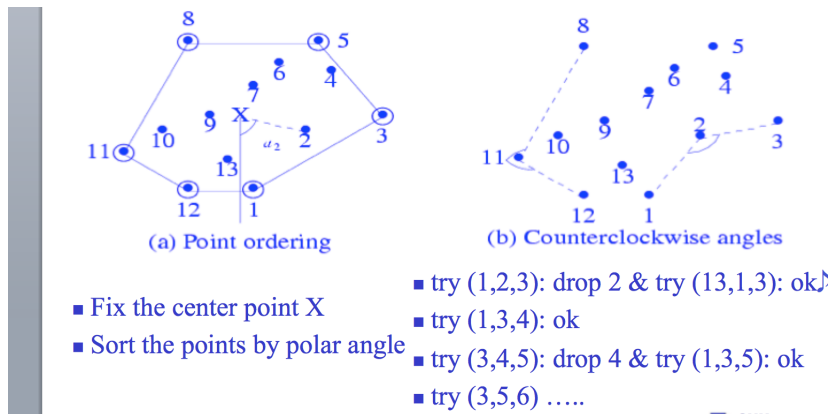
```

1 // Returns true if two rectangles (l1, r1) and (l2, r2) overlap
2 bool doOverlap(Point l1, Point r1, Point l2, Point r2){
3     // If one rectangle is on left side of other
4     if (l1.x > r2.x || l2.x > r1.x)
5         return false;
6     // If one rectangle is above other
7     if (l1.y < r2.y || l2.y < r1.y)
8         return false;
9     return true;
10 }

```

2.2.1.1 Convex hull

1. Find a point C that is inside the convex hull of S ((sum of x coordinates)/ n, (sum of y coordinates)/n)
2. Sort S by polar angle and within polar angle by distance from C. Use atan2(y-Cy , x-Cx) c++ function.
3. Create a doubly linked circular list of points using above order Let right link to the next point in the order and left link to the previous point



4. Let p be the point that the smallest y-coordinate (break a tie, if any, by selecting the one with largest x-coordinate)
5. use Law of cosines to calculate angle formed by x, rx and rrx

$$c^2 = a^2 + b^2 - 2ab \cos \lambda$$

```

1  for (x = p; rx = point to the right of x; x != rx) {
2      rrx = point to the right of rx;
3      if (angle formed by x, rx, and rrx is <=180 degrees) {
4          delete rx from the list;
5          rx = x;
6          x = point on left of rx;
7      }
8      else {
9          x = rx; rx = rrx; }
10 }
```

6. For convex hull problem, you can use double link list to store all the extreme points. Because it need to use three consequence points to measure the degree to judge if it's less than 180 degrees. So double link list is the best data structure to use.

2.2.2 Permutation and combination

repeated / unrepeated (Permutation/Combination)

repeated permutation: Three number, how many permutation 10^3 . Another Three position, For each position, you can select 10 options and fill it. If three position, For each position, you can select 2 options and fill it. You get powerset problem 2^3 {a,b, c} all the subset.

unrepeated permutation: $n! = \text{factorial function}$

unrepeated combination:

$$C(n, r) = {}^nC_r = {}_nC_r = \binom{n}{r} = \frac{n!}{r!(n-r)!}$$

repeated combination:

$$\binom{r+n-1}{r} = \frac{(r+n-1)!}{r!(n-1)!}$$

where n is the number of things to choose from, and we choose r of them (Repetition allowed, order doesn't matter)

Detail can be seen here: <https://www.mathsisfun.com/combinatorics/combinations-permutations.html>

2.2.3 Bit

Basic operations are: and, or, xor, not, right shift, left shift.

```
1 & | ^ ! << >> //six operators related with bits.
2
3 number |= 1UL << n; // set
4 number &= ~(1UL << n); //clear
5 number ^= 1UL << n; //toggle, ^= can be used for toggle.
```

Many C compilers choose which right shift to perform depending on what type of integer is being shifted; often signed integers are shifted using the arithmetic shift, and unsigned integers are shifted using the logical shift.

```
1 a|b|c|d //after we right shift, we change it to below
2 x|a|b|c
3 1) with a logical shift put 0 in x
4 2) with a arithmetic shift put a in x
```

XOR is exclusive or, You can think that it's subset of OR, for OR, if both bits are 1, the result is 1. but In XOR, we exclusive this possible, You can think that it's "pure" or. Based on previous definition. There are two characteristics:

```
1 x^x = 0
2 x^0 = x
```

from previous two examples, we can use it to find any number occur even number in the array.

change the last 1 to zero. The basic idea is very interesting.

```
1 n&(n-1);
2 //110100-> 110000
3
4 //when we n-1: 110100, the right most 1 will become 0, all zero after
5 //right most 1 will become 1.
6 110100
7 110011
8 *—
```

Based on previous trip, we can test if a number is power of 2. There is only one 1 in the binary formation.

```
1 bool isPowerOfTwo(int n) {
2     if (n <= 0) return false;
3     return (n & (n - 1)) == 0;
4 }
```

Hamming weight, Hamming weight of an integer is defined as the number of set bits in its binary representation.

```

1 int hammingWeight(uint32_t n) {
2     int res = 0;
3     while (n != 0) {
4         n = n & (n - 1); //change n, you need to assign new value back to n.
5         res++;
6     }
7     return res;
8 }

```

As noted in this answer, $n \& (n-1)$ unsets the last set bit. So, if we unset the last set bit and xor it with the number; by the nature of the xor operation, the last set bit will become 1 and the rest of the bits will return 0

```

1 int set_bit = n ^ (n & (n-1));

```

2.2.4 Usage of modulus %

Some applicatoin you can use with modulus (reminder).

- circular queue
- hash
- to know the last three digit.

We usually use the

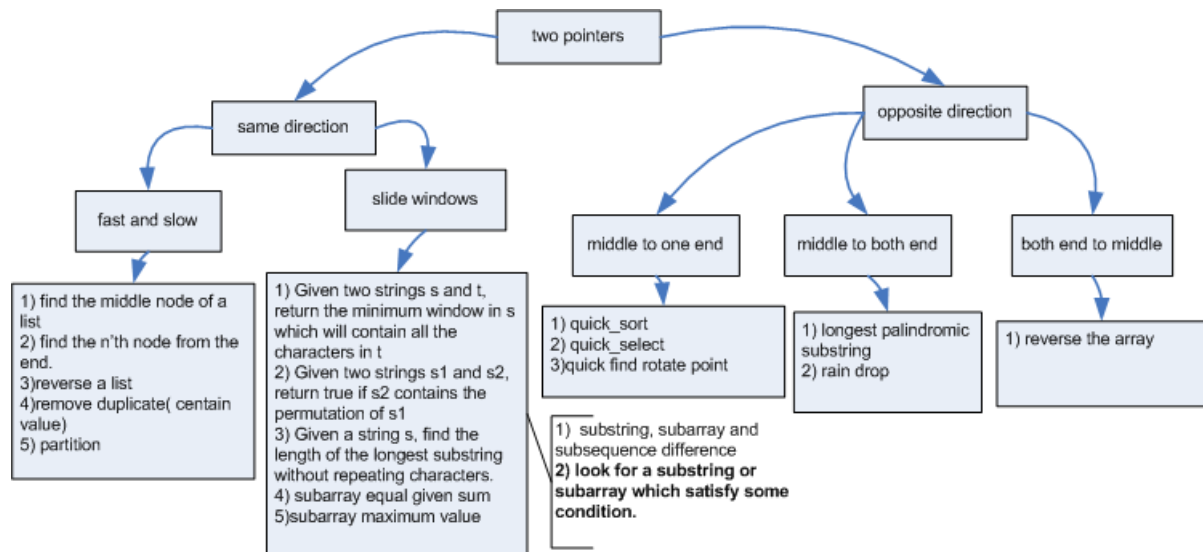
```

1 int[] arr = {1,2,3,4,5};
2 int n = arr.length, index = 0;
3 while (true) {
4     // Looping through a circular array
5     print(arr[index % n]);
6     index++;
7 }
8
9 int[] nextGreaterElements(int[] nums) {
10    int n = nums.length;
11    int[] res = new int[n];
12    Stack<Integer> s = new Stack<>();
13    // Doubling the array length to simulate a circular array
14    for (int i = 2 * n - 1; i >= 0; i--) {
15        // Modulo operation for index i, the rest follows the template
16        while (!s.isEmpty() && s.peek() <= nums[i % n]) {
17            s.pop();
18        }
19        res[i % n] = s.isEmpty() ? -1 : s.peek();
20        s.push(nums[i % n]);
21    }
22    return res;
23 }

```

2.3 Two pointers

Two pointers are very popular algorithms in many interview questions.



2.3.1 Middle to both ends

The longest palindrome substring

```

1 string palindrome(string& s, int l, int r) {
2     //from l go to left, from r go to right.
3     //<-----l   r----->
4
5     // check if it is out of boundary
6     while (l >= 0 && r < s.size()
7     && s[l] == s[r]) {
8         // both side move
9         l--; r++;
10    }
11    // return result
12    return s.substr(l + 1, r - l - 1);
13 }
14
15 string longestPalindrome(string s) {
16     string res;
17     for (int i = 0; i < s.size(); i++) {
18         // s[i] is middle
19         string s1 = palindrome(s, i, i);
20         // s[i] and s[i+1]
21         string s2 = palindrome(s, i, i + 1);
22         // res = longest(res, s1, s2)
23         res = res.size() > s1.size() ? res : s1;
24         res = res.size() > s2.size() ? res : s2;
25     }
26     return res;
27 }

```

Rain drop

```

1 int trap(vector<int>& height) {
2     if (height.empty()) return 0;
3     int n = height.size();
4     int res = 0;
5     // memo
6     vector<int> l_max(n), r_max(n);

```

```

7 // base case
8 l_max[0] = height[0];
9 r_max[n - 1] = height[n - 1];
10 // calculate l_max from left to right
11 for (int i = 1; i < n; i++)
12     l_max[i] = max(height[i], l_max[i - 1]);
13 // calculate r_max from right to left
14 for (int i = n - 2; i >= 0; i--)
15     r_max[i] = max(height[i], r_max[i + 1]);
16
17 // calculate the last answer.
18 for (int i = 1; i < n - 1; i++)
19     res += min(l_max[i], r_max[i]) - height[i];
20 return res;
21 }

```

2.3.2 slow and fast

remove duplicate

```

1 int removeDuplicates(int[] nums) {
2     if (nums.length == 0) {
3         return 0;
4     }
5     int slow = 0, fast = 0;
6     while (fast < nums.length) {
7         if (nums[fast] != nums[slow]) {
8             slow++;
9             // maintain nums[0..slow] without duplicate
10            nums[slow] = nums[fast];
11        }
12        fast++;
13    }
14    // length should be + 1
15    return slow + 1;
16 }

```

partition

```

1 template<class ForwardIt, class UnaryPredicate>
2 ForwardIt partition(ForwardIt first, ForwardIt last, UnaryPredicate p)
3 {
4     first = std::find_if_not(first, last, p);
5     if (first == last) return first;
6
7     for (ForwardIt i = std::next(first); i != last; ++i) {
8         if (p(*i)) {
9             std::iter_swap(i, first);
10            ++first;
11        }
12    }
13    return first;
14 }

```

Note here, in removeDuplication, we ++ first, then assign, but in partition, we swap first, then ++. you need to know that slow pointer is pointing by these two methods.

2.3.3 slide windows

Code template of slide windows. Slide windows is suitable for some sub-string problem.

```

1  int left = 0, right = 0;
2
3  while (right < n) {
4      // increase windows
5      window.add(a[right]);
6      right++;
7
8      //update windows
9      //.....
10
11     while (window needs shrink) {
12         // decrease window
13         window.remove(s[left]);
14         left++;
15
16         //update windows here.
17         //.....
18     }
19 }

```

An example. minimal substring in source which includes all the letters of in target. leetcode 76. s = "ADOBECODEBANC", t = "ABC, output "BANC"

```

1  string minWindow(string s, string t) {
2      unordered_map<char, int> need, window;
3      for (char c : t) need[c]++;
4
5      int left = 0, right = 0;
6      int valid = 0;
7      // Variables to record the starting index and length of the minimum window substring
8      int start = 0, len = INT_MAX;
9      while (right < s.size()) {
10         // 'c' is the character that will enter the window
11         char c = s[right];
12         right++; // Expand the window
13         if (need.count(c)) { // Update the data inside the window
14             window[c]++;
15             if (window[c] == need[c])
16                 valid++;
17         }
18
19         // Check if the left side of the window needs to shrink
20         while (valid == need.size()) {
21             // Update the minimum window substring here
22             if (right - left < len) {
23                 start = left;
24                 len = right - left;
25             }
26             // 'd' is the character that will exit the window
27             char d = s[left];
28             // Shrink the window
29             left++;
30             // Update the data inside the window
31             if (need.count(d)) {
32                 if (window[d] == need[d])
33                     valid--;
34                 window[d]--;

```



```

35     }
36   }
37 }
38 // Return the minimum window substring
39 return len == INT_MAX ? "" : s.substr(start, len);
40 }

```

Basic rules:

1. If a subarray or substring has constraints, if not, use two for brute force, an example is leetcode 2913
2. if there is constraints. double pointers, leetcode 3, 76, 567 and 3258.
3. Do we need map to store substring information? for 3, 76, and 567. we need it. For 3258, we don't need map to store character information in substring, that is very important clue, for 3, we need to judge if there is repeated character, so we need a map. A similar questions can be found.

2.4 Sort

2.4.1 Kth element

There is a few options to implement Kth element:

- $n \cdot \log(n)$ sort
- $n \cdot k$ partial select
- $n \cdot \log(k)$ heap
- quick select, average $O(n)$

k small number, use maxium heap(default)

```

1  std::priority_queue<int> pq;
2  for(auto e: arr){
3      if(pq.size() < 3){
4          pq.push(e);
5      }
6      else if(pq.top() > e){
7          pq.push(e);
8          pq.pop();
9      }
10 }
11
12 while(pq.size() > 0){
13     cout << pq.top() << endl;
14     pq.pop();
15 }

```

Basic idea is **quick select**, but we use median of median method to get good pivot, to make at the worst 30-70 partition. Pay attention to the k value need to be adjusted when you call a function recursively.

- WE should use left close, and right close method to deal with Divide conquer array problem. It will make your code a little easier.
- k-1, index is based on 0, but k is based on 1, that is why we need k-1 in line 18

```

1  int partition(vector<int>& arr, int l, int r) {
2      int x = arr[r], i = l;

```

```

3   for (int j = l; j <= r - 1; j++) {
4       if (arr[j] <= x) {
5           swap(arr[i], arr[j]);
6           i++;
7       }
8   }
9   swap(arr[i], arr[r]);
10  return i;
11 }
12
13 int quickselect(vector<int>& A, int left, int right, int k){
14     //p is position of pivot in the partitioned array
15     int p = partition(A, left, right);
16
17     //k equals pivot got lucky
18     if (p == k-1){
19         return A[p];
20     }
21     //k less than pivot
22     else if (k - 1 < p){
23         return quickselect(A, left, p - 1, k);
24     }
25     //k greater than pivot
26     else{
27         return quickselect(A, p + 1, right, k);
28     }
29 }
30
31 vector<int> A = {1,3,8,2,4,9,7};
32 int k = 5;
33 cout<<quickselect(A, 0, 6, 5); //output is 7

```

Stable_partition. The whole idea is just like merge sort, Pay attention here, merge sort is also stable sort.

```

1  #include <algorithm>
2
3  template <typename Iterator, typename Predicate>
4  Iterator my_stable_partition(Iterator first, Iterator last, Predicate predicate) {
5      auto n = std::distance(first, last);
6      if (n <= 1) {
7          if (n == 1 && predicate(*first))
8              ++first;
9          return first;
10     }
11     auto middle = first;
12     std::advance(middle, n / 2);
13     auto a = my_stable_partition(first, middle, predicate);
14     auto b = my_stable_partition(middle, last, predicate);
15     return std::rotate(a, middle, b);
16 }

```

median of medians (advance part, skip it if you don't have enough time.)

```

1  int array[] = { 1,12,3,4,1,      5,2,7,8,88,      5,2,32,1,35,      -1,7,5,38,-11 };
2
3  int insertSort(int left, int right) {
4      for (int i = left + 1; i <= right; i++) {
5          int temp = array[i], j;
6          for (j = i; j > left && array[j - 1] > temp; j--) array[j] = array[j - 1];
7          array[j] = temp;
8      }

```

```

9   return (left + right) >> 1;
10 }
11
12 int BFPRT(int, int, int);
13
14 int getPivotIndex(int left, int right) {
15     if (right - left < 5) return insertSort(left, right);
16     int back = left - 1;
17     for (int i = left; i + 4 < right; i += 5) {
18         std::cout << "i" << i << " " << right << std::endl;
19         int index = insertSort(i, i + 4);
20         std::swap(array[++back], array[index]);
21     }
22     return BFPRT(left, back, ((left + back) >> 1) + 1);
23 }
24
25
26 int partition(int left, int right, int pivotIndex) {
27     std::swap(array[right], array[pivotIndex]);
28     int mid = left;
29     for (int i = left; i < right; i++) {
30         if (array[i] < array[right])
31             std::swap(array[i], array[mid++]);
32     }
33     std::swap(array[right], array[mid]);
34     return mid;
35 }
36
37 int BFPRT(int left, int right, int k) {
38     int pivotIndex = getPivotIndex(left, right);
39     int mid = partition(left, right, pivotIndex);
40     int count = mid - left + 1;
41     if (count == k) {
42         return mid;
43     }
44     else if (count > k) {
45         return BFPRT(left, mid - 1, k);
46     }
47     else {
48         return BFPRT(mid + 1, right, k - count);
49     }
50 }
51
52 int main() {
53     int k = 5;
54     int length = sizeof(array) / sizeof(array[0]);
55     for (int i = 0; i < length; i++) {
56         std::cout << array[i] << " ";
57     }
58     std::cout << std::endl << " " << k << " ";
59     std::cout << array[BFPRT(0, length - 1, k)] << std::endl;
60     return 0;
61 }

```

2.4.2 Simple sort

All sort algorithm don't compare equal relationship. So some algorithm is Stable, and other is not stable. Bubble is simple source code stable sorting. All the sorting is from minimum to maximum default.

for all sort comparsion, there is good ref website:

<https://www.toptal.com/developers/sorting-algorithms/selection-sort>

Selection sort is an in-place comparison sort. $O(n^2)$. The algorithm finds the minimum value, swaps it with the value in the first position. And repeat From the comparisons presented here, one might

conclude that selection sort should never be used. It does not adapt to the data in any way (notice that the four animations above run in lock step), so its runtime is always quadratic. However, selection sort has the property of minimizing the number of swaps. In applications where the cost of swapping items is high, selection sort very well may be the algorithm of choice. Selection sort only need $O(n)$ write, so if write is expensive operation, you need to use it.

```

1 For (int size=n ; size >1; size —){
2     Int j = Max(a, size);
3     Swap(a[j], a[size-1]);
4 }

```

Insertion sort is suitable for small list and mostly sorted list. Although it is one of the elementary sorting algorithms with $O(n^2)$ worst-case time, insertion sort is the algorithm of choice either when the data is nearly sorted (because it is adaptive) or when the problem size is small (because it has low overhead). For these reasons, and because it is also stable, insertion sort is often used as the recursive base case (when the problem size is small) for higher overhead divide-and-conquer sorting algorithms, such as merge sort or quick sort. Insertion sort is also stable sort.

```

1 For (int I = 0; i < n; i++){
2     Int iv = a[i];
3     Insert(a, I, iv);
4 }
5 Insert(a[], int n, int lv){
6     For (int I = n-1; i >= 0 && lv < a[i]; i —)
7         A[i+1] = a[i] //move each element from tail to head one by one
8     A[i] = lv; //insert here.
9 }

```

Bubble sort is stable. Bubble sort can stop after reaching a sorted array. In the best case (already sorted), every insert requires constant time. So Bubble and insert can reach $O(n)$ time in Best context. Bubble sort has many of the same properties as insertion sort, but has slightly higher overhead. In the case of nearly sorted data, bubble sort takes $O(n)$ time, but requires at least 2 passes through the data (whereas insertion sort requires something more like 1 pass).

```

1 For (int I = n; i > 0; i —){
2     For (int j = 0; j < I; j++)
3         If (a[j] < a[j+1]) swap(a[j], a[j+1]);
4 }

```

In insertion sort elements are bubbled into the sorted section, while in bubble sort the maximums are bubbled out of the unsorted section.

```

1 //insert sort
2 sorted | unsorted
3 1 3 5 8 | 4 6 7 9 2
4 1 3 4 5 8 | 6 7 9 2
5
6 bubble sort
7 unsorted | biggest
8 3 1 5 4 2 | 6 7 8 9
9 1 3 4 2 | 5 6 7 8 9

```

Difference between insertion sort and selection sort? The real question is the speed of comparison vs. copying. The time a selection sort will win is when a comparison is a lot faster than copying. Just for example, let's assume two fields: a single int as a key, and another megabyte of data attached to it. In such a case, comparisons involve only that single int, so it's really fast, but copying involves the entire

megabyte, so it's almost certainly quite a bit slower. Since the selection sort does a lot of comparisons, but relatively few copies, this sort of situation will favor it. The insertion sort does a lot more copies, so in a situation like this, the slower copies will slow it down quite a bit.

As far as worst case for a insertion sort, it'll be pretty much the opposite – anything where copying is fast, but comparison is slow. There are a few more cases that favor insertion as well, such as when elements might be slightly scrambled, but each is still within a short distance of its final location when sorted. In another word, insertion sort is more efficient than selection sort when the input array is partially sorted or almost sorted.

If the data doesn't provide a solid indication in either direction, chances are pretty decent that insertion sort will work out better.

2.4.3 quick sort

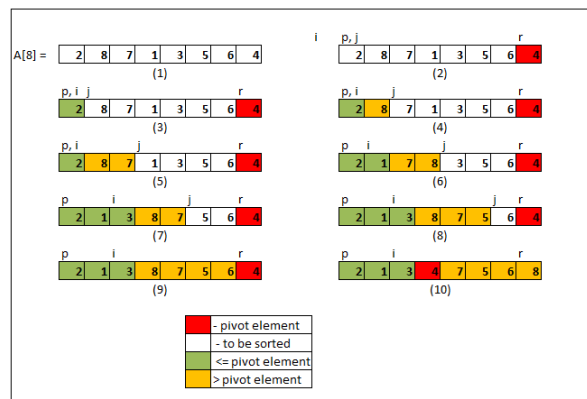
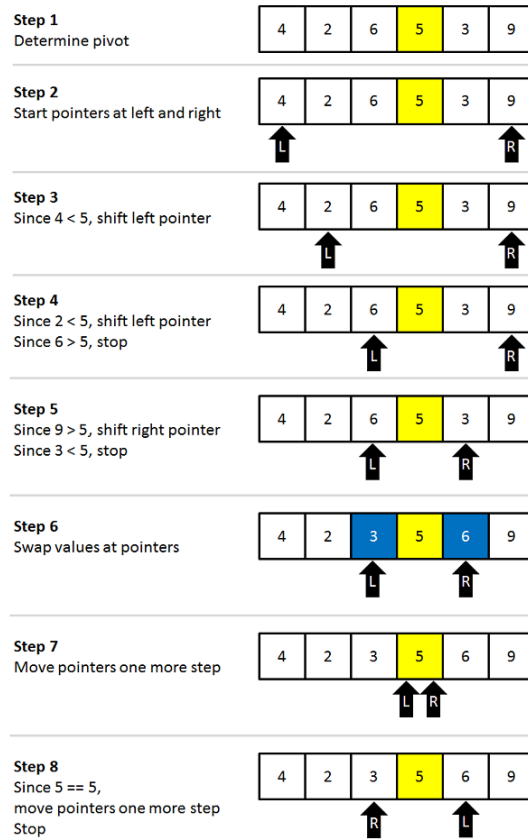
2.4.3.1 Quick sort

Three quick heapSort, MergeSort, quickSort. The time complexity are all ($O(\log n)$). MergeSort need $O(N)$ extra space, When you merge two lists, this shortcoming can be avoid.

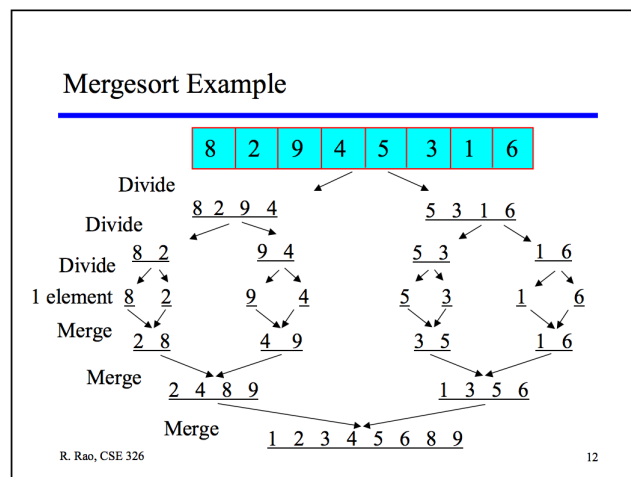
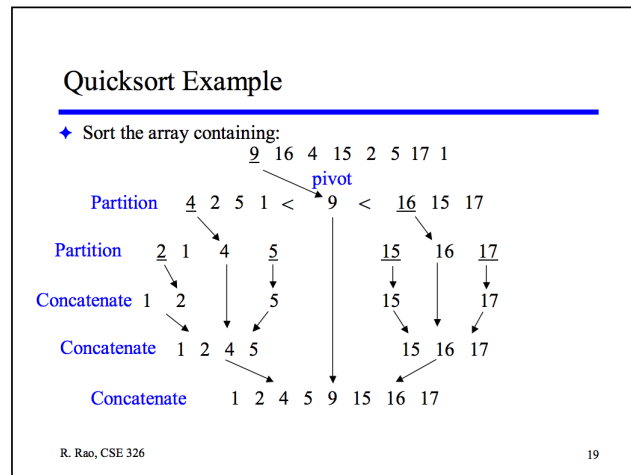
partition

```

1  /* This function takes last element as pivot, places
2  the pivot element at its correct position in sorted
3  array, and places all smaller (smaller than pivot)
4  to left of pivot and all greater elements to right
5  of pivot */
6
7  partition (arr[], low, high){
8      // pivot (Element to be placed at right position)
9      pivot = arr[high];
10     i = (low - 1) // Index of smaller element
11     for (j = low; j <= high- 1; j++) {
12         // If current element is smaller than or
13         // equal to pivot
14         if (arr[j] <= pivot) {
15             i++; // increment index of smaller element
16             swap arr[i] and arr[j]
17         }
18     }
19     swap arr[i + 1] and arr[high])
20     return (i + 1)
21 }
```



Quick sort need random access ability.



2.4.3.2 Merge sort

Merge sort is often preferred for sorting a linked list. The slow random-access performance of a linked list makes some other algorithms (such as quicksort) perform poorly, and others (such as heapsort) completely impossible. MergeSort(headRef)

- 1) If head is NULL or there is only one element in the Linked List then return.
- 2) Else divide the linked list into two halves.
FrontBackSplit(head, &a, &b); /* a and b are two halves */
- 3) Sort the two halves a and b.
MergeSort(a);
MergeSort(b);
- 4) Merge the sorted a and b (using SortedMerge() discussed here) and update the head pointer using headRef.
*headRef = SortedMerge(a, b);

Idea: 1) No random access 2) recursive

```

1  /* sorts the linked list by changing next pointers (not data) */
2  void MergeSort(struct node** headRef)
3  {
4      struct node* head = *headRef;
5      struct node* a, b;
6
7      /* Base case -- length 0 or 1 */
8      if ((head == NULL) || (head->next == NULL)){
9          return;
10     }
11
12     /* Split head into 'a' and 'b' sublists */
13     FrontBackSplit(head, &a, &b);
14
15     /* Recursively sort the sublists */
16     MergeSort(&a);
17     MergeSort(&b);
18
19     /* answer = merge the two sorted lists together */
20     *headRef = SortedMerge(a, b);
21 }

```

```

1  /* See http://geeksforgeeks.org/?p=3622 for details of this function */
2  struct node* SortedMerge(struct node* a, struct node* b){
3      struct node* result = NULL;
4      /* Base cases */
5      if (a == NULL)
6          return(b);
7      else if (b==NULL)
8          return(a);
9
10     /* Pick either a or b, and recur */
11     if (a->data <= b->data){
12         result = a;
13         result->next = SortedMerge(a->next, b);
14     }
15     else{
16         result = b;
17         result->next = SortedMerge(a, b->next);
18     }
19     return(result);
20 }

```

Source code: Idea: recursive

Inplace merge

```

1  void swapNums(vector<int>&ivNums, int lo, int hi){
2      --hi;
3      while (lo<hi){
4          swap(ivNums[lo++], ivNums[hi--]);
5      }
6  }
7
8  void MergeInPlace(vector<int>&ivNums, int lo, int mid, int hi){
9      int i = lo, j = mid;
10     while (i<mid && j<hi){ //stop when the first or latter half is empty
11         while (i < j&&ivNums[i] <= ivNums[j]){
12             ++i;
13         }

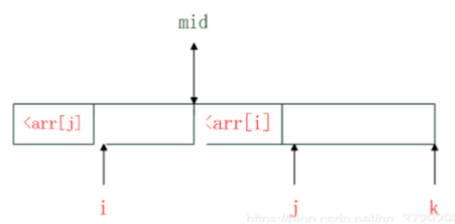
```



```

14  while (j < hi && ivNums[i] > ivNums[j]) {
15      ++j;
16  }
17  //just std::rotate
18  swapNums(ivNums, i, j);
19  swapNums(ivNums, i, i + j - mid);
20  swapNums(ivNums, i + j - mid, j);
21
22  //reset the boundary and continue.
23  i = i + j - mid;
24  mid = j;
25  }
26  }

```



2.4.3.3 heap sort

The most important thing is heapify, then push the maximum element to the 0 index, then swap to the last position, then heapify the (0, len-1), then (0, len-2)....

```

1  // To heapify a subtree rooted with node i which is
2  // an index in arr[]. n is size of heap
3  void heapify(int arr[], int n, int i)
4  {
5      int largest = i; // Initialize largest as root Since we are using 0 based indexing
6      int l = 2 * i + 1; // left = 2*i + 1
7      int r = 2 * i + 2; // right = 2*i + 2
8
9      // If left child is larger than root
10     if (l < n && arr[l] > arr[largest])
11         largest = l;
12
13     // If right child is larger than largest so far
14     if (r < n && arr[r] > arr[largest])
15         largest = r;
16
17     // If largest is not root
18     if (largest != i) {
19         swap(arr[i], arr[largest]);
20
21         // Recursively heapify the affected sub-tree
22         heapify(arr, n, largest);
23     }
24 }
25
26 // main function to do heap sort
27 void heapSort(int arr[], int n)
28 {
29     // Build heap (rearrange array)
30     for (int i = n / 2 - 1; i >= 0; i--)

```

```

31  heapify(arr, n, i);
32
33  // One by one extract an element from heap
34  for (int i = n - 1; i >= 0; i--) {
35      // Move current root to end
36      swap(arr[0], arr[i]);
37
38      // call max heapify on the reduced heap
39      heapify(arr, i, 0);
40  }
41  }

```

2.4.3.4 introsort

This is the method used in STL.

```

1  // A Utility function to perform intro sort
2  void IntrosortUtil(int arr[], int * begin, int * end, int depthLimit){
3      // Count the number of elements
4      int size = end - begin;
5
6      // If partition size is low then do insertion sort
7      if (size < 16){
8          InsertionSort(arr, begin, end);
9          return;
10     }
11
12     // If the depth is zero use heapsort
13     if (depthLimit == 0){
14         make_heap(begin, end+1);
15         sort_heap(begin, end+1);
16         return;
17     }
18
19     // Else use a median-of-three concept to
20     // find a good pivot
21     int * pivot = MedianOfThree(begin, begin+size/2, end);
22
23     // Swap the values pointed by the two pointers
24     swapValue(pivot, end);
25
26     // Perform Quick Sort
27     int * partitionPoint = Partition(arr, begin-arr, end-arr);
28     IntrosortUtil(arr, begin, partitionPoint-1, depthLimit-1);
29     IntrosortUtil(arr, partitionPoint+1, end, depthLimit-1);
30
31     return;
32 }

```

2.4.4 Bucket and radix sort

Bucket sort is not comparison algorithm. we must know the maximum value in the unsorted array and how many objects in the unsorted array. If maximum value is not big, and objects is a lot. such as all student score (<100) in a country(1000000). At this time, we can use Bucket sort.

Firstly, we must know how to handle duplicates. Secondly, Thirdly, we must have enough memory. If you sort telephone which will not allow duplicated value, you can build an bit array. And set bit 1

if there are number exist. It will save a lot of memories. For duplicate, you can store a link in each bucket. (array item)

Radix sort is cousin of Bucksort. BucketSort is more efficient for 'Dense' arrays, while RadixSort can handle sparse (well, not exactly sparse, but spaced-out) arrays well. Use % operator to get radix each digit.

$1 \sim n^c - 1$, such as $1 \sim 10^3 - 1$ You can use 10 as radix, and sort it 3 times.

2.4.4.1 Bucket sort

Basic step: Here, Link list is best option to implement a bucket. Because Nobody will now how many elements will be put into the each bucket.

1. Set up an array of initially empty "buckets".
2. Scatter: Go over the original array, putting each object in its bucket.
3. Sort each non-empty bucket.(Optional)
4. Gather: Visit the buckets in order and put all elements back into the original array.

2.4.4.2 Radix sort

r is radix, $x\%r$, $(x\%r^2)/r$, $(x\%r^3)/r^2$ get each digit from least significant digit (LSD) radix sorts to most significant digit.

```

1 int i ;
2 int k;
3 while(i%10){
4     k = i%10;
5     i = i/10;
6 }
```

1. according to r , prepare two arrays buckets A, B.
2. Sort according to one position and put them into A array buckets.
3. loop through A array buckets, and get next position digit. and then according to its value put into B array buckets.
4. swap A,B two array, then goto step 2.

Difference between radix sort and quick sort. Quicksort/Introsort is more flexible:Quicksort and Introsort work well with all kinds of data. All you need for sorting is the possibility to compare items. This is trivial with numbers but you can sort other data as well. Radix sort on the other hand just sorts things by their binary representation. It never compares items against each other. Radix sort needs more memory. All radix sort implementations that I've seen use a secondary buffer to store partial sorting results. This increases the memory requirements of the sorting algorithm. That may not be a problem if you only sort a couple of kilobytes, but if you go into the gigabyte range it makes a huge difference. If I remember right a in place radix-sort algorithm exist on paper though.

```

1 void countSort(int arr[], int n, int exp){
2
3     int output[n]; // Output array
4     int i, count[10] = { 0 };
5
6     // Store count of occurrences in count[]
7     for (i = 0; i < n; i++)
8         count[(arr[i] / exp) % 10]++;
9 }
```

```

10 // Change count[i] so that count[i] now contains actual position
11 // of this digit in output[]
12 for (i = 1; i < 10; i++)
13     count[i] += count[i - 1];
14
15 // Build the output array
16 for (i = n - 1; i >= 0; i--) {
17     output[count[(arr[i] / exp) % 10] - 1] = arr[i];
18     count[(arr[i] / exp) % 10]--;
19 }
20
21 // Copy the output array to arr[], so that arr[] now contains sorted
22 // numbers according to current digit
23 for (i = 0; i < n; i++)
24     arr[i] = output[i];
25 }
26
27 // The main function to that sorts arr[] of size n using Radix Sort
28 void radixsort(int arr[], int n){
29
30     // Find the maximum number to know number of digits
31     int m = getMax(arr, n);
32
33     // Do counting sort for every digit. Note that instead of passing digit
34     // number, exp is passed. exp is 10^i where i is current digit number
35     for (int exp = 1; m / exp > 0; exp *= 10)
36         countSort(arr, n, exp);
37 }

```

In order to understand previous code, you need to understand counting sort, Detail can be found here:
<https://www.geeksforgeeks.org/counting-sort/> // it use prefix sum(std::partial_sum). it's also interesting application of std::partial_sum, you need to remember it.

2.4.5 Summary

You can compare sorting algorithms against the following criteria:

- Time Complexity (Big-O notation). You should note that best-case, worst-case and average run-time can have different time complexity. For example best-case for Bubble Sort is only $O(n)$, making it faster than Selection Sort when the original list is mostly in order (not many elements out of place).
- Memory Complexity. How much more memory is required to sort a list as n grows?
- Stability. Does the sort preserve the relative ordering of elements that have equivalent sort values? (For example if you were sorting a list of catalog items by their price, some elements may have equal prices. If the catalog was originally sorted alphabetically by item name, will the chosen sort algorithm preserve the alphabetical ordering within each group of equal-priced items.)
- Best/Worst/Average number of comparisons required. Important when compare operations are expensive. (For example: comparing efficiencies of alternative designs where efficiency is calculated via some simulation or otherwise complex calculation).
- Best/Worst/Average number of swap operations required. Important when swap operations are expensive. (For example: sorting shipping containers that must be physically moved on the deck of a ship)

- Code size. Bubble-sort is known for its small code footprint.

2.5 Greedy

Idea of Greedy:

- Solve a problem by making a sequence of decisions.
- Decisions are made one by one in some order.
- Each decision is made using a greedy criterion. At each stage we make a decision that appears to be the best at the time.
- A decision, once made, is (usually) not changed later.

You need a **greedy criterion** to make a local decision. For examples: making change, 0/1 knapsack, activity selection (largest subset) sorted according to their finishing time, topological orders. Dijkstra, Kruskal, prim and collin.

Applications: Container Loading, 0/1 knapsack problem, Topological sorting, Bipartite cover, Single-source shortest paths, Minimum-cost spanning trees.

2.5.0.1 Intervals questions

two basic tricks, sort, then do something when it overlap.

Meeting room questions.

1. Minimum empty time. That is 0/1 knapsack problem.
2. Two group collision intervals.

Overlap (three questions):

1. Non-overlap number. one meeting room, maximum meeting number. Sort by end time, delete(ignore) if overlap.

```

1 public int intervalSchedule(int [][] intvs) {
2     if (intvs.length == 0) return 0;
3     // sort
4     Arrays.sort(intvs, new Comparator<int[]>() {
5         public int compare(int[] a, int[] b) {
6             return a[1] - b[1];
7         }
8     });
9
10    int count = 1;
11    int x_end = intvs[0][1];
12    for (int[] interval : intvs) {
13        int start = interval[0];
14        if (start >= x_end) {
15            // find next non-overlap interval,
16            // update x_end.
17            count++;
18            x_end = interval[1];
19        }
20    }
21    return count;
22 }
```

2. Maximum overlap number. Minimum meeting room numbers. sort by start time. you can use scan line.

- Sort the intervals by their start times.
- Use a min-heap to keep track of the end times of meetings currently using a room.
- Iterate through the sorted intervals and for each interval:
 - (a) If the room due to free up the earliest is free, remove it from the heap.
 - (b) Add the current meeting's end time to the heap.
 - (c) The size of the heap at any point will give the number of rooms required at that time.

```

1 // Function to find the minimum number of meeting rooms required
2 int minMeetingRooms(vector<vector<int>>& intervals) {
3     if (intervals.empty()) return 0;
4
5     // Sort the intervals by start time
6     sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<
7         int>& b) {
8         return a[0] < b[0];
9     });
10
11    // Min-heap to keep track of end times
12    priority_queue<int, vector<int>, greater<int>> minHeap;
13
14    // Add the end time of the first meeting
15    minHeap.push(intervals[0][1]);
16
17    for (int i = 1; i < intervals.size(); ++i) {
18        // If the room due to free up the earliest is free, remove it from the heap
19        if (intervals[i][0] >= minHeap.top()) {
20            minHeap.pop();
21        }
22        // Add the current meeting's end time to the heap
23        minHeap.push(intervals[i][1]);
24    }
25
26    // The size of the heap is the number of rooms required
27    return minHeap.size();
28 }
29
30 int main() {
31     vector<vector<int>> intervals = {{5, 10}, {0, 30}, {15, 20}};
32     cout << "Minimum_number_of_meeting_rooms_required:_ " << minMeetingRooms(
33         intervals) << endl;
34     return 0;
35 }

```

3. Merge overlap. sort by left. If overlap, then update right point; if non-overlap, find a new current intervals.

```

1 vector<vector<int>> mergeOverlap(vector<vector<int>>& arr) {
2
3     // Sort intervals based on start values
4     sort(arr.begin(), arr.end());
5
6     vector<vector<int>> res;
7     res.push_back(arr[0]);
8
9     for (int i = 1; i < arr.size(); i++) {
10         vector<int>& last = res.back();
11         vector<int>& curr = arr[i];

```

```

12
13     // If current interval overlaps with the last merged
14     // interval, merge them
15     if (curr[0] <= last[1])
16         last[1] = max(last[1], curr[1]);
17     else
18         res.push_back(curr);
19 }
20
21 return res;
22 }
23
24 int main() {
25     vector<vector<int>> arr = {{7, 8}, {1, 5}, {2, 4}, {4, 6}};
26     vector<vector<int>> res = mergeOverlap(arr);
27
28     for (vector<int>& interval: res)
29         cout << interval[0] << " " << interval[1] << endl;
30
31     return 0;
32 }

```

Interval cover. (two questions)

1. Minimum cover

```

1 int videoStitching(int[][] clips, int T) {
2     if (T == 0) return 0;
3     // Sort in ascending order by the start point, and in descending order if the
4     // start points are the same.
5     Arrays.sort(clips, (a, b) -> {
6         if (a[0] == b[0]) {
7             return b[1] - a[1];
8         }
9         return a[0] - b[0];
10    });
11    // Keep track of the number of selected short videos
12    int res = 0;
13
14    int curEnd = 0, nextEnd = 0;
15    int i = 0, n = clips.length;
16    while (i < n && clips[i][0] <= curEnd) {
17        // Greedily select the next video within the interval of the res-th video.
18        while (i < n && clips[i][0] <= curEnd) {
19            nextEnd = Math.max(nextEnd, clips[i][1]);
20            i++;
21        }
22        // Find the next video and update curEnd.
23        res++;
24        curEnd = nextEnd;
25        if (curEnd >= T) {
26            // The interval [0, T] can already be assembled.
27            return res;
28        }
29    }
30    // Unable to continuously assemble the interval [0, T].
31    return -1;
32 }

```

2. Erase cover. Once find overlap, update right, find non-overlap, update left and right.

```

1 int removeCoveredIntervals(int [][] intvs) {
2     // Sort in ascending order by the start point, and in descending order if the
      start points are the same.
3     Arrays.sort(intvs, (a, b) -> {
4         if (a[0] == b[0]) {
5             return b[1] - a[1];
6         }
7         return a[0] - b[0];
8     });
9
10    // Record the start and end points of the merged intervals.
11    int left = intvs[0][0];
12    int right = intvs[0][1];
13
14    int res = 0;
15    for (int i = 1; i < intvs.length; i++) {
16        int[] intv = intvs[i];
17        // Case 1: Find the covering intervals.
18        if (left <= intv[0] && right >= intv[1]) {
19            res++;
20        }
21        // Case 2: Find the overlapping intervals and merge them.
22        if (right >= intv[0] && right <= intv[1]) {
23            right = intv[1];
24        }
25        // Case 3: Completely non-overlapping, update the start and end points.
26        if (right < intv[0]) {
27            left = intv[0];
28            right = intv[1];
29        }
30    }
31
32    return intvs.length - res;
33 }

```

2.5.1 Application

Machine Scheduling, sort all task according to start time, and minheap of machine end time.

Container Loading and 0/1 knapsack problem are different. For 0/1 knapsack problem, we consider the value and weight together, For greed algorithm, it change time complexity from 2^n to $n \cdot \log(n)$, but 583/600 is withing 10%, so it's very practical algorithm. Don't underestimate it.

Topological sorting:

- 1) Select any one among vertices having no incoming edge
- 2) Put the node into the solution & Remove the node and its outgoing edges from the graph
- 4) Repeat the above steps until no nodes remain

You need to InDegree[n] array to save and update all the vetex InDegree, and stack to store all

Bipartite-cover problems are NP-hard

A greedy method to develop a fast heuristic

Construct the cover A' in stages

Select a vertex of A using the greedy criterion:

Select a vertex of A that covers the largest # of uncovered vertices of B

Dijkstra algorithm.

```

1:  function Dijkstra(Graph, source):
2:      for each vertex v in Graph:           // Initialization
3:          dist[v] := infinity               // initial distance from source to vertex v is set to infinite
4:          previous[v] := undefined          // Previous node in optimal path from source
5:      dist[source] := 0                     // Distance from source to source
6:      Q := the set of all nodes in Graph    // all nodes in the graph are unoptimized - thus are in Q
7:      while Q is not empty:                 // main loop
8:          u := node in Q with smallest dist[ ]
9:          remove u from Q
10:         for each neighbor v of u:         // where v has not yet been removed from Q.
11:             alt := dist[u] + dist_between(u, v)
12:             if alt < dist[v]               // Relax (u,v)
13:                 dist[v] := alt
14:                 previous[v] := u
15:     return previous[ ]

```

- The greedy idea lies in line 8
- In line 8, it must shortest path to this vertex, Because if it's not, There is shorter path, then, according to we select min in line 8, shorter one will be selected first, and this value HAS BEEN updated before.
- $O(n^2)$
- the shortest value is saved, it's also idea of dynamic programming.
- Just like vertabi, you can think that they are the same.

2.6 Divide Conquer

2.6.1 Binary search

A binary search (or divide-and-conquer) idea can be applied to the problem of finding the closest pair of points on a plane. By utilizing a subproblem, you can prune some candidate points. Another application is using merge sort to count the number of smaller elements to the right of each element, which is introduced by labuladong. The fundamental idea is still binary search, but the specifics come from the fact that merge sort maintains order, and this characteristic can be used to count the number of smaller elements on the right. This is a rather unique application.

The key idea here is leveraging the sorting order inherent in merge sort to solve a problem related to counting elements, and it also shows the pruning advantage in divide-and-conquer algorithms.

Binary search.

```

1  int binarySearch(int arr[], int target, int i, int j) {
2      int left = i;
3      int right = j;
4      while ( left <= right ) {
5          int mid = (left+right)/2;
6          if (arr[mid] > target)
7              right = mid - 1;
8          else if (arr[mid] < target)
9              left = mid + 1;
10         else if (arr[mid] == target)

```

```

11     return mid;
12 }
13 return -1;
14 }

```

```

1 int left_bound(int arr[], int target, int i, int j) {
2     int left = i;
3     int right = j;
4     while ( left <= right) {
5         int mid = (left+right)/2;
6         if (arr[mid] > target)
7             right = mid - 1;
8         else if (arr[mid] < target)
9             left = mid + 1;
10        else if (arr[mid] == target)
11            right = mid-1; //1) reduce to right in order to find left bound
12    }
13
14    if (left > j || arr[left] != target)
15        return -1; //3) in order to return left, left will go to the right boundary,
16    return left; //2) return left.
17 }

```

```

1 int right_bound(int arr[], int target, int i, int j) {
2     int left = i;
3     int right = j;
4     while ( left <= right) {
5         int mid = (left+right)/2;
6         if (arr[mid] > target)
7             right = mid - 1;
8         else if (arr[mid] < target)
9             left = mid + 1;
10        else if (arr[mid] == target)
11            left = mid+1; //1) step left to find right bound
12    }
13
14    if (right < 0 || arr[right] != target)
15        return -1; //3) in order to return right, right will be too small
16    return right; //2) return right.
17 }

```

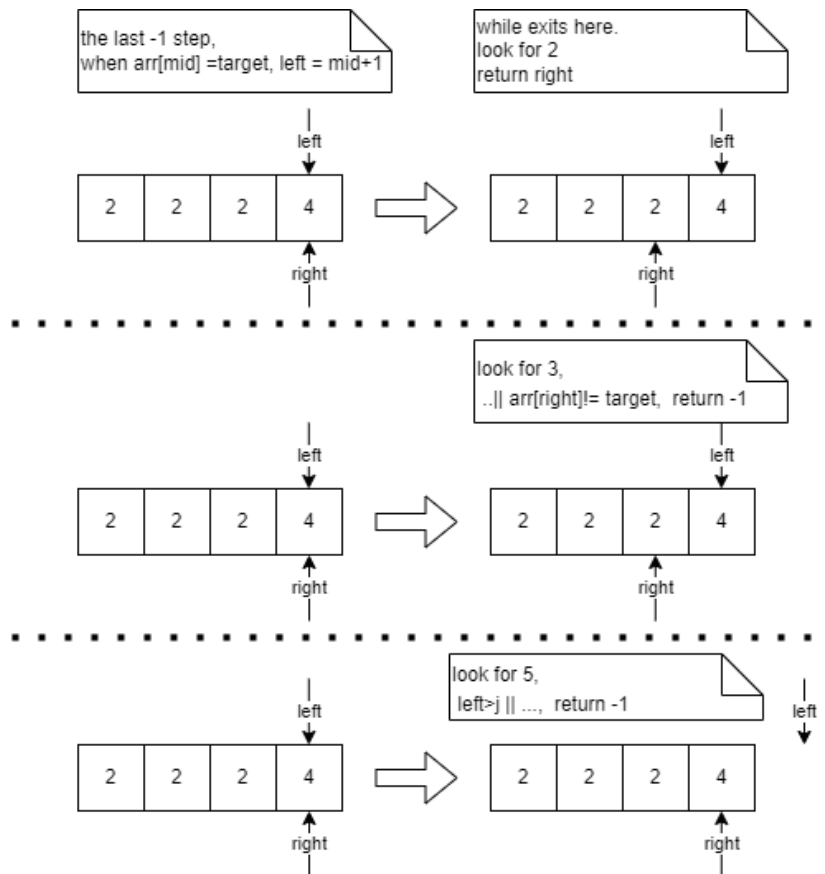
Three methods summary are below:

- close left and right
- left bound reduce right, right bound add left;
- left bound return left, right bound return right.
- left may be too big, right maybe too small.
- if no found,

```

1 //if we want to find 4, then right point to 3, left point to 5
2 //when we quit while loop.
3 //so left_bound return the 5 (smallest bigger than target),
4 //right bound return 3( biggest smaller than target.)
5 right left
6     3     5

```



Right boundary logic can be used to found square root.

```

1  int mySqrt(int x) {
2      int l = 0;
3      int r = x/2;
4      while(l<=r){
5          int mid = l + (r-l)/2;
6          int temp = mid*mid;
7          if(temp<x){
8              l = mid+1;
9          }
10         else if(temp>x){
11             r = mid-1;
12         }
13         else if(temp == x){
14             r = mid;
15             break;
16         }
17     }
18     return r; //round down value, for 8, result is 2
19     return l; //round up value, for 8, result is 3
20 }
21
22 1
23 2 3    // (l+r)/2 is till l
24     r
25
26 1
27 2 3    //so ++l,
28     r

```

```

29 1
30 1
31 2 3 //mid>x, so --r; l<r, while end.
32 r //at this time l points 3, r points 2.

```

2.6.2 Other

An element in a sorted array can be found in $O(\log n)$ time via binary search. But suppose we rotate an ascending order sorted array at some pivot unknown to you beforehand. So for instance, 1 2 3 4 5 might become 3 4 5 1 2. Devise a way to find an element in the rotated array in $O(\log n)$ time.

The basic idea is DC(divided conquer.)

1. Use DC, The first part is end condition. For this question, the end atomic condition is $e-b == 1$ (only two elements in it)
2. If use array, better use index directly.
3. When you have last atomic condition, you need to decide boundary b , $(e-b)/2$, e . **When you decide boundary, you have to make half part of contents still includes answer. That is very useful hint for you to decide boundary value**

```

1 size_t find_rot(const vector<int> &vi, int b, int e){
2     if (e-b ==1){
3         return b;
4     }
5     int pivot = b + (e-b)/2;
6     cout<<"pivot"<<pivot<<endl;
7     if (vi[b]<vi[pivot]){
8         return find_rot(vi, pivot, e);
9     }
10    else{
11        return find_rot(vi, b, pivot);
12    }
13 }

```

label If you can divide it according certain condition, then you can use DC

2.7 Backtracking

2.7.1 Basic

Backtracking is particularly useful for solving problems where you need to explore all possible solutions, but you can prune certain paths that are unlikely to lead to a valid solution (i.e., "backtrack" when a partial solution cannot be completed). Problems suitable for backtracking typically involve making a sequence of decisions and can be described as "constraint satisfaction" problems.

Here are common types of questions suitable for backtracking:

- Combinatorial Problems Permutations and Combinations: Finding all permutations or combinations of a set of numbers or characters. Example: Generate all permutations of a given array of numbers. Subsets: Generating all subsets of a set. Example: Finding all subsets of a given set of numbers (power set).
- Constraint Satisfaction Problems (CSPs) Sudoku Solver: Filling in numbers on a Sudoku board while satisfying the row, column, and grid constraints. N-Queens Problem: Placing N queens on

an $N \times N$ chessboard so that no two queens attack each other. Crossword Puzzle Filler: Filling a crossword grid while ensuring valid word placements.

- Graph and Tree Search Problems Maze Solver: Finding a path from start to finish in a maze. Hamiltonian Path/Cycle: Finding a path or cycle in a graph that visits every vertex exactly once. Word Search: Searching for words in a grid of characters by moving horizontally, vertically, or diagonally.
- Partitioning Problems Partition to K Equal Sum Subsets: Partitioning an array into K subsets such that the sum of elements in each subset is equal. Palindrome Partitioning: Partitioning a string into substrings, where each substring is a palindrome.
- Combinatorial Optimization Problems Knapsack Problem: Finding combinations of items that maximize value without exceeding the capacity of a knapsack (although dynamic programming is often more efficient for this problem). Job Scheduling: Scheduling jobs on a limited number of resources (machines) without conflicts.
- String Matching Regular Expression Matching: Checking if a string matches a given pattern with wildcard characters.
- Puzzle Solving Crossword or Word Puzzle: Solving word puzzles by filling blanks with valid words from a dictionary. Solving Puzzles like the 8-Puzzle: Rearranging pieces on a grid to reach a specific target configuration.

In all these cases, backtracking explores the possible decisions recursively but abandons a path (backtracks) if it becomes clear that it cannot yield a valid solution, thus reducing unnecessary computation.

The basic backtracking code

```

1 result = []
2 def backtrack(track, select_list):
3     if end_condition:
4         result.add(track)
5         return
6
7     for option in select_list:
8         select one
9         backtrack(track, select_list)
10        revert the select

```

If we don't want to return all the result, but one, we can change it to below code.

```

1 result = []
2 bool backtrack(track, select_list):
3     if end_condition:
4         result.add(track)
5         return true;
6
7     for option in select_list:
8         select one
9         if(backtrack(track, select_list))
10            return true;
11        revert the select

```

Line9-10 This if will stop the recursive and return the true direct to the root.

If we want to remember all the result, we need to List<List>. List inside remember a track, List outside remember all the valid result.

```

1 List<List<Integer>> res = new LinkedList<>();
2 LinkedList<Integer> track = new LinkedList<>(); //remember recursive path
3

```

```

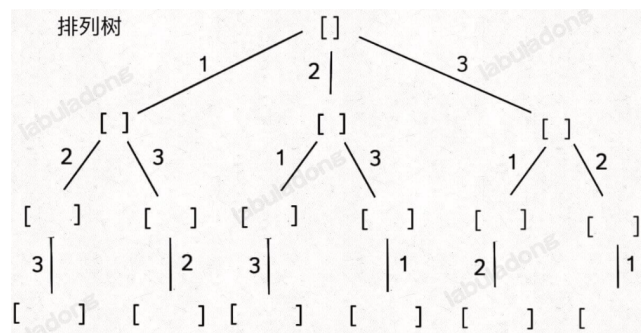
4 public List<List<Integer>> subsets(int[] nums) { // main function
5     backtrack(nums, 0);
6     return res;
7 }
8
9 void backtrack(int[] nums, int start) {
10     res.add(new LinkedList<>(track));
11     for (int i = start; i < nums.length; i++) { // main frame
12         track.addLast(nums[i]); // select
13         backtrack(nums, i + 1); // use start to trim
14         track.removeLast(); // unselect
15     }
16 }

```

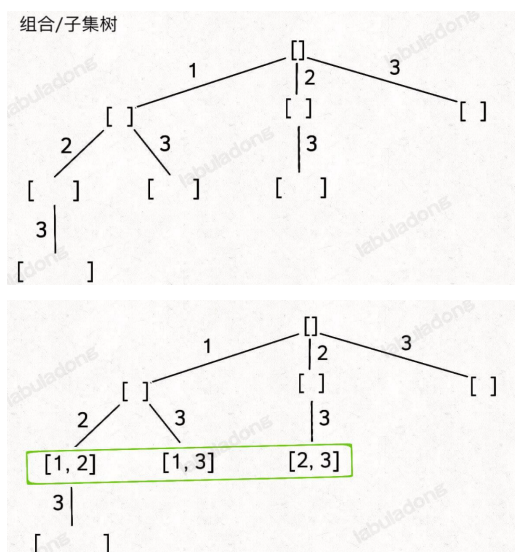
2.7.2 Permutation, combination and subset

2.7.2.1 No duplicate, single select

Two important tree structures.



Combination can be seen as a child question of subset. permutation $n!$, subset is 2^n , combination is $c(n, k)$. six elements. $5! = 120$, $2^5 = 32$. $c(5, 2) = 10$.



Subset problem.

1. Each node(from current node to root,including root(empty set)) is an answer

2. Don't need base case here. when $i \geq \text{len}$, for will return without any execution.
3. Refer the combination tree figure to help to understand this code

```

1 void sub(int arr[], int len, int start, vector<int>& result) {
2     cout << "one_answer_is_(_";
3     for (int i = 0; i < result.size(); i++) {
4         cout << result[i] << "_";
5     }
6     cout << ")" << endl;
7
8     for (int i = start; i < len; ++i) {
9         result.push_back(arr[i]);
10        sub(arr, len, i + 1, result);
11        result.pop_back();
12    }
13 }
14
15 sub(arr, 3, 0, result)

```

Combination problem, the same idea as subset, just add a parameter k. When one answer length is k, print it out.

```

1 void com(int arr[], int len, int start, int k, vector<int>& result) {
2     if (result.size() == k) {
3         cout << "one_answer_is_(_";
4         for (int i = 0; i < result.size(); i++) {
5             cout << result[i] << "_";
6         }
7         cout << ")" << endl;
8     }
9
10    for (int i = start; i < len; ++i) {
11        result.push_back(arr[i]);
12        com(arr, len, i + 1, k, result);
13        result.pop_back();
14    }
15 }

```

Permutation problem.

- Base case is size == len
- You must use used array here to remember which element you have used.

```

1 void per(int arr[], int len, int used[], vector<int>& result) {
2     if (result.size() == len) {
3         cout << "one_answer_is_(_";
4         for (int i = 0; i < result.size(); i++) {
5             cout << result[i] << "_";
6         }
7         cout << ")" << endl;
8     }
9     for (int i = 0; i < len; ++i) {
10        if (used[i] == 1)
11            continue;
12        result.push_back(arr[i]);
13        used[i] = 1;
14        per(arr, len, used, result);
15        result.pop_back();
16        used[i] = 0;
17    }

```

18 }

For permutation problem, there is another easy implementation. You don't need extra result vector, but you have to use swap. They use the same basic backtrack idea.

```

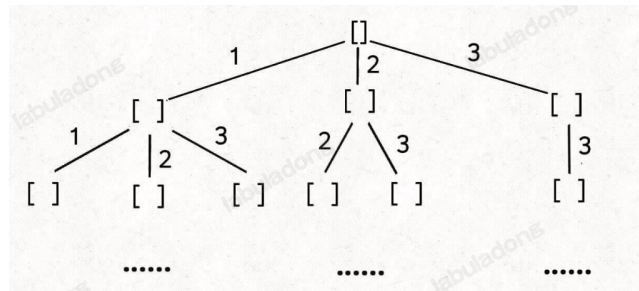
1 //all permutation. here there is a trick, we don't need track, but the basic idea is
  the same.
2 void permute(char a[], int i, int n){
3     int j;
4     if (i == n){ //end_condition,
5         cout << a << endl;
6         return;
7     }
8
9     for (j = i; j <= n; j++){ // all options in select_list
10        swap(a[i], a[j]); //select one
11        permute(a, i+1, n);
12        swap(a[i], a[j]); //revoke select.
13    }
14 }
```

2.7.2.2 No duplicate, multi select

The basic idea is just like before, for combination, we use i+1 to call recursion implementation single select, if you can multi select, just use i here. In order to avoid dead loop, this questions need a target to end loop, A typical question is coin problem.

```

1 void sub_multi_select(int arr[], int len, int start, vector<int>& result, int target) {
2     auto sum = std::accumulate(result.begin(), result.end(), 0);
3     if (sum > target)
4         return;
5     if (sum == target) {
6         cout << "one_answer_is_(_";
7         for (int i = 0; i < result.size(); i++) {
8             cout << result[i] << "_";
9         }
10        cout << ")" << endl;
11    }
12
13    for (int i = start; i < len; ++i) {
14        result.push_back(arr[i]);
15        sub_multi_select(arr, len, i, result, target); //Pay attention here, not pass i+1,
16        //but pass i.
17        result.pop_back();
18    }
19
20    int arr[3] = { 1,2,5};
21    vector<int> result;
22    sub_multi_select(arr, 3, 0, result, 10);
23
24    one answer is ( 1 1 1 1 1 1 1 1 1 1 ) //Ten result
25    .....
26    one answer is ( 1 1 2 2 2 2 )
27    one answer is ( 1 2 2 5 )
28    one answer is ( 2 2 2 2 2 )
29    one answer is ( 5 5 )
```

multi select permutation, Code is very easy, don't use used array. The time complexity is $\text{power}(n, n)$. That is the only question which has so high time complexity.

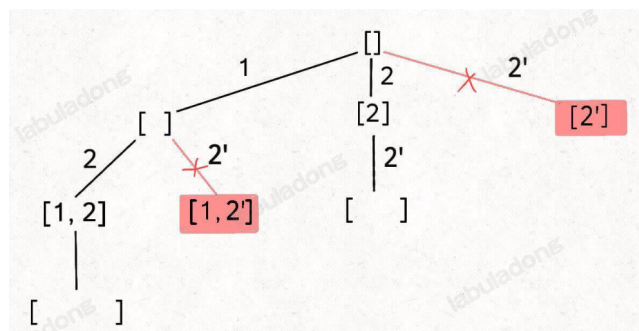
2.7.2.3 Duplicate, single select

For combination, need to sort, then trim(prune) a branch which doesn't meet condition.

```

1  ....
2  for (int i = start; i < len; ++i) {
3      if(i>start && arr[i] == arr[i-1]) // trimming happens in these two
4          continue;                  // statements, so we need to sort first.
5      result.push_back(arr[i]);
6      sub(arr, len, i + 1, result);
7      result.pop_back();
8  }
9  ....

```



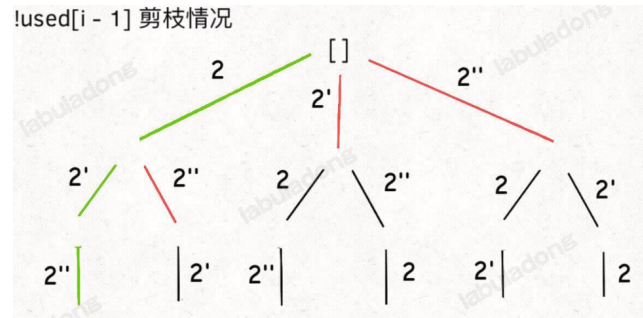
For permutation, need to sort, then trim (prune) a branch which doesn't meet condition.

```

1  ....
2  for (int i = start; i < len; ++i) {
3      if(i>start && arr[i] == arr[i-1] && !used[i-1]) // trimming happens in these two
4          continue;                  // statements, so we need to sort first.
5      result.push_back(arr[i]);
6      sub(arr, len, i + 1, result);
7      result.pop_back();
8  }
9  ....

```

How to understand !used[i-1]?



2.7.2.4 Parenthesis

The trimming idea can be applied in parenthesis problem, the basic problem is also permutation problem, but you need to trim the branch. anytime, left parenthesis should be greater or equal right parenthesis.

```

1 vector<string> generateParenthesis(int n) {
2     vector<string> res; // all results
3     string track;
4     backtrack(n, n, track, res); // left and right parenthesis number are both n
5     return res;
6 }
7
8 void backtrack(int left, int right, string& track, vector<string>& res) {
9
10    if (right < left) return; // left is less right, illegal.
11
12    if (left < 0 || right < 0) return;
13    // all is 0, get a legal result.
14    if (left == 0 && right == 0) {
15        res.push_back(track);
16        return;
17    }
18
19    track.push_back('('); // try to put a left
20    backtrack(left - 1, right, track, res);
21    track.pop_back();
22
23    track.push_back(')'); // try to put a right.
24    backtrack(left, right - 1, track, res);
25    track.pop_back();
26 }
27 // output is ((())) ((()) ) ((()) ) ()(()) ()()()

```

2.7.2.5 Summary

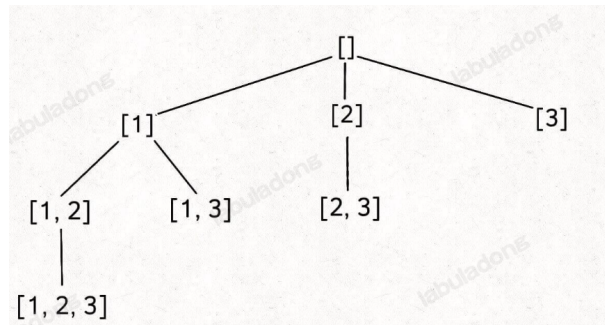
Four code compare: They are the most common used code template for back trace, you need to understand and

1. code 1: multi tree, call backtrack(i), backtrack(i-1), backtrack(i-2)... This will cause a multi branch tree, and the first level has n branches, The second level has n-1 branches, untill the last level has only 1 branches. $O(n*n)$

```

1 for(i .. n )
2     backtrack( i+1)

```

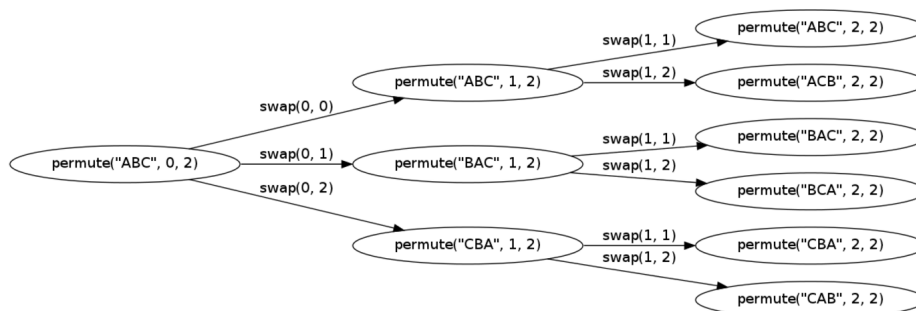


2. code 2: multi tree, on each level, i, i-1, i-2 backtrace $O(n!)$

```

1  for(j = i .. n) // on i level, each backtrace trun (n-i) times.
2  backtrace(i+1)

```



3. code 3: binary tree, global result, need reverse

4. code 4: binary tree, stack partial result, don't need reverse. global result.

2.7.3 Interview questions

Eight queen

```

1  vector<vector<string>> res;
2  vector<vector<string>> solveNQueens(int n) {
3
4  // '.' is void, 'Q' is queen
5  vector<string> board(n, string(n, '.'));
6  backtrack(board, 0);
7  return res;
8  }
9
10
11 void backtrack(vector<string>& board, int row) {
12 // base case
13 if (row == board.size()) {
14     res.push_back(board);
15     return;
16 }
17
18 int n = board[row].size();
19 for (int col = 0; col < n; col++) {
20 // exclude
21 if (!isValid(board, row, col)) {
22     continue;
23 }
24 // make decision

```

```

25     board[row][col] = 'Q';
26     // go deeper
27     backtrack(board, row + 1);
28     // reverse
29     board[row][col] = '.';
30 }
31 }

```

k equal subset.

```

1  class Solution {
2      public:
3      bool canPartitionKSubsets(vector<int>& nums, int k) {
4          int sum = accumulate(nums.begin(), nums.end(), 0);
5          if (sum % k != 0) return false;
6          sort(nums.begin(), nums.end(), greater<int>());
7          vector<bool> visited(nums.size(), false);
8          return helper(nums, k, sum / k, 0, 0, visited);
9      }
10     bool helper(vector<int>& nums, int k, int target, int start, int curSum, vector<bool>
11                & visited) {
12         if (k == 1) return true;
13         if (curSum > target) return false; //return false is used to trim branch
14         if (curSum == target) return helper(nums, k - 1, target, 0, 0, visited);
15         for (int i = start; i < nums.size(); ++i) {
16             if (visited[i]) continue;
17             visited[i] = true;
18             if (helper(nums, k, target, i + 1, curSum + nums[i], visited)) return true;
19             visited[i] = false;
20         }
21         return false;
22     };

```

2.8 Dynamic programming

2.8.1 Basic Idea

Dynamic Programming is a technique for solving problems with overlapping subproblems. Each sub-problem is solved only once and the result of each sub-problem is stored in a table (generally implemented as an array or a hash table) for future references. These sub-solutions may be used to obtain the original solution and the technique of storing the sub-problem solutions is known as memoization. You may think of DP = recursion + re-use

The principle of optimality. No matter what the first decision, the remaining decisions must be optimal with respect to the state that results from this first decision. Dynamic programming may be used only when the principle of optimality holds. Useful when the sub-problems are overlapping. Solve an optimization problem by caching sub-problem solutions rather than recomputing them

- Verify that the principle of optimality holds
- Set up the dynamic-programming recurrence equations.
- Solve the dynamic-programming recurrence equations for the value of the optimal solution.
- Perform a traceback step in which the solution itself is constructed.

Three key words in dynamic programming are: state, transition and choice. state is most time defined by question directly, the most difficult question is **How the current states is transitioned through**

choices from previous states, which is the transition equation. If the state is defined in terms of the problem and you find it difficult to write the transition equation, then you must redefine the state; one example is Given an integer array nums, find the subarray with the largest sum, and return its sum. (<https://leetcode.com/problems/maximum-subarray/description/>)

The top-down approach is generally recursive (but less efficient) and more intuitive to implement as it is often a matter of recognizing the pattern in an algorithm and refactoring it as a dynamic programming solution. The bottom-up approach is generally iterative (and more efficient), but less intuitive and requires us to solve (and know!) the smaller problems first then use the combined values of the smaller problems for the larger solution. We refer to top-down solutions as memorization and bottom-up as tabulation. Memorization is generally more intuitive to implement especially when we don't know the solution to subproblems, whereas tabulation requires us to know the solutions, or bottom, in advance, in order to build our way up.

Difference between DP and BT: The basic question can be asked: can(bool), min or count(one number), all(list or vector). Usually, can and all can be used backtracking algorithm, but max/min is a typical indication of greed or dynamic programming.

Difference between DP and greed: Dynamic Programming often has higher time complexity due to solving all subproblems, typically $O(n^2)$ or $O(n^3)$ for certain problems. Greedy Algorithms Generally faster, with time complexity often $O(n \log n)$ or $O(n)$, depending on the problem.

- Use **DP** when all subproblems need to be solved and combined for the final answer.
- Use **Greedy** when a locally optimal choice at every step leads directly to the global optimum.

Difference between DP and DC: In Divide and Conquer, the sub-problems are independent of each other while in case of Dynamic Programming, the sub-problems are not independent of each other (Solution of one sub-problem may be required to solve another sub-problem). Divide and Conquer works by dividing the problem into sub-problems, conquer each sub-problem recursively and combine these solutions. solves a problem by combining the solutions to sub-problems

- Partition the problem into non-overlapping sub-problems
- Solve the sub-problems recursively
- Combine their solutions to solve the original problem

2.8.2 Three questions

Write a function to return the minimum number of jumps to reach the end of the array (starting from the first element). Below is Min jump question

```
int arr[] = {1, 3, 5, 8, 9, 2, 6, 7, 6, 8, 9};

int minj(int arr[], int last){
    if (last ==1 && arr[last]>=1){
        return 1;
    }

    int min = last;
    for(int i = last-1; i>=0;i--){
        if (i+arr[i]>=last){
            int step = 1+minj(arr, i);
            if( step<min){
                min = step;
            }
        }
    }
}
```

```

    return min;
}

```

Below is getting all questions.

```

int arr[] = {1, 3, 5, 8, 9, 2, 6, 7, 6, 8, 9};

vector<vector<int>> minj(int arr[], int last){
    if (last ==1 && arr[last]>=1){
        vector<int> v1 = {1};
        vector<vector<int>> v = {v1};
        return v;
    }
    vector<vector<int>> result;

    int min = last;
    for(int i = last -1; i>=0;i--){
        if (i+arr[i]>=last){
            vector<vector<int>> result1 = minj(arr, i);
            for(auto e: result1){
                e.push_back(arr[i]);
                result.push_back(e);
            }
        }
    }
    return result;
}

```

below is Can questions.

```

int arr[] = {1, 3, 5, 8, 9, 2, 6, 7, 6, 8, 9};

bool minj(int arr[], int last){

    if (last ==1 && arr[last]>=1){
        return true;
    }

    bool result = false;

    for(int i = last -1; i>=0;i--){
        if (i+arr[i]>=last){
            result = true && minj(arr, i);
            if(result)
                break;
        }
    }
    return result;
}

```

Conclusion:

- can and min is little easy, min return one number, can return one bool.
- "all" type question need return a list, because each answer is a list, so here we return vector<vector>.
- "all" type question is suitable Top-bottom method. can and min is suitable for bottom-top method.

2.8.3 one dimension

Longest Increasing Subsequence, top bottom method. There are a lot of overlap problem, so efficient is not good.

```
int _lis( int arr[], int n, int *max_ref){
    if (n == 1) /* Base case */
        return 1;

    // 'max_ending_here' is length of LIS ending with arr[n-1]
    int res, max_ending_here = 1;

    /* Recursively get all LIS ending with arr[0], arr[1] ... arr[n-2]. If arr[i-1] is
       smaller
       than arr[n-1], and max ending with arr[n-1] needs to be updated, then update it */
    for (int i = 1; i < n; i++){
        res = _lis(arr, i, max_ref);
        if (arr[i-1] < arr[n-1] && res + 1 > max_ending_here)
            max_ending_here = res + 1;
    }

    // Compare max_ending_here with the overall
    // max. And update the overall max if needed
    if (*max_ref < max_ending_here)
        *max_ref = max_ending_here;

    // Return length of LIS ending with arr[n-1]
    return max_ending_here;
}
```

Longest Increasing Subsequence, bottom-top method. There are no overlap problem, so efficiency is better.

```
int lis( int arr[], int n ){
    int lis[n];
    lis[0] = 1;

    /* Compute optimized LIS values in
       bottom up manner */
    for (int i = 1; i < n; i++){
        lis[i] = 1;
        for (int j = 0; j < i; j++)
            if ( arr[i] > arr[j] && lis[i] < lis[j] + 1)
                lis[i] = lis[j] + 1;
    }
    // Return maximum value in lis[]
    return *max_element(lis, lis+n);
}
```

House robber

```
for (i -> n)
    for (j -> i)
        max(dp[j]...)
```

Can target string be built by input string, top-bottom.

```
vector<string> input = {"ab", "abc", "cd", "def", "abcd", "ef", "c"};
string target = "abcdef";
```

```

vector<vector<string>> ac(string target, const vector<string> & in ){
    if (target.size() == 0){
        return vector<vector<string>> {{{}}};
    }

    vector<vector<string>> last_result;
    for(auto i: input){
        if (target.find(i) == 0){
            string sub = target;
            sub.erase(0, i.size());
            vector<vector<string>> re = ac(sub, in);
            for(auto j : re){
                j.push_back(i);
                last_result.push_back(j);
            }
        }
    }
    return last_result;
}

```

How many way can target string be built by input string, top-bottom.

```

vector<string> input = {"purp", "p", "ur", "le", "purpl"};
string target = "purple";

int ac(string target, const vector<string> & in ){
    int arr[7] = {1, 0, 0, 0, 0, 0, 0};
    for(int i = 0; i < 7; i++){
        for (auto s: input){
            int length = s.length();
            if(target.substr(i, length) == s)
            {
                arr[i+length] += arr[i];
            }
        }
    }
    return arr[6];
}

```

2.8.3.1 Conclusion

For this problem, first, decide to use one dimension dp (lis array in previous example). Then dp definition(state) should be match with your questions, here, state is the length of LIS.

dp and recursive use the same state. For example, dp and return value of recursive are both length of LIS.

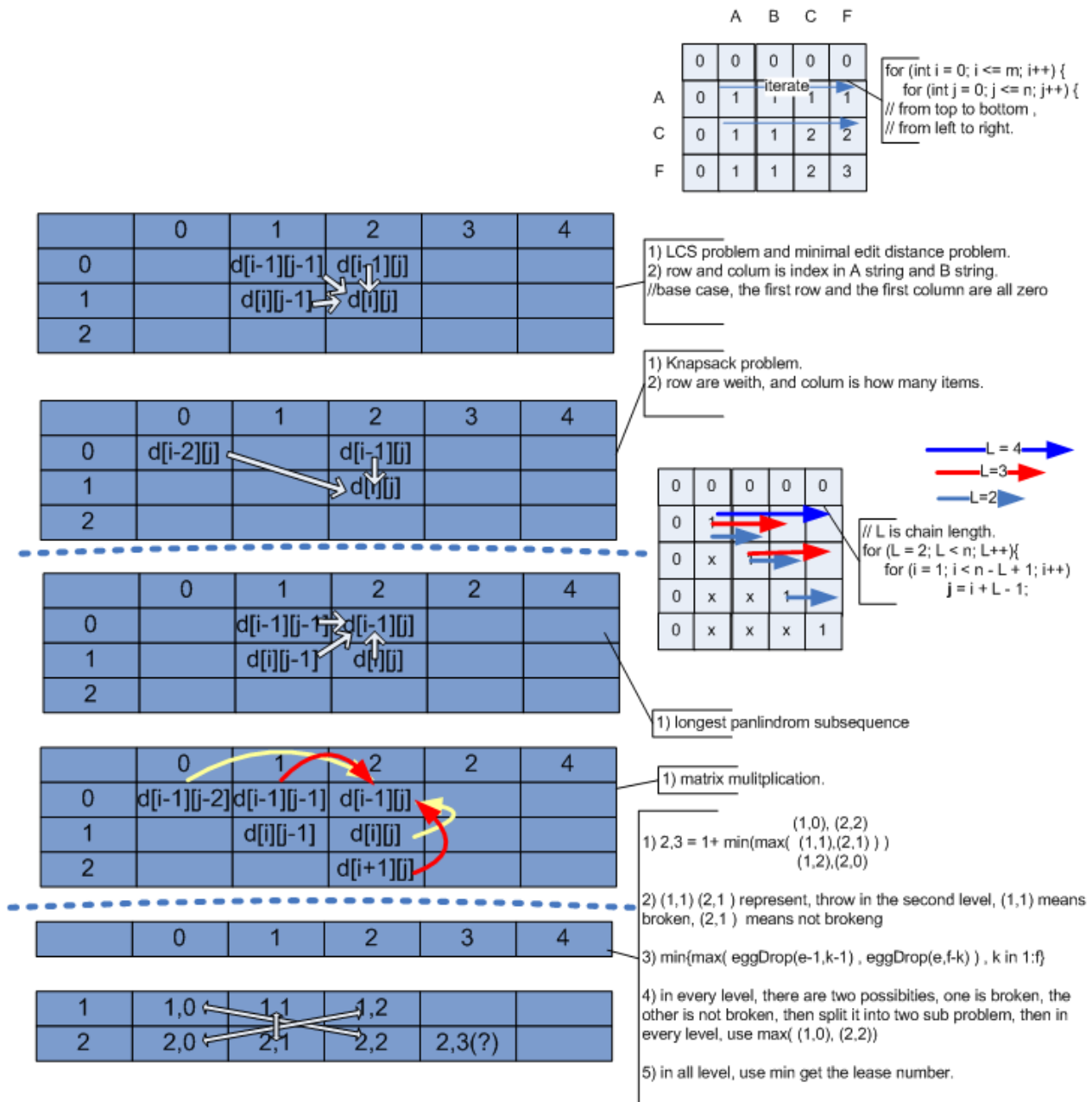
```

for (i -> n) // A common template
for (j -> i)
max(dp[j]...)

```

2.8.4 Two dimension

The basic idea is below:



LCS problem

```

1  /* Returns length of LCS for X[0..m-1], Y[0..n-1] */
2  int lcs( char *X, char *Y, int m, int n )
3  {
4      int L[m + 1][n + 1];
5      int i, j;
6
7      /* Following steps build L[m+1][n+1] in
8       bottom up fashion. Note that L[i][j]
9       contains length of LCS of X[0..i-1]
10      and Y[0..j-1] */
11
12     for ( i = 0; i <= m; i++) {
13         for ( j = 0; j <= n; j++) {
14             if ( i == 0 || j == 0 )

```

```

15     L[i][j] = 0;
16     //base case, the first row and the first column are all zero
17
18     else if (X[i - 1] == Y[j - 1])
19         L[i][j] = L[i - 1][j - 1] + 1;
20
21     else
22         L[i][j] = max(L[i - 1][j], L[i][j - 1]);
23     }
24 }
25
26 /* L[m][n] contains length of LCS
27 for X[0..n-1] and Y[0..m-1] */
28
29 return L[m][n];
30 }

```

minimal edit distance problem

```

1  int min(int x, int y, int z) { return min(min(x, y), z); }
2  int editDistDP(string str1, string str2, int m, int n)
3  {
4      // Create a table to store results of subproblems
5      int dp[m + 1][n + 1];
6
7      // Fill d[][] in bottom up manner
8      for (int i = 0; i <= m; i++) {
9          for (int j = 0; j <= n; j++) {
10             // If first string is empty, only option is to
11             // insert all characters of second string
12             if (i == 0)
13                 dp[i][j] = j; // Min. operations = j
14
15             // If second string is empty, only option is to
16             // remove all characters of second string
17             else if (j == 0)
18                 dp[i][j] = i; // Min. operations = i
19
20             // If last characters are same, ignore last char
21             // and recur for remaining string
22             else if (str1[i - 1] == str2[j - 1])
23                 dp[i][j] = dp[i - 1][j - 1];
24
25             // If the last character is different, consider
26             // all possibilities and find the minimum
27             else
28                 dp[i][j] = 1 + min(dp[i][j - 1], // Insert
29                 dp[i - 1][j], // Remove
30                 dp[i - 1][j - 1]); // Replace
31         }
32     }
33
34     return dp[m][n];
35 }

```

longest panlindrom subsequence (LPS)

```

1  // Returns the length of the longest palindromic subsequence in seq
2  int lps(char *str)
3  {
4      int n = strlen(str);

```

```

5  int i, j, cl;
6  int L[n][n]; // Create a table to store results of subproblems
7
8  // Strings of length 1 are palindrome of length 1
9  for (i = 0; i < n; i++)
10 L[i][i] = 1;
11
12 // Build the table. Note that the lower diagonal values of table are
13 // useless and not filled in the process. The values are filled in a
14 // manner similar to Matrix Chain Multiplication DP solution (See
15 // https://www.geeksforgeeks.org/matrix-chain-multiplication-dp-8/). cl is length of
16 // substring
17 for (cl=2; cl<=n; cl++){
18     for (i=0; i<=n-cl; i++){
19         j = i+cl-1;
20         if (str[i] == str[j] && cl == 2)
21             L[i][j] = 2;
22         else if (str[i] == str[j])
23             L[i][j] = L[i+1][j-1] + 2;
24         else
25             L[i][j] = max(L[i+1][j-1], L[i+1][j]);
26     }
27 }
28 return L[0][n-1];
29 }

```

Matrix multiply

```

1  int MatrixChainOrder(int p[], int n){
2  /* For simplicity of the program, one
3   extra row and one extra column are
4   allocated in m[][]. 0th row and 0th
5   column of m[][] are not used */
6  int m[n][n];
7
8  int i, j, k, L, q;
9
10 /* m[i][j] = Minimum number of scalar
11 multiplications needed to compute the
12 matrix A[i]A[i+1]...A[j] = A[i..j] where
13 dimension of A[i] is p[i-1] x p[i] */
14
15 // cost is zero when multiplying
16 // one matrix.
17 for (i = 1; i < n; i++)
18 m[i][i] = 0;
19
20 // L is chain length.
21 for (L = 2; L < n; L++){
22     for (i = 1; i < n - L + 1; i++){
23         j = i + L - 1;
24         m[i][j] = INT_MAX;
25         for (k = i; k <= j - 1; k++){
26
27             // q = cost/scalar multiplications
28             q = m[i][k] + m[k+1][j]
29             + p[i-1] * p[k] * p[j];
30             if (q < m[i][j])
31                 m[i][j] = q;
32         }
33     }
34 }

```

```

34 }
35
36 return m[1][n - 1];
37 }

```

- $M_{ij} \leftarrow M_i \times M_{i+1} \times \dots \times M_j, i \leq j$
- $c(i, j) \leftarrow$ cost of an optimal way to compute M_{ij}
- $kay(i, j) = k$ be such that the optimal computation of M_{ij} computes $M_{ik} \times M_{k+1, j}$

$$\begin{aligned}
 c(i, i) &= 0, \quad 1 \leq i \leq q \quad (M_{ii} = M_i) \\
 c(i, i+1) &= r_i r_{i+1} r_{i+2}, \quad 1 \leq i < q \quad (M_{ii+1} = M_i \times M_{i+1}) \\
 c(i, i+s) &= \min_{i \leq k < i+s} \{ c(i, k) + c(k+1, i+s) + r_i r_{k+1} r_{i+s+1} \}
 \end{aligned}$$

$kay(i, i+s)$ is the value of k that yields the above min.

knapsack capacity

```

1 int knapSack(int W, int wt[], int val[], int n)
2 {
3     int i, w;
4     int K[n + 1][W + 1];
5
6     // Build table K[][] in bottom up manner
7     for (i = 0; i <= n; i++)
8     {
9         for (w = 0; w <= W; w++)
10        {
11            if (i == 0 || w == 0)
12                K[i][w] = 0;
13            else if (wt[i - 1] <= w)
14                K[i][w] = max(val[i - 1] +
15                    K[i - 1][w - wt[i - 1]],
16                    K[i - 1][w]);
17            else
18                K[i][w] = K[i - 1][w];
19        }
20    }
21    return K[n][W];
22 }

```

egg drop

```

1 int eggDrop(int n, int k){
2     /* A 2D table where entry
3     eggFloor[i][j] will represent
4     minimum number of trials needed for
5     i eggs and j floors. */
6     int eggFloor[n + 1][k + 1];
7     int res;
8     int i, j, x;
9
10    // We need one trial for one floor and 0
11    // trials for 0 floors
12    for (i = 1; i <= n; i++) {

```

```

13     eggFloor[i][1] = 1;
14     eggFloor[i][0] = 0;
15 }
16
17 // We always need j trials for one egg
18 // and j floors.
19 for (j = 1; j <= k; j++)
20     eggFloor[1][j] = j;
21
22 // Fill rest of the entries in table using
23 // optimal substructure property
24 for (i = 2; i <= n; i++) {
25     for (j = 2; j <= k; j++) {
26         eggFloor[i][j] = INT_MAX;
27         for (x = 1; x <= j; x++) {
28             res = 1 + max(
29                 eggFloor[i - 1][x - 1],
30                 eggFloor[i][j - x]);
31             if (res < eggFloor[i][j])
32                 eggFloor[i][j] = res;
33         }
34     }
35 }
36
37 // eggFloor[n][k] holds the result
38 return eggFloor[n][k];
39 }

```

below is dp implementation of knapsack.

```

1  int knapsack_dp() {
2      int W = 10;
3      int N = 5;
4      int val[] = {40, 50, 100, 95, 30};
5      int wt[] = { 2, 4, 2, 5, 3 };
6      int dp[6][11];
7      for (int i = 0; i <= N; i++) {
8          for (int w = 0; w <= W; w++) {
9              if (i == 0)
10                 dp[i][w] = 0;
11                 if (w == 0)
12                     dp[i][w] = 0;
13             }
14         }
15         for (int i = 1; i <= N; i++) {
16             for (int w = 1; w <= W; w++) {
17                 if (w - wt[i - 1] < 0) {
18
19                     dp[i][w] = dp[i - 1][w];
20                 }
21                 else {
22
23                     dp[i][w] = max(
24                         dp[i - 1][w - wt[i - 1]] + val[i - 1],
25                         dp[i - 1][w]
26                     );
27                 }
28             }
29         }
30     }
31     for (int i = 1; i <= N; i++) {

```

```

32     for (int w = 1; w <= W; w++) {
33         cout << dp[i][w] << " ";
34     }
35     cout << endl;
36 }
37
38 return dp[N][W];
39 }
40 // 0 40 40 40 40 40 40 40 40 40
41 // 0 40 40 50 50 90 90 90 90 90
42 // 0 100 100 140 140 150 150 190 190 190
43 // 0 100 100 140 140 150 195 195 235 235
44 // 0 100 100 140 140 150 195 195 235 235

```

2.8.5 Three dimension

A typical question is stock buy and sell.

```

1  for (int i = 0; i < n; i++)
2  for (int k = max_k; k >= 1; k--) {
3      if (i - 1 == -1) {
4          // deal i = -1 base case
5          dp[i][k][0] = 0;
6          dp[i][k][1] = -prices[i];
7          continue;
8      }
9      dp[i][k][0] = Math.max(dp[i-1][k][0], dp[i-1][k][1] + prices[i]);
10     dp[i][k][1] = Math.max(dp[i-1][k][1], dp[i-1][k-1][0] - prices[i]); // k-1 here.
11     // three dimension array, go to the next level, please refer the below picture.
12 }
13 return dp[n-1][max_k][0];

```



2.8.6 Summary

The general form of a dynamic programming problem is to find the optimal value. Since the goal is to find the optimal value, the core issue in solving dynamic programming problems is enumeration. The steps are: define the base case, clarify the "**states**", clarify the "**choices**", and define the meaning of the dp array/function. I summarize this theory as "one model with three characteristics."

First, let's look at what "one model" refers to. It describes the model of problems that dynamic programming is suited for solving. I define this model as the "optimal solution model with multi-stage

decision-making." Each decision stage corresponds to a set of states. We then search for a sequence of decisions that, when followed, results in the optimal value we seek.

Now, let's explore what the "three characteristics" are. They are **optimal substructure**, **no after-effect**, and **overlapping subproblems**. These concepts are somewhat abstract, so I'll explain them one by one:

- Optimal substructure means that the optimal solution to the problem contains the optimal solutions to its subproblems. In other words, we can derive the optimal solution of the problem from the optimal solutions of its subproblems. When we map optimal substructure to the dynamic programming problem model we defined earlier, it means that the states in the later stages can be derived from the states in the earlier stages.
- No after effect (or Memorylessness). This has two meanings. The first is that when deriving the state of a later stage, we only care about the state values of the earlier stages, not how those states were derived step by step. The second meaning is that once the state of a certain stage is determined, it is not affected by decisions made in later stages. No after effect is a relatively "relaxed" requirement. As long as the dynamic programming problem model mentioned earlier is satisfied, the no after effect property will generally hold.
- Overlapping Subproblems This concept is easier to understand. As mentioned several times in the previous section, in summary, it means that different decision sequences may reach the same stage, potentially resulting in repeated states.

Dynamic programming involves the following steps:

1. Transform the current problem into subproblems: Once you break it down into subproblems, define three things:
 - (a) states
 - (b) choice.
 - (c) the state transition equation, which explains how the solution to the current problem is derived from the combination (or transformation) of the subproblems.
2. Decide whether to use top-down DP (subproblem, subproblem state 1, subproblem state 2, etc.) or bottom-up DP: For example, `dp[subproblem]`, `dp[subproblem state 1]`, `dp[subproblem state 2]`, etc.
3. Determine the traversal direction and base case based on the meaning of the dp function.
4. Apply the template code to implement the solution.

Below are some common examples: the egg-drop problem, subsequence problems and finding the minimum. The 0/1 knapsack problem and stock trading problem.

Why does the n-floor egg-drop problem require considering all subproblems from 0 to i-1 for each i, while the stock trading problem only needs to consider the previous state? One thing to note is that in the stock trading problem, k represents the maximum number of allowed transactions, not the number of transactions already completed. It's a way of describing the state.

(Based on the problem, define the dp, then take `dp[i-1]` and try to see if you can combine `dp[i-1]` to form dp. Generally, there are two possibilities: either it involves all previous states (as in the egg-drop problem), or it involves just the previous two states (as in the stock trading problem). The current state can transition from buying or not buying.)"

Four questions: matrix multiply, shoot ballon, gambling(piles stone, take from left and right) and maxium panlinmon subsequene. The answer is located in the top-right corner of a 2D array, and you need to use diagonal traversal or reverse traversal. The transition equations are almost identical.

```
1 for (int i = n; i >= 0; i--) {
```

```

2 // j from left to right
3 for (int j = i + 1; j < n + 2; j++) {
4     // "Which balloon is the last to be burst?"
5
6     for (int k = i + 1; k < j; k++) {
7         // choose the best option
8         dp[i][j] = Math.max(
9             dp[i][j],
10            dp[i][k] + dp[k][j] + points[i]*points[j]*points[k]
11        );
12    }
13 }
14 }

```

Two questions: Regular expressions and edit distance: The result is in the bottom-right corner, and a forward traversal is needed. The subfunction represents an edit or match operation.

```

1 bool dp(string& s, int i, string& p, int j) {
2     if (s[i] == p[j] || p[j] == '.') {
3         // match
4         if (j < p.size() - 1 && p[j + 1] == '*') {
5             // 1.1 regex match 0 or more
6             return dp(s, i, p, j + 2)
7             || dp(s, i + 1, p, j);
8         } else {
9             // 1.2 normal match 1
10            return dp(s, i + 1, p, j + 1);
11        }
12    } else {
13        // not match
14        if (j < p.size() - 1 && p[j + 1] == '*') {
15            // 2.1 regex match 0 times
16            return dp(s, i, p, j + 2);
17        } else {
18            // 2.2 can't continue match
19            return false;
20        }
21    }
22 }
23
24 // return s1[0..i] and s2[0..j] minimum edition distance
25 int dp(String s1, int i, String s2, int j) {
26     // base case
27     if (i == -1) return j + 1;
28     if (j == -1) return i + 1;
29
30     if (s1.charAt(i) == s2.charAt(j)) {
31         return dp(s1, i - 1, s2, j - 1); // do nothing
32     }
33     return min(
34         dp(s1, i, s2, j - 1) + 1, // insert
35         dp(s1, i - 1, s2, j) + 1, // delete
36         dp(s1, i - 1, s2, j - 1) + 1 // replace
37     );
38 }

```

```

1 for (int i = 1; i <= n; ++i) {
2     for (int j = 1; j <= min(i, m); ++j) {
3         if (j == 1) {
4             dp[i][j] = sum[i];

```



```

5     } else {
6         for (int k = 1; k <= i - 1; ++k) {
7             dp[i][j] = min(dp[i][j], max(dp[k][j - 1], sum[i] - sum[k]));
8         }
9     }
10 }
11 }
12 return dp[n][m]

```

2.9 Branch and bound

Backtracking mainly use BFS, not DFS.

Backtracking is used to find all possible solutions available to the problem. It traverse state tree by DFS (Depth First Search) until it found a solution.. It realizes that it has made a bad choice & undoes the last choice by backing up. It involves feasibility function.

Branch-and-Bound, it realizes that it already has a better optimal solution that the pre-solution leads to so it abandons that pre-solution. It involves bounding function. Usually searches a tree in a breadth-first / least-cost manner (unlike backtracking)

Difference between branch-bound and backtracking is about size

- backtracking use stack and only add one child each time, so it use less memory compared with branch-bound
- branch-bound store all possible child in queue, so if answer is near root, it will find answer quickly.
- win in size, but lose in time. backtracking will find answer slower if answer is near the root.

Below I present three different methods to resolve 0-1 back pack problem. dp only can deal with integral weight. but branch-bound can deal with float value. This code can run and print out all the value, it can help you to understand the idea behind these two important algorithms.

below is knapSack recursive implementation, it's also backtracking, different with using stack. The idea is same, you can see the code is simpler.

```

1 int knapSack(int W, int wt[], int val[], int n){
2     if (n == 0 || W == 0) // Base Case
3         return 0;
4
5     // If weight of the nth item is more
6     // than Knapsack capacity W, then
7     // this item cannot be included
8     // in the optimal solution
9     if (wt[n - 1] > W)
10        return knapSack(W, wt, val, n - 1);
11
12    // There are two options, so there's no need to write a for-loop. Based on the options
13    // , change the state, pass the state into the recursive call, and take the maximum
14    // of the returned results.
15
16    // Return the maximum of two cases:
17    // (1) nth item included
18    // (2) not included
19    else
20
21        return max(
22            val[n - 1]
23            + knapSack(W - wt[n - 1],

```

```

22 wt, val, n - 1),
23 knapSack(W, wt, val, n - 1));
24 }

1 // Structure for Item which store weight and corresponding value of Item
2 struct Item{
3     float weight;
4     int value;
5 };
6 // Node structure to store information of decision tree
7 struct Node{
8     // level → Level of node in decision tree (or index in arr[])
9     // profit → Profit of nodes on path from root to this node (including this node)
10    // bound → Upper bound of maximum profit in subtree of this node/
11    int level, profit, bound;
12    float weight;
13 };
14 // Comparison function to sort Item according to val/weight ratio
15 bool cmp(Item a, Item b){
16     double r1 = (double)a.value / a.weight;
17     double r2 = (double)b.value / b.weight;
18     return r1 > r2;
19 }
20
21 // below is knapsack backtracking implementation.
22 int knapsack_back(int W, Item arr[], int n){
23     sort(arr, arr + n, cmp); // sorting Item on basis of value per unit weight.
24
25     stack<Node> S; // make a queue for traversing the node
26     Node u, v;
27
28     u.level = -1; // dummy node at starting
29     u.profit = u.weight = 0;
30     S.push(u);
31
32     // One by one extract an item from decision tree compute profit of all
33     // children of extracted item and keep saving maxProfit int maxProfit = 0;
34     while (!S.empty()){
35         u = S.top();
36         S.pop();
37         // If it is starting node, assign level 0
38         if (u.level == -1)
39             v.level = 0;
40
41         // If there is nothing on next level
42         if (u.level == n - 1)
43             continue;
44
45         // Else if not last node, then increment level,
46         // and compute profit of children nodes.
47         v.level = u.level + 1;
48
49         // Do the same thing, but Without taking the item in knapsack
50         v.weight = u.weight;
51         v.profit = u.profit;
52         S.push(v);
53
54         // Taking current level's item add current level's
55         // weight and value to node u's weight and value
56         v.weight = u.weight + arr[v.level].weight;
57         v.profit = u.profit + arr[v.level].value;

```

```

58     if(v.weight>W)
59         continue;
60
61     // If cumulated weight is less than W and profit
62     // is greater than previous profit, update maxprofit
63     if (v.weight <= W && v.profit > maxProfit)
64         maxProfit = v.profit;
65
66     // If bound value is greater than profit,
67     // then only push into queue for further consideration
68     S.push(v);
69 }
70 return maxProfit;
71 }
72
73 //-----
74 // Returns bound of profit in subtree rooted with u.
75 // This function mainly uses Greedy solution to find
76 // an upper bound on maximum profit.
77 int bound(Node u, int n, int W, Item arr[]){
78     // if weight overcomes the knapsack capacity, return 0 as expected bound
79     if (u.weight >= W)
80         return 0;
81
82     // initialize bound on profit by current profit
83     int profit_bound = u.profit;
84
85     // start including items from index 1 more to current item index
86     int j = u.level + 1;
87     int totweight = u.weight;
88
89     // checking index condition and knapsack capacity condition
90     while ((j < n) && (totweight + arr[j].weight <= W)){
91         totweight += arr[j].weight;
92         profit_bound += arr[j].value;
93         j++;
94     }
95     // If k is not n, include last item partially for upper bound on profit
96     if (j < n)
97         profit_bound += (W - totweight) * arr[j].value / arr[j].weight;
98     return profit_bound;
99 }
100
101 // Returns maximum profit we can get with capacity W
102 int knapsack_bb(int W, Item arr[], int n){
103     // sorting Item on basis of value per unit weight.
104     sort(arr, arr + n, cmp);
105     queue<Node> Q; // make a queue for traversing the node
106     Node u, v;
107
108     u.level = -1; // dummy node at starting
109     u.profit = u.weight = 0;
110     Q.push(u);
111
112     // One by one extract an item from decision tree compute
113     // profit of all children of extracted item and keep saving maxProfit
114     int maxProfit = 0;
115     while (!Q.empty()) {
116         u = Q.front();
117         Q.pop();
118         if (u.level == -1)

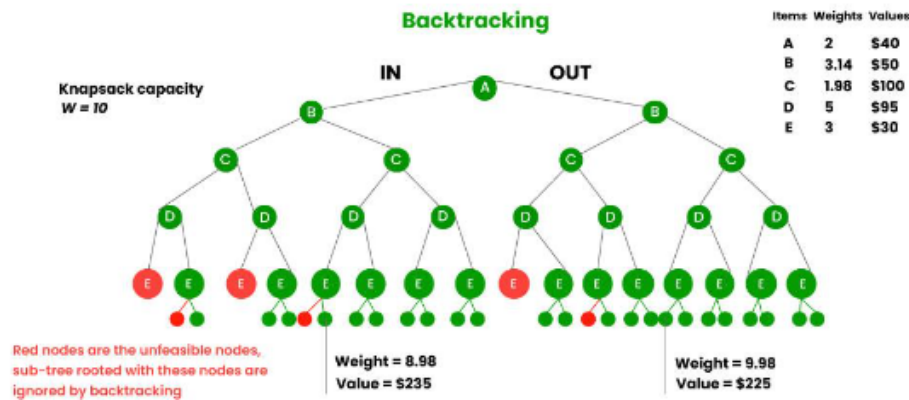
```

```

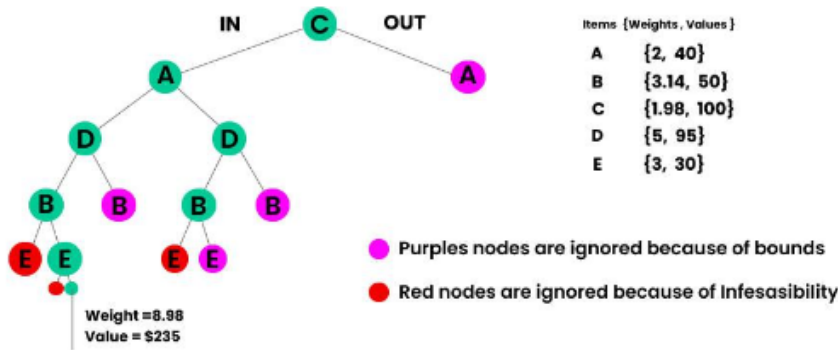
119     v.level = 0;
120
121     // If there is nothing on next level
122     if (u.level == n - 1)
123         continue;
124
125     // Else if not last node, then increment level,
126     // and compute profit of children nodes.
127     v.level = u.level + 1;
128
129     // Taking current level's item add current level's
130     // weight and value to node u's weight and value
131     v.weight = u.weight + arr[v.level].weight;
132     v.profit = u.profit + arr[v.level].value;
133
134     // If cumulated weight is less than W and profit
135     // is greater than previous profit, update maxprofit
136     if (v.weight <= W && v.profit > maxProfit)
137         maxProfit = v.profit;
138
139     // Get the upper bound on profit to decide whether to add v to Q or not. If bound
140     // value
141     // is greater than profit, then only push into queue for further consideration
142     v.bound = bound(v, n, W, arr);
143     if (v.bound > maxProfit)
144         Q.push(v);
145
146     // Do the same thing, but Without taking the item in knapsack
147     v.weight = u.weight;
148     v.profit = u.profit;
149     v.bound = bound(v, n, W, arr);
150     if (v.bound > maxProfit)
151         Q.push(v);
152 }
153 return maxProfit;
154 }
155 int main() {
156     int W = 10; // Weight of knapsack
157     Item arr[] = { {2, 40}, {3.14, 50}, {1.98, 100}, {5, 95}, {3, 30} };
158     int n = sizeof(arr) / sizeof(arr[0]); // n object.
159
160     knapsack_back(W, arr, n); // use backtracking
161     knapsack_bb(W, arr, n); // use branch and bound
162 }

```

Below figure illustrates the backtracking



Below figure illustrates the branch and bound



Recursive 0-1 pack pack. This is back trace, but why don't we have reverse decision? because in eight queen, we use reference parameter pass it to function(all function share). Here, we use stack, in each level, we have saved the local variable(you can think that it has been reversed by popping stack.) That is the difference. Another important thing is that for 0-1 pack and eight queen, we need a logically recursive tree. So we need to simulate a "level" conception. **0-1 pack stack solution, we save level information to the each node. 0-1 recursive solution, we use int i (array index) as level information. in eight queen, we use row information as level information . level is the level information in the recursive solution tree.**

```

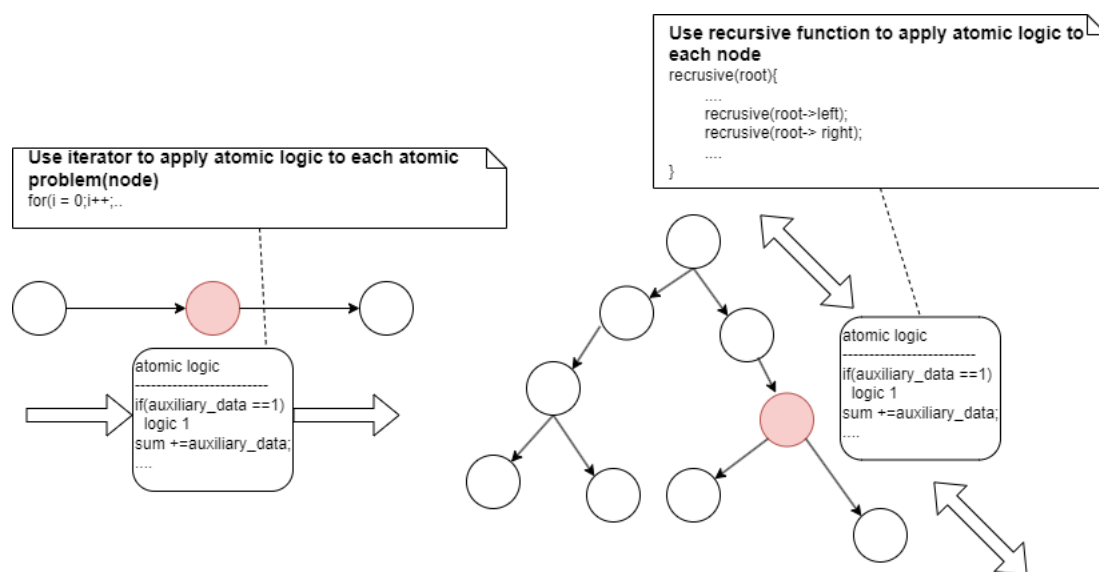
1 public int maxW = Integer.MIN_VALUE;
2 // cw current weight, i: current item call with f(0, 0, a, 10, 100)
3 public void f(int i, int cw, int[] items, int n, int w) {
4     if (cw == w || i == n) { // cw==w is full or select all items
5         if (cw > maxW) maxW = cw;
6         return;
7     }
8     f(i+1, cw, items, n, w);
9     if (cw + items[i] <= w) { // bigger than w, give up
10        f(i+1, cw + items[i], items, n, w);
11    }
12 }

```

2.10 Summary of algorithms

The most import rule: Break the big problem down into atomic problems. If the atomic problem requires some global conditions, then include the necessary data and data structures. Write the logic to handle this atomic problem (you may need to use a `if..else` to handle different conditions inside this atomic problem). Then apply this atomic problem handling logic into **recursion** or **iteration**. Difference between recursive and iterate.

- Recursive: Simple, intuitive code unless care is taken to avoid recomputing previously computed values, the recursive program will have prohibitive complexity
- Iterative: Not require additional space for the recursion stack. To minimize run time overheads, and hence to reduce actual run time, dynamic programming recurrences are almost always solved iteratively. (no recursion).



Three levels:

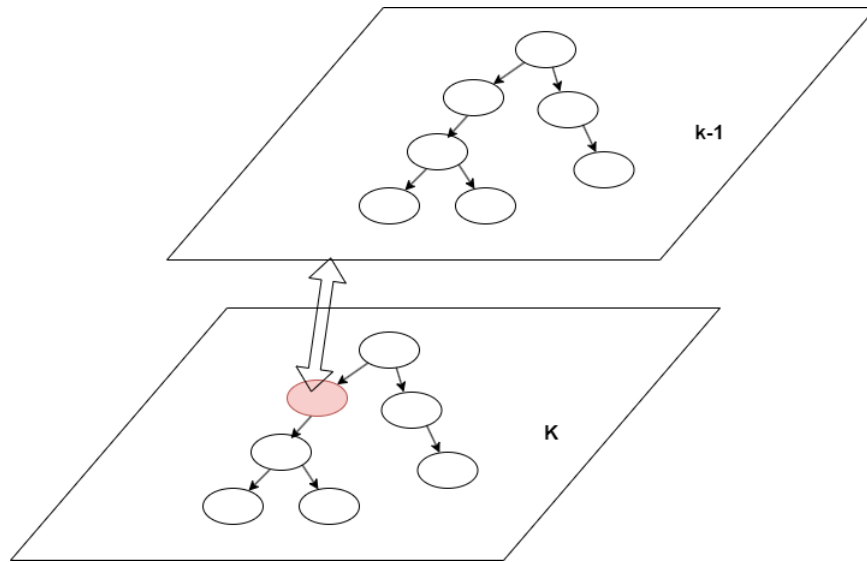
1. The first level consists of two basics: recursion and iteration.
2. The second level builds on these two. In the domain of trees, recursion manifests as three types of traversals; in the domain of graphs, recursion is represented by DFS, while iteration is represented by BFS.
3. The third level involves practical applications, where recursion and iteration are expressed through five major algorithms: divide-and-conquer (DC), dynamic programming (DP), backtracking (BT), branch-and-bound (BB), and greedy algorithms.

Difference between backtracking and DP:

- Both backtracking and dynamic programming involve a series of decisions. The difference is that in backtracking, certain constraints must be satisfied during the decision-making process (such as in the N-Queens problem). If the constraints are not met, you need to undo that decision. Dynamic programming, on the other hand, deals with the global optimal solution. So for each decision, you typically compare and record the optimal solution for the subproblem in order to eventually derive the global optimal solution.
- Another difference is that backtracking often uses recursion, while dynamic programming uses iteration (usually over a 2D array). However, sometimes the two approaches can be combined, such as in the partition problem, where backtracking is primarily used, but you can also record

solutions to subproblems (using dynamic programming techniques) to reduce the number of recursive calls.

- typical question for DP is stock buy and sell k time. typical question for backtrack is k equal subset. They all involve extra k parameter. With k parameter, you can think that it's k level, on each level, there is tree or array structure. That is the most complex problem you can meet in the interview questions. With k , you can think it's multi layer, for back track, each layer has a tree (k equal subset). For dp, each layer has two dimension array.



Chapter 3

Interview preparation

3.1 Basic principle

Basic strategy: (Four steps!)

- If it's recursive? if no, if it's iterate? if yes, what atomic operation is?
- Define all necessary states in one atomic operation? what need? what change?
- Init condition and end condition? init condition should be same in each iteration.
- Iterate or recursive? what value can be saved to avoid iterator again?

Pay more attention to below points when you write C++ code:

- **The first step, consider the edge case!**
- Write slowly, try you best to write zero bug code.
- Prefer to use std container, algorithm and views, especially views:: if the compiler supports it.
- Use auto as many as possible. Must know if you need add reference & or const when you use auto and structure bind.
- prefer to use what you know and sure, not new feature which you are not quite sure.
- If there are two options, ask first, then coding.

3.2 Basic data structure and algorithms

- Circular queue: 1) use %, 2) when is full? `front == k-1` 3) when is empty? `back == front`
- `reverse(double pointer + iter_swap)` and `rotate(based on reverse)`.
- reverse list. (three pointers)
- inplace merge, just like reverse, need to know init and end condition, and in each step, what to do?
- multi merge based on heap.
- remove and partition based on double pointer. remove (++ first, then copy) partition (swap first, then ++)
- sort, insert sort, selection sort, quick sort (based on partition) and merge sort. Quick sort based on partition, merge sort based on merge.
- shuffle (swap + random)
- matrix traversal.
 1. transpose and rotate,

- 2. bottom to up
- 3. Diagonal traverse
- 4. matrix multiplication traverse
- 5. spiral print
- 6. DFS, and BFS.
- tree traversal(pre-order, inorder and post-order)
- trie tree, just multi-branch tree, use pointer array, and is_leaf.

3.2.0.1 Math

- Know the difference,

```

1 -1<<1    still -1
2 -1/2     equal 0
3 -3%2     equal -1

```

- know gcd and lcm. How to get prime number.

3.3 Code Style

3.3.1 STL container

- 4753, 47743. container methods summary.
- Some new containers, such as `std::forward_list`, `std::array` and `std::bitset` and other a few other range-like views, such as `std::span` and `std::string_view`.
- You can use `erase` for(`vector`, `string`) and `contains`(for `associate`) and `starts_with` and `ends_with`(for `string`) if compiler supports.
- For `std::array`, the main interface are `access`, `iterator` and `fill`. `std::span` also has `access`, `iterator` and `first()`, `last()` and `subspan()`.
- how to construct? use `std::vector/std::string.reserve`.
 1. `iota`
 2. `fill`, `generate` + `mt19937`
 3. `()` and `{}`
 4. from other container.

3.3.1.1 construct and init

- No. Not all STL iterators support `<`. Only random access iterators do. when I iterate list, I can't use `it<l.end()`, only can use `it!=l.end()`,
- Know common container initialization:

```

1 //1) use copy constructor
2 vector<vector<int>> vv{{1,2,3},{4, 5, 6}};
3 vector<vector<int>> ww{vv};
4
5 //2) all zero
6 vector<vector<int>> ww(2, vector<int>(10, 0));
7
8 //3) build from different container.
9 set<int> s {1, 2, 3};
10 vector<int> v2{s.begin(), s.end()};

```

```

11 vector<int> v2{1, 2, 3, 3, 3,4};
12 set<int> s {v2.begin(), v2.end()};
13
14 map<int, int> m {{1, 2}, {2, 4}};
15 vector<pair<int, int>> v1{m.begin(), m.end()}; //vi[0] is pair<int, int>
16 vector v1(m.begin(), m.end()); // v1[0] is pair<const int, int>
17
18 map<int, int> m {{1, 2}, {2, 4}};
19 vector v1{m.begin(), m.end()}; // That is not correct, because {} not ()
20
21
22 vector<pair<int, int>> v1{{1, 2}, {2, 4}};
23 map<int, int> m {v1.begin(), v1.end()};
24

```

- Prefer to use range and reserve

```

1 vector v1 {1, 2, 3};
2 vector v2;
3 v2.reserve(2*v1.size()); // first reserve
4 v2.assign(v1.begin(), v1.end()); //range assign
5 v2.insert(v2.end(), v1.begin(), v1.end()); //range insert.

```

3.3.1.2 insert and erase

- for map, insert, emplace, emplace_hint, try_emplace difference. [] and insert_or_assign difference. Use more emplace if it is possible.
- for different container, insert has different overload, for sequence, need a position for insert position, for associate, usually doesn't need insert position, but have a insert(hint, value) to keep consistent with sequence insert. Usually they all support range insert(iter1, iter2).
- erase+remove for vector, remove for list and erase for associate. in C++20, vector has non-member erase and erase_if. map has non-member erase_if for easier use. std::erase and std::erase_if for vector and string and deque
- fill and generate can be thought as another insert.

3.3.1.3 update

major method to update key value or value value in associate container. first erase(i++); then insert(i,e)

A better method is to use extract in c++23

```

1 auto n = m.extract()
2 n.key() = 3;
3 m.insert(std::move(n));

```

3.3.1.4 look up

- find, count, contains, find_if, low/upper_bound and binary_search
- all/none/any_of return bool.

3.3.1.5 operations

- basic shuffle and sort operation.

- Use `list.splice` in `list` to finish rotate.
- range assign, construct and insert and erase
- min priority queue

```
1 priority_queue<int, vector<int>, greater<int>> > min_pq;
```

- Some typical auto usages:

```
1 for(auto &[f, s] :map1) //use & here, no need const,
2
3 auto [it, ins] = map1.insert_or_assign(1, 3);
4
5 const auto [f, s] = ranges::remove() //no & here, use const for better security.
6
7 auto [fi, si] = multimap.equal_range(1); // struct binding, fi is first iterator
8 for(auto p = fi; p!= si; ++p){
9     auto &[key, value] = *p; //struct bind, key is first element
10 }
```

3.3.1.6 string

- Own member functions:
 1. `string.find/rfind(ch/str, pos)`
 2. `string.substr(pos, count)`
 3. `replace(it1, it2..), replace(pos, count ...)`,
 4. `compare`
 5. `starts_with, ends_with, contains`
- common functions: `::toupper, ::tolower, isupper, islower, isdigit, isalnum, isalpha, isspace,`
- `to_string` and `stoi, stof, stoul, from_chars` and `to_chars` is much faster.
- `trim`(use `find_if_not` and `::isspace`) and `split` (`getline` + `stringstream`).
- `find` + `while` pattern.

3.3.2 STL algorithms

- SIMD vs. Multithreading
 SIMD and multithreading improve performance in different ways:
 SIMD Multithreading One core processes multiple data elements per instruction Multiple CPU cores work simultaneously Good for arrays, matrices, image processing Good for independent tasks or dividing large workloads Examples: AVX, AVX2, AVX-512, NEON `std::thread`, `std::jthread`, OpenMP, TBB
 They are often used together. For example, a program may split an image across multiple threads, and each thread uses AVX or NEON instructions to process its assigned pixels efficiently.
- Use `std::execution::par` when:
 Processing millions of elements. each iteration is independent. the work per element is significant (sorting, numeric computation, image processing, etc.). You've verified that your standard library provides an effective parallel implementation.
 Avoid it when:
 The container is small. Operations modify shared state without synchronization. The algorithm depends on iteration order or has side effects that aren't thread-safe.
 Most algorithms with execution-policy overloads have one characteristic:

They process every element (or large chunks of elements) independently, making them suitable for parallelization.

Algorithms that do not support execution policies often rely on a sequential search or maintain a strict state between steps, such as:

lower_bound binary_search make_heap next_permutation

These don't parallelize as naturally while preserving their semantics.

Finally, remember that execution policies apply to the classic STL algorithms (e.g., `std::sort`), not the ranges algorithms (`std::ranges::sort`) in C++20/C++23.

- ACCMFS, RRRRSSSFFPPCMT, HMSSS
- If compiler supports, use `ranges::algorithm`, pay attention 1) 4 math is not in ranges. 2) range return value are different, especially know Rotate Remove Search Shift-left Equal_range Find_last_if Partition return subrange.
- `ranges::starts(ends)_with`, `find_last_if` and `contains` and `contains_subrange`.
- `Find(_if)(_not)` and `find_first_of` and `adjacent_find` is the basic. range add `find_last_if_not`. `std::string` add `find_last_not_of`.
- Ranges doesn't support four math algorithms.
- Customization with callable: 1) change operation(accumulate change + to *), 2) change projection. (compare `pair::first`) 3) action 4) map 5) generator 6) predicate 7) compare
- `emplace` use `()` inside, not `.`
- `ranges::` return value
 1. find use iterator to mark position, `ranges::` version don't change it.
 2. most out iterator, add one or two in
 3. Some `ranges::` return subrange. 1) search 2) remove/unique 3) rotate, shift and partition.
- How to use projection on stl function? can't use directly, must use your own lambda. A good example of leetcode 1637,

```
1 sort(c, {}, &vector<int>::front);
2 //That is not correct, must use your own lambda to wrap it.
3
4 ranges::sort(points, {}, [](auto &e){return e.front();});
```

- `ranges::` algorithm use project more, and the same time, pay attention to some algorithm return two iterators. such as `search`.

3.3.3 Views

- Yes. In general, `std::views::join_with` does not return a `common_range`.
- `filter` is common, `take_while` is not common, Structure-changing adapters (often not `common_range`), such as `join`, `join_with`, `split`, `lazy_split`, and `istream`. `join` and `split` are not common because the iterator has much more state than the end.

This surprises many people because most containers are common ranges.

```
1 vector<vector<int>> > vv
2 auto vw = vv|join, //that is common
3 auto vw = v | chunk(2) | join, //that is NOT common
```

- sort vector of vector by the first element, `v.front` has overload, you can't use it directly, there are two methods you can overcome this problem.

```
1 std::ranges::sort(
2     vv,
3     std::ranges::less{}),
```

```

4      [] (const auto& v)
5      {
6          return v.front();
7      };
8
9  std::ranges::sort(
10     vv,
11     std::ranges::less {},
12     static_cast<const int& (std::vector<int>::*)() const>(
13         &std::vector<int>::front()));

```

- `std::clamp` (introduced in C++17) is used when you want to restrict a value to a given range.
- `std::span` is designed to be a view of contiguous memory. So you can construct a span from anything that provides a contiguous block of elements. C array `std::array` `std::vector` Another `std::span` Pointer + size Two pointers (since C++20) Only if the iterators are contiguous iterators.
- `auto pa = make_pair(2.3, "abc");` `pa` type is `std::pair<double, const char*>` if you need string, use

```

1 using namespace std::string_literals;
2 auto pa = make\_pair(2.3, "abc"s);

```

The other twos are `chrono_literals`; and `std::string_view_literals`;

- GPTCSSRZS

No argument	Number	Lambda	Another range
<code>reverse</code>	<code>take(n)</code>	<code>filter(pred)</code>	<code>zip(r2)</code>
<code>enumerate</code>	<code>drop(n)</code>	<code>transform(fn)</code>	<code>join_with(r2)</code>
<code>keys</code>	<code>stride(n)</code>	<code>take_while(pred)</code>	<code>concat(r2)</code>
<code>values</code>	<code>chunk(n)</code>	<code>drop_while(pred)</code>	
<code>common</code>	<code>slide(n)</code>		
<code>as_const</code>			

Table 3.1: Common C++20/23 Range Views by Parameter Type

- there are some concepts you can use judge if a view is common, size or others.
- `views::itoa(1)` and `ranges::to<string>()` works in leetcode.

```

1 string s = "12345678";
2
3 auto vw = s | views::chunk(3);
4 auto vw1 = vw | views::join\_with(' ', ' ') | ranges::to<string>();
5 cout<<vw1<<endl;
6 -std=c++23 -O2 -Wall -Wextra -Wpedantic

```

- `filter` is not `SIZE`, `take_while` is neither `COMMON` nor `SIZE`. Can't use `auto &` for `views::transform`. `element_view` iterator has no `->` operator
- `element` in `views` has been reference, prefer to use `auto` or `const auto &`, don't use `auto &`
- for `split` or `chunk_by` or `slide`, it return subrange, use `string_view` to wrap subrange.
- for `filter`, `take_while`, receive a function, better to use `ranges::to<string>()` to change it to real container. Use `ranges::to` more in interview to avoid some complex problems. it's in `ranges::`, not `views::`
- `adjacent pairwise`, `zip` and `enumerate` all return tuple, just use `auto [t1, t2, t3]` to structure bind
- error

```

1 std::string num = "0123456789";

```

```

2
3 auto vw = num | std::views::stride(2);
4 vw = num | std::views::drop(1) //vw has type inside, dont' reuse it.
5 | std::views::stride(2);
6
7 auto vw1 = num | std::views::drop(1)
8 | std::views::stride(2);

```

3.3.4 functor and lambda

- lambda init expression and mutable

```

1 auto counter = [x = 0]() mutable {
2     return ++x;
3 };
4
5 std::cout << counter() << "\n"; // 1
6 std::cout << counter() << "\n"; // 2

```

- logic(3), bit(4), math(6), compare(6), (3466)

3.3.5 Bit

- $n \& (n-1)$, clear right most, $n \& -n$, only keep the right most bit
- big and small endian,
- set, clear, flip
- find right and left bit.
- $(-1 \ll n)$, -1 is all set bit, use as left shift with 1, not 0.

3.3.6 Tips and Precautions

3.3.6.1 Tips

- give me some examples where I can use rvalue reference

This is the right question to ask after learning when copies happen.

There are two different perspectives:

Using rvalue references (calling code with `std::move()` or temporaries) Writing functions/classes that accept rvalue references (`T&&`).

If you write normal application code, you'll probably use `std::move` much more often than you'll write `T&&` yourself.

- when to use `std::move`?

This is probably the most important practical question about move semantics.

The rule is actually quite simple:

Use `std::move` when you are finished using an object and want to allow its resources to be transferred instead of copied.

Notice the wording:

finished using it allow transfer

Not:

"I want it to be faster."

`std::move` doesn't move anything by itself—it merely says, "You may treat this object as an rvalue."

```

1 1) Passing to a sink function
2 void setName(std::string name);
3 std::string s = "Alice";
4 setName(std::move(s));
5
6 2) Inserting into a container
7 std::vector<std::string> v;
8 std::string s = "Hello";
9 v.push_back(std::move(s));
10
11 3) Transferring ownership (unique_ptr)
12
13 4) Inside a move constructor and move assignment
14 class Person {
15     std::string name_;
16 };
17
18 Person(Person&& other)
19     : name_(std::move(other.name_))
20 {
21 }
22
23 5) Local variables you're done with
24 std::string s = getString();
25 std::vector<std::string> v;
26 v.push_back(std::move(s));

```

don't use in return, temporary or const objects.

- **first use views::, then projection, then lambda, then common algorithm.**
-
- in order to understand where to use smart pointer, need to know where to use pointer? know where to use rvalue reference, need to know where copy happen? There are two level, The first is RVO, it exists before rvalue reference. to resolve same problem, avoid extra copy. but rvo only helps to return. there are some other contexts need rvalue reference.

Context	Copy?	Move Possible?
Initialization	Yes	Yes
Pass by value	Yes	Yes
Return by value	Yes	Yes
Assignment	Yes	Yes
push_back()	Yes	Yes
Vector reallocation	Yes	Yes
std::swap()	Yes	Yes
Lambda capture	Yes	Yes
Exception by value	Yes	Yes
std::sort() internals	Yes	Yes

Table 3.2: Contexts Where Copy and Move Operations May Occur

```

1 Case 1: Function parameters
2 void setData(std::vector<int> v);
3 setData(std::vector<int>(1000000));
4
5 Case 2: Container reallocation
6 This is actually one of the most important reasons move semantics were added.

```

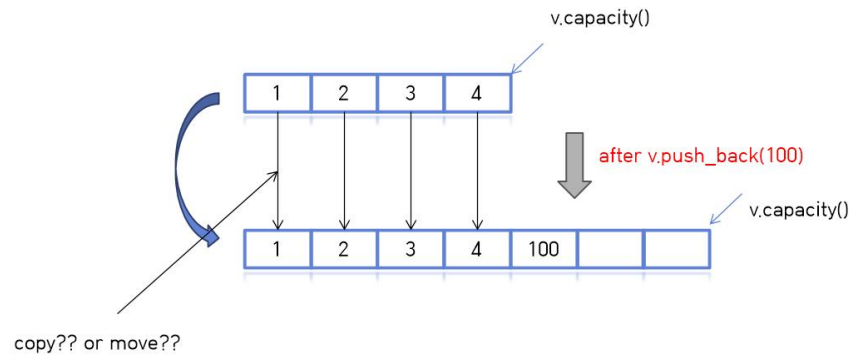



Figure 3.1:

```

7
8 Case 3: Swapping objects
9 template<class T>
10 void swap(T& a, T& b)
11 {
12     T temp(std::move(a));
13     a = std::move(b);
14     b = std::move(temp);
15 }
16
17 case 4: returning from different branches
18 #include <vector>
19 std::vector<int> f(bool b)
20 {
21     std::vector<int> a(1000);
22     std::vector<int> c(1000);
23
24     if (b)
25         return a;
26
27     return c;
28 }

```

In one word, RVO removes some moves. Rvalue references make the remaining moves cheap

- We often call these sink functions.

A sink function:

accepts ownership, stores the object, expects the caller to be finished with it.

forward is after rvalue reference. Perfect forwarding underpins many modern C++ facilities, such as `std::make_unique`, `std::make_shared`, `std::thread`, and the standard containers' `emplace` functions.

```

1 C++98s
2
3 copy only
4
5 too slow for STL
6
7 need to recognize temporaries
8
9 C++11 introduces &&
10
11 move constructor

```

```

12 move assignment
13 std::move
14 perfect forwarding

```

```

1 std::vector<int> v = std::vector<int>(1000000);

```

Task	printf	stringstream	std::format
String	%s	« name	{}
Integer	%d	« age	{}
Float (2 decimals)	%.2f	std::fixed « std::setprecision(2)	{:.2f}
Hex	%x	std::hex	{:x}
Zero padding	%06d	std::setw(6) « std::setfill('0')	{:06}
Type safety	No	Yes	Yes
Readability	Medium	Medium	Best

Table 3.3: Comparison of C++ Formatting Methods

C++98: Copy the entire vector. C++11: Move the vector by stealing its internal pointers. C++17: Don't create the temporary at all—construct `v` directly in its final location.

This progression also explains why move semantics are still essential: copy elision only applies in specific initialization and return scenarios, while moves improve performance in many other situations (function parameters, container reallocations, algorithms, swaps, assignments, etc.).

C++17 guaranteed copy elision is one of the biggest language changes since C++11. Before C++17, copy elision was an optimization. Since C++17, in certain cases it is part of the language semantics.

Code	C++11	C++17
<code>Foo f = Foo();</code>	Usually elided	Guaranteed
<code>return Foo();</code>	Usually elided	Guaranteed
<code>Foo f = make();</code> where <code>make()</code> returns <code>Foo()</code>	Usually elided	Guaranteed
<code>return local;</code>	NRVO (optional)	NRVO still optional
<code>return std::move(local);</code>	Move	Move (and often disables NRVO)

Table 3.4: Copy Elision in C++11 vs. C++17

- so new FOO is just syntax candy, right?

Yes, mostly. new Foo is a piece of syntax sugar (or "syntactic sugar"), but it's a little more than just a simple function call. such as `delete`, `range for` and

Task	printf	stringstream	std::format
String	%s	« name	{}
Integer	%d	« age	{}
Float (2 decimals)	%.2f	std::fixed « std::setprecision(2)	{:.2f}
Hex	%x	std::hex	{:x}
Zero padding	%06d	std::setw(6) « std::setfill('0')	{:06}
Type safety	No	Yes	Yes
Readability	Medium	Medium	Best

Table 3.5: Comparison of C++ Formatting Methods

-

- Most C++ programmers avoid naming variables

```

1 min    //use minValue
2 max    //use maxValue
3 begin
4 end
5 swap
6 distance
7 count
8 size

```

- **Most of complex data structure need hash and list combine.**
- -std=c++23 use this flag to supports a lot of new feature.
- Some useful patterns.

```

1 for(int i = 0; auto e: container){ //loop without know the size.
2     ...
3     ++i;
4 }
5
6 if( int pos = find; pos != string::npos ) //init-statement in if
7
8
9 flag == 2 ? ++sum1 : ++sum2; //ternary
10
11 maxs = max(e, maxs); //find max
12
13 numeric_limits<int>::max()

```

- variable name: cnt, result, st, ans, max1, max2
- nest if can be written &&

```

1 if(s==1){
2     if(t==2){
3     }
4 }
5
6 if(s==1 && t == 2) //a better way to express.

```

- ::toupper, ::tolower, ::isalnum, ::isalpha, ::isdigit, ::ispunct, ::isspace
- sqrt, cbrt, pow, abs
- dumb node at end.
- views::chunk_by and views::split and views::slide are very useful for some interview questions. if views::chunk_by is not useful, then use adjacent_find.

3.3.6.2 Precautions

```

1 std::generate(answer.begin(), answer.end(), [&v, i = 0]() { return v[v[i++]]; }) i
   doesn't need_int
2
3 std::set<int>_s(std::from_range, _v); //usage_of_from_range

```

- If you want to loop from length to 0, index must be int. can't be size_t, otherwise, -index will never small than 0. (Must remember)
- For vector and array, pay attention to size-1 is the last index. Also stick to left close and right open when using ranges algorithm. Pay left edge (0) and right edge (last element). Understand index and length.

- Check variable name, especially long variable name.
- Don't forget return in lambda and recursive. Don't forget ; in the end.
- Add () if expression is complicated.
- When while exist, understand index position and what it represent?

```

1 while(index<length){
2 }
3 //what index is now

```

3.4 Code template

two methods to split.

```

1 std::string str = "abc,bde,adfa";
2 int init = 0;
3 int pos = 0;
4 while( (pos = str.find( ',', pos)) != std::string::npos){
5     std::string temp = str.substr(init, pos-init);
6     std::cout<<temp<<std::endl;
7     ++pos;
8     init = pos;
9 }
10 std::cout<<str.substr(init)<<std::endl;
11
12
13 std::stringstream ss(str);
14 std::string word;
15 while(std::getline(ss, word, ',')){
16     std::cout<<word<<std::endl;
17 }

```

You can remember three methods by using slow and fast pointer, they are unique, remove and partition. in initial status, slow pointer points to what? in unique, slow is last element of valid set, so you need to ++slow, first, then $v[\text{slow}] = v[\text{fast}]$, in remove and partition, the slow is waiting check/invalid set the first element, so you need to $v[\text{slow}] = v[\text{fast}]$ first, then ++slow. that is one important thing. The second thing is to use swap in partition, that is all.

```

1 //put all element>3 to the front.
2 vector v{1, 1, 2, 3, 5, 4, 4, 4,4};
3 int len = v.size();
4 auto it = find_if(v, [](auto e){return e<=3;});
5 int slow = distance(v.begin(), it);
6 int fast = slow;
7 while(fast< len){
8     if(fast>3){
9         swap(v[slow], v[fast]);
10        ++slow;
11    }
12    ++fast;
13 }

```

You need to remember some famous algorithms implementations:

- Reverse link-list (recursive and iterative methods).
- Partition and partition_if, use slow and fast pointer and swap.
- Remove and unique, use slow and fast pointer.

- Binary search, and left/right boundary, use binary search, left close and right close.
- Find inserting point and $\text{sqrt}(x)$ use $\text{mid} = 1 + (r - l) / 2$
- Reverse, two pointers and swap
- Rotate, three reverse.
- Insertion sort, `std::upper_bound` and `std::rotate`.
- `nth_element`, quick select.
- Sort, quick sort, merge sort and heap sort.
- Inplace merge.

split, use init,

```

1 "abc-def-gh" -> abc  def  gh
2 init = 0;
3 while( (pos = date.find('-', init)) != string::npos ){
4     string temp = date.substr(init, pos-init);
5     cout<<temp<<endl;
6     ++pos;
7     init = pos;
8 }
9 temp = date.substr(init);
10 cout<<temp<<endl;

```

```

1 "abc-def-gh"
2
3 int init = date.size()-1;
4 while( (pos = date.rfind('-', init)) != string::npos ){
5     string temp = date.substr(pos+1, init-pos);
6     cout<<temp<<endl;
7     --pos;
8     init = pos;
9 }
10 string temp = date.substr(0, init+1); //use init+1 here.
11 cout<<temp<<endl;

```

Sliding windows template

```

1 int left = 0, right = 0;
2
3 while (right < s.size()) {
4     window.add(s[right]); // increase windows
5     right++;
6     *** //put you logic here.
7
8     //anytime you increase window, check condition.
9     while (window needs shrink) {
10         // reduce windows.
11         window.remove(s[left]);
12         left++;
13         *** //put your logic here.
14     }
15 }
16
17 //1) only use right < s.size as main loop
18 //2) "windows nees shrink" condition need to be customized.
19 //3) modify "window.add" and *** to your own logic.
20 //4) both left and right increments ++

```

When writing the backtrack function, you need to maintain the "path" that has been traversed and

the current "list of available choices." When the "termination condition" is triggered, add the "path" to the result set.

```

1 result = []
2 def backtrack(path , "list_of_available_choices"):
3     if (meet condition)
4         result.add(path)
5     return
6
7     for choic in "list_of_available_choices":{
8         choose choice;
9         backtrack(path , "maybe_new_list_of_available_choices");
10        revoke choice;
11    }

```

DFS and BFS code. For BFS, when D.pop we don't judge if it's visited. For all next current, visit them all and push them into the D. That is the difference with DFS.

```

1 DFS( start ){
2     S.push( start )
3     while( !S.empty() ){
4         current = S.pop
5         if( unvisited( current ){
6             visit current.
7             for all( next to current and unvisited )
8                 S.push( next );
9         }
10    }
11 }

```

```

1 BFS( start ){
2     D.push( start )
3     while( !D.empty() ){
4         current = D.pop //2) no if here
5         for all( next to current and unvisited ){
6             visit current. //1) push and visit at the same time.
7             D.push( next );
8         }
9     }
10 }

```

Best sell problem,

```

1 std::array<int, 9> arr = {2, 3, 8, 100, 20, 1, 1, 2, 1 };
2 std::unordered_map<int, int> dic;
3 for( auto e: arr ){
4     dic[e]++; //don't need find
5 }
6
7 std::vector< std::pair<int, int> > data( dic.begin(), dic.end() ); //build vector
    directly.
8
9 std::partial_sort( data.begin(), data.begin()+2, data.end(), []( auto &e1, auto &e2 ){
10    return e1.second > e2.second; } );
11 //use lambda to descend order.
12
13 for( auto &e: data ){
14     cout << e.first << "——>" << e.second << endl;
15 }

```

3.5 Typical interview questions

3.5.1 data structure

3.5.1.1 array

1) maximum difference from right element(one time stock buy and sell). 2) circular queue, three points: 1) empty, 2) full and 3) $\text{front} + 1 / \text{length} = \text{rear}$. 3) stock one buy and sell. 4) next larger (leader problem) (mono stack). 5) maximum sliding window(mono queue). 6) partial sum used for random select with weight.

3.5.1.2 list

1) Reverse link list. 2) judge if there is loop, where is the entry of loop?

3.5.1.3 tree

1) LCA, (How to resolve if only one node is here?) 2) Judge if tree is valid BST? (how to understand min and max). 3) Number of BST? 4) Build tree from serialize. 5) delete node if path sum $< k$.

3.5.1.4 graph

Offline union-find algorithm and use this to find spanning tree.

3.5.1.5 interval

- one meeting room and minimum meeting room (need end sort.)
- delete overlap interval
- combine overlap interval.
- video combine,

3.5.1.6 Two pointers-slow and fast

duplicate and partition

3.5.1.7 Two pointers-sliding windows

leetcode 76. Minimum Window Substring

3.5.1.8 D&C

nth element, find rotate, and left boundary. These three methods has different left and right boundary. In these method, we usually input first index and last index (not size of array), and also left close, right close interval, this makes programming a little easier.

A variant is index is in the middle. these questions includes all parenthesis, child tree and matrix multiplication.

```

1 fun( int l, int r)
2
3 for(int k = l ; k<r; k++){
4     fun( l, k);
5     fun( k, r)
6 }

```

3.5.1.9 Binary search

1) monkey eat banana

3.5.1.10 Backtracking

Some common examples: 1) permutation, subset and combination. Knapsack problem. 2) Legal parentheses combination. 3) The most complicated example is k equal subset problem.

3.5.2 Famous questions in leetcode

There are four parts:

1. The labuladong.
2. top 100/150 leetcode.
3. All questions above.
4. All easy level questions below.

3.5.2.1 Math, permutation and combination.

- 3010, once find min, replace with max, then do next find. flood island. Another method is to find two minimal elements.

We set the first element of the array as a, the smallest element among the remaining elements as b, and the second smallest element as c. The answer is $a+b+c$.

```

1 int minimumCost( vector<int>& nums) {
2     int a = nums[0], b = 100, c = 100;
3     for (int i = 1; i < nums.size(); ++i) {
4         if (nums[i] < b) {
5             c = b;
6             b = nums[i];
7         } else if (nums[i] < c) {
8             c = nums[i];
9         }
10    }
11    return a + b + c;
12 }

```

- 1175, how to get prime? The start point is 2. permutation number, use $cnt*n-cnt$.
- 2652, Given a positive integer n, find the sum of all integers in the range $[1, n]$ inclusive that are divisible by 3, 5, or 7. Return an integer denoting the sum of all numbers in the given range satisfying the constraint. You need to know the inclusion-exclusion principle.

```

1 int sumOfMultiples(int n) {
2     auto f = [&](int x) {

```



```

3   int m = n / x;
4   return (x + m * x) * m / 2;
5   };
6   return f(3) + f(5) + f(7) - f(3 * 5) - f(3 * 7) - f(5 * 7) + f(3 * 5 * 7);
7   }

```

- 2928, Combination Mathematics + Principle of Inclusion-Exclusion. see the reference

```

1   https://leetcode.cn/problems/distribute-candies-among-children-i/solutions
    /2522970/o1-rong-chi-yuan-li-pythonjavacgo-by-end-smj5/

```

- leetcode 1539. binary search. another similar question is leetcode 41, they all need extra index information to help you to resolve the problem.
- 2970, and 1539 are not very directly, but you need to change the question to a logic on a single element. just like a rain drop problem, need a little "building model" problem.

3.5.2.2 String

- 2273, use array to judge if it's anagram. don't use map here, you can use char cnt [26] directly, it's easier than map or unordered_map. Don't build two cnt[26] either, you can break the loop earlier, that is the common style to judge if it's an anagram.
- 1668 and 1566 a little similar. 1566 just check pattern, I don't know what pattern content is. For 1668, we know what the pattern is. so I can use string.starts_with

1. 1566 Given an array of positive integers arr, find a pattern of length m that is repeated k or more times.

A pattern is a subarray (consecutive sub-sequence) that consists of one or more values, repeated multiple times consecutively without overlapping. A pattern is defined by its length and the number of repetitions.

Return true if there exists a pattern of length m that is repeated k or more times, otherwise return false.

2. 1668: For a string sequence, a string word is k-repeating if word concatenated k times is a substring of sequence. The word's maximum k-repeating value is the highest value k where word is k-repeating in sequence. If word is not a substring of sequence, word's maximum k-repeating value is 0.

Given strings sequence and word, return the maximum k-repeating value of word in sequence.

```

1   int cntRepeating(string_view sv, string word){
2   int cnt{};
3   int ws = word.size();
4   while(sv.starts_with(word)){
5       ++cnt;
6       sv = sv.substr(ws);
7   }
8   return cnt;
9   }
10
11  int maxRepeating(string sequence, string word) {
12      /* doesn't work,
13      aaabaaaabaabaaaabaabaaaabaabaaaaba
14      aaaba should give 5, but below just give 4
15      auto pos = sequence.find(word, 0);
16      int ws = word.size();
17      if(pos == string::npos) {
18          return 0;

```

```

19     }
20     int ans{1};
21     int result{};
22     auto init = pos+ws;
23     while((pos = sequence.find(word, init)) != string::npos){
24         if(pos - init == 0){
25             ++ans;
26             init = pos+ws;
27         }
28         else{
29             result = max(result, ans);
30             ans = 1;
31             init = pos+ws;
32         }
33     }
34     result = max(result, ans);
35     return result;*/
36
37
38
39     int n = sequence.size();
40     int i = 0;
41     int result{};
42     while(i<n){
43         auto vv = sequence | views::drop(i);
44         string_view sv(vv);
45         result = max(result, cntRepeating(sv, word));
46         ++i;
47     }
48     return result;
49 }

```

- leetcode 1576. look for a different characters with left and right, you can loop through "abc" and it's enough, don't need to loop through 26 character, it's a little trick.
- 1078, Given two strings first and second, consider occurrences in some text of the form "first second third", where second comes immediately after first, and third comes immediately after second. Return an array of all the words third for each occurrence of "first second third".
Answer: learn how to use views::split and views::slide. **The result is subrange of subrange.**

```

1  auto vw = text | views::split(' ') | views::slide(3);
2  vector<string> result;
3
4  for(auto e : vw){
5      auto it = e.begin();
6      auto e1 = string_view{*it};
7      ++it;
8      auto e2 = string_view{*it};
9      if(e1 == first && e2 == second){
10         ++it;
11
12         result.emplace_back( (*it).begin(), (*it).end() );
13     }
14 }
15
16 return result;
17
18
19 //a better version is adjacent<3>
20 vector<string> findOccurrences(string text, string first, string second) {
21     auto vv = text | views::split(' ') | views::adjacent<3>;

```

```

22     vector<string> ans;
23     for( auto const &[f, s, t] : vv){
24         if(string_view{f} == first && string_view{s} == second){
25             ans.push_back(string(t.begin(), t.end()));
26         }
27     }
28     return ans;
29 }

```

- 1071, For two strings s and t, we say "t divides s" if and only if $s = t + t + t + \dots + t + t$ (i.e., t is concatenated with itself one or more times). Given two strings str1 and str2, return the largest string x such that x divides both str1 and str2.

```

1 string gcdOfStrings(string str1, string str2) {
2     if (str1 + str2 != str2 + str1) return "";
3     int n = __gcd(str1.size(), str2.size());
4     return str1.substr(0, n);
5 }

```

3.5.2.3 Double pointer

- 3318, map and set combine, roof of usage of different containers. A very good example, need to remember it.
- 2903, double pointers.
- 3095, subarray, have to use double pointer(sliding windows.), count all set 1, revoke or.
- 3090, a typical double pointer, but you need to get maximum, so this maximum lie in when you expand right pointer, if you want to know the minimum, the value is in the shrink of left pointer.
- 680, Given a string s, return true if the s can be palindrome after deleting at most one character from it. We use two pointers to point to the left and right ends of the string, respectively. Each time, we check whether the characters pointed to by the two pointers are the same. If they are not the same, we check whether the string is a palindrome after deleting the character corresponding to the left pointer, or we check whether the string is a palindrome after deleting the character corresponding to the right pointer. If the characters pointed to by the two pointers are the same, we move both pointers towards the middle by one position, until the two pointers meet.

If we have not encountered a situation where the characters pointed to by the pointers are different by the end of the traversal, then the string itself is a palindrome, and we return true.

Any palindrome questions can be resolved by two pointers.

3.5.2.4 STL container

- 2696, A typical usage of stack.
- 1700 usage of stack and queue, pay attention to stack push need reverse order. use stack, that is very good idea, need to remember it.

```

1     int countStudents(vector<int>& students, vector<int>& sandwiches) {
2         int cnt[2] = {0};
3         for (int& v : students) ++cnt[v];
4         for (int& v : sandwiches) {
5             if (cnt[v]-- == 0) {
6                 return cnt[v ^ 1];
7             }
8         }
9         return 0;
10    }

```

- 3238, map< map>

```

1  int winningPlayerCount(int n, vector<vector<int>>& pick) {
2      vector<int> result(n+1, 0);
3      int ans{};
4
5      map<int, map<int, int>> ms;
6      for(const auto & e : pick){
7          auto f = e[0];
8          auto s = e[1];
9
10         // ++v[f];
11         auto &bs = ms[f];
12         ++bs[s];
13
14         if(bs[s] > f){
15             result[f] = 1;
16         }
17     }
18
19     return ranges::count(result, 1);
20 }

```

- 2558, priority_queue usage. A good example.
- 2956, O(1) use unordered_map, or log(n) use sort and binary search. set operation is not suitable for this question.
- 2006, two_sum variant.

3.5.2.5 STL Algorithm

- 2231, to_string, stoi, sort, accumulate.

```

1  int largestInteger(int num) {
2      string str = to_string(num);
3      vector<char> even;
4      vector<char> odd;
5      for(auto e: str){
6          if( (e-'0') %2 == 0){
7              even.push_back(e);
8          }
9          else{
10             odd.push_back(e);
11         }
12     }
13     ranges::sort(even);
14     ranges::sort(odd);
15     vector<char> result;
16     cout<<n<<endl;
17
18     for(auto e: str){
19         if( (e-'0') %2 == 0){
20             result.push_back(even.back());
21             even.pop_back();
22         }
23         else{
24             result.push_back(odd.back());
25             odd.pop_back();
26         }
27     }
28     string rs{};

```

```

29     rs = accumulate(result.begin(), result.end(), rs, [](string s, auto e){
30         s += e;
31         return s;
32     });
33
34     return stoi(rs);
35 }

```

- 3174, how to use `std::erase` directly.
- 1913, max or min two, max three(`nth_element` or `priority_queue`?).

```

1 1) numeric<int>::max()
2 2) numeric<int>::min()
3
4 3) compare with max1 and max2, if bigger then max1, then max2 = max1. That is a
    common pattern.

```

- 1636, use `sort`, that idea is very interesting. How to second increasing, the first decreasing, **sort on nums directly**

```

1 vector<int> frequencySort(vector<int>& nums) {
2     vector<int> cnt(201);
3     for (int v : nums) {
4         ++cnt[v + 100];
5     }
6     sort(nums.begin(), nums.end(), [&](const int a, const int b) {
7         if (cnt[a + 100] == cnt[b + 100]) return a > b;
8         return cnt[a + 100] < cnt[b + 100];
9     });
10    return nums;
11 }

```

```

1 map<int, int> m;
2 for(auto e: nums){
3     ++m[e];
4 }
5
6 vector<pair<int, int>> v(m.begin(), m.end());
7 sort(v.begin(), v.end(), [](const auto& e1, const auto& e2){
8     if(e1.second != e2.second){ //The second increasing
9         return e1.second < e2.second;
10    }
11    return e1.first > e2.first; //The first decreasing.
12 });
13
14 vector<int> result;
15 result.reserve(nums.size());
16 for(auto [f, s] : v){
17     for(int i = 0; i < s; ++i){
18         result.push_back(f);
19     }
20 }
21 return result;

```

- 2465, a typical question can be resolved by the STL algorithms very efficiently. Use `std::set` to calculate the different items count.

```

1 int distinctAverages(vector<int>& nums) {
2     set<int> s;
3     ranges::sort(nums);

```

```

4
5   transform(nums.begin(), nums.begin()+nums.size()/2, nums.rbegin(), inserter(s, s
      .begin()), [](auto e1, auto e2){return e1+e2;});
6
7   return s.size();
8
9 }

```

3.5.2.6 fancy code

- Find all space, don't need while(find), use for() directly. find one max and second max, use for loop is ok, find three max, use min-heap, find three min, use max-heap.
- 1800, a typical boundary question, add one element to break the condition will reduce the programming complexity a lot. that is trick I should follow.

```

1 int maxAscendingSum(vector<int>& nums) {
2     int i = 0;
3
4     nums.push_back(nums.back()-1);
5
6     int n = nums.size();
7     int sum{};
8     int result{};
9     while(i<n-1){
10         if(nums[i]<nums[i+1]){
11             sum +=nums[i];
12         }
13         else{
14             sum += nums[i];
15             if(sum>result){
16                 result = sum;
17             }
18             sum = 0;
19         }
20         ++i;
21     }
22
23     return result;
24 }

```

- 2138, dumb node idea. add $n\%k$ fill to the end. For any complex boundary condition, add dumb node to make programming easier. that is a good idea.

```

1 vector<string> divideString(string s, int k, char fill) {
2     int n = s.size();
3     if(n%k != 0){
4         s.append(k-n%k, fill);
5     }
6     auto v = s | ranges::views::chunk(k);
7     vector<string> result;
8     for(auto e : v){
9         result.emplace_back(e.begin(), e.end()); //emplace_back also use very well
            here.
10    }
11    return result;
12 }

```

- 2733, how to get middle of three?

- 2108, use more lambda

```

1 auto ll = [](const auto & e){
2     int n = e.size();
3     return equal(e.begin(), e.begin()+n/2, e.rbegin());
4 };
5
6 auto it = ranges::find_if(words, ll);
7 return it == words.end() ? "" : *it;

```

- 1816, don't need split and stringstream. A classical answer is the bottom line, after you know the bottom line, then you need to think that if there is better method to do that.

```

1 string truncateSentence(string s, int k) {
2     for (int i = 0; i < s.size(); ++i) {
3         if (s[i] == '.' && (--k) == 0) {
4             return s.substr(0, i);
5         }
6     }
7     return s;
8 }

```

3.5.2.7 views

- 2190, how to get best sell book name. Pay attention to the decltype and value_type usage.

```

1 auto vw = nums | views::slide(2);
2 unordered_map<int, int> um;
3 for(auto e: vw){
4     int f = *(e.begin());
5     int s = *(e.begin()+1);
6     if(f == key){
7         ++um[s];
8     }
9 }
10
11 //1) use ranges.
12 //2) use decltype.
13 //3) use value_type and projection.
14 auto it = ranges::max_element(um, {}, &decltype(um)::value_type::second);
15 return it->first;

```

- leetcode 1556. use reverse, chunk and join_with common.

```

1 string thousandSeparator(int n) {
2     string str = to_string(n);
3     auto vw = str | views::reverse | views::chunk(3) | views::join_with('.') | views::common;
4
5     string result(vw.begin(), vw.end());
6
7     ranges::reverse(result);
8     return result;
9 }
10 }

```

- 561, Given an integer array nums of 2n integers, group these integers into n pairs (a1, b1), (a2, b2), ..., (an, bn) such that the sum of min(ai, bi) for all i is maximized. Return the maximized sum. In order to make the sum as large as possible, the b we choose should be as close to a as

possible, so as to retain a larger number. Therefore, we can sort the array then divide every two adjacent numbers into a group, and add the first number of each group.

You need to sort it first, then use views function to deal with it.

```

1  ranges::sort(nums);
2  auto vw = nums | views::chunk(2);
3  int sum{};
4  for(auto e: vw){
5      int f = e.front();
6      sum+=f;
7  }
8  return sum;

```

- 1694, pay attention to boundary check all the times.

```

1  string reformatNumber(string number) {
2      auto v1 = number | views::filter([](auto e){return ::isdigit(e);}) | ranges::to<
        string>();
3      string result{};
4      if(v1.size()%3 == 1){
5          string v2 = v1 | views::take(v1.size() - 4) | views::chunk(3) | views::join_with
            ('-') | ranges::to<string>();
6
7          string v3 = v1 | views::drop(v1.size() - 4) | views::chunk(2) | views::
            join_with('-') | ranges::to<string>();
8          result = v2 + '-' + v3;
9
10     }
11     else{
12         result = v1 | views::chunk(3) | views::join_with('-') | ranges::to<string>();
13     }
14     return result;
15 }

```

- 1108. Defanging an IP Address. if join_with string, then change it to string_view first.

```

1  string defangIPAddr(string address) {
2      string temp = "[.]";
3      auto v1 = address | views::split('.') | views::join_with(temp);
4      string s = v1 | ranges::to<string>();
5  }

```

- 3162, Use cartesian_product to simulate two for loop. cartesian_product returns the tuples.

```

1  int numberOfPairs(vector<int>& nums1, vector<int>& nums2, int k) {
2      int sum{};
3      for(auto [f, s]: views::cartesian_product(nums1, nums2)){
4          if(f % (s*k) == 0)
5              ++sum;
6      }
7      return sum;
8  }

```

- 1662, Learn how to use views::join. Another useful function is views::join_with can be used to add , to number.

```

1  //1) views::join usage
2  //2) ranges::equal
3  auto vw1 = word1 | views::join;
4  auto vw2 = word2 | views::join;
5
6  return ranges::equal(vw1, vw2);

```


3.5.2.8 subset and subarray

- 2062, how to get all substring? double pointer is $O(n)$ algorithm, all substring must be $O(n^2)$, so we can't use double pointer here.
- 2264, use `adjacent_find` to chunk equal. pay attention to the last statement after while.

```

1  std::vector<int> v1{0, 1, 2, 3, 40, 40, 41, 41, 5, 5, 5};
2
3  auto it = v1.begin();
4  auto pos = v1.begin();
5  while( (pos = std::adjacent_find(it, v1.end(), not_equal_to{})) != v1.end()){
6      for(auto i = it; i < next(pos); ++i){
7          cout << *i << "_";
8      }
9      cout << endl;
10     it = pos + 1;
11 }
12 if(it != v1.end()){
13     cout << "Yan";
14     for(auto i = it; i < v1.end(); ++i){
15         cout << *i << "_";
16     }
17     cout << endl;
18 }

```

1. chunk the equal,
2. chunk the digit and alphabet. leetcode 1805, use `chunk_by` to separate abc and number.

```

1  string str = "123abc456efg";
2
3  auto v = str | views::chunk_by([](auto e1, auto e2){
4      if(::isdigit(e1)){
5          if(::isdigit(e2))
6              return true;
7          return false;
8      }
9      else{
10         if(::isalpha(e2))
11             return true;
12         return false;
13     }
14 });
15
16 for(auto e: v){
17     string temp{e.begin(), e.end()};
18     cout << temp << endl;
19 }

```

3. only chunk the vowel. 2062, you can use `cnt >= 1`, once meet consonant, `cnt = 0`; that will be easier. it's different with 2264, 2264 need to check two element to see if they are not equal, 2062 just check one element to see if it's consonant, so can implement a little easier.
- 1863, how to get subset, how to calculate sum? calculating the only one sum, don't need calculate the subsum of each subset, you can add each item in the subset in the last sum. The second question is how to get all subset? two for doesn't work, because it's suitable for subarray. not subset. subset is 2^n , not n^2 . Three common ways 1) get n , build 2^n bit, then use bit to select 2) recursive bucket point of view, 3) recursive ball point of view.

```

1  int n = nums.size();
2  int ans = 0;

```

```

3 function<void (int, int)> dfs = [&](int i, int s) {
4     if (i >= n) {
5         ans += s;
6         return;
7     }
8     dfs(i + 1, s);
9     dfs(i + 1, s ^ nums[i]);
10 };
11 dfs(0, 0);
12 return ans;
13
14 //1) dfs should be function, auto doesn't work here.
15 //2) nums can use directly inside the lambda, because you have &
16 //3) s is subsum each subset, don't use reference here.
17 //4) Base case is i == n, then accumulate s to the last result ans.

```

```

1 void bt1(vector<int>& v, vector<int>& result, int index, int len, int xor_sum, int
    & sum) {
2
3     /*
4     for(auto e: result){
5         cout<<e<<" ";
6     }
7     cout<<endl;
8     */
9     cout<<xor_sum<<endl;
10
11     sum +=xor_sum;
12
13     for(int i = index; i< len; ++i){
14         result.push_back(v[i]);
15         xor_sum^= v[i];
16         bt1(v, result, i+1, len, xor_sum, sum);
17
18         result.pop_back();
19         xor_sum^=v[i];
20     }
21 }
22
23
24 void bt(vector<int>& v, vector<int>& result, int index, int len){
25     if(index == len){
26         for(auto e: result){
27             cout<<e<<"_";
28         }
29         cout<<endl;
30         return;
31     }
32
33     bt(v, result, index+1, len);
34
35     result.push_back(v[index]);
36     bt(v, result, index+1, len);
37     result.pop_back();
38 }
39

```

- 1588, dp, also subarray question, but have easy solution.
- 2913, how to get all subarray? use two for loop can get all subarray, use set to count all different nums.

3.5.2.9 tree

- 108 use recursive to build a tree;

```

1  TreeNode* buildTree(vector<int>& nums, int i, int j){
2      if( i == j )
3          return nullptr;
4
5
6      int m = i + (j-i)/2;
7      int v = nums[m];
8      TreeNode *root = new TreeNode(v);
9      root->left = buildTree(nums, i, m);
10     root->right = buildTree(nums, m+1, j);
11     return root;
12 }
13 TreeNode* sortedArrayToBST(vector<int>& nums) {
14     int n = nums.size();
15     return buildTree(nums, 0, n);
16
17 }
```

- 101, use recursive to judge mirror, just like to judge if it's same. the same idea. recursive is power weapon.

```

1  bool isSymmetric(TreeNode* root) {
2      auto dfs = [&](this auto&& dfs, TreeNode* root1, TreeNode* root2) -> bool {
3          if (root1 == root2) {
4              return true;
5          }
6          if (!root1 || !root2 || root1->val != root2->val) {
7              return false;
8          }
9          return dfs(root1->left, root2->right) && dfs(root1->right, root2->left);
10     };
11     return dfs(root->left, root->right);
12 }
```

- 700. Search in a Binary Search Tree

```

1  TreeNode* searchBST(TreeNode* root, int val) {
2      if(root == nullptr){
3          return nullptr;
4      }
5      TreeNode* l = searchBST(root->left, val);
6      if(l)
7          return l;
8      if(root->val == val)
9          return root;
10     return searchBST(root->right, val);
11 }
```

- 572. Given the roots of two binary trees root and subRoot, return true if there is a subtree of root with the same structure and node values of subRoot and false otherwise.

```

1  class Solution {
2  public:
3      bool isSubtree(TreeNode* root, TreeNode* subRoot) {
4          if (!root) {
5              return false;
6          }
7      }
```

```

7      return same(root, subRoot) || isSubtree(root->left, subRoot) ||
      isSubtree(root->right, subRoot);
8  }
9
10     bool same(TreeNode* p, TreeNode* q) {
11         if (!p || !q) {
12             return p == q;
13         }
14         return p->val == q->val && same(p->left, q->left) && same(p->right, q->
15             right);
16     }
};

```

- 530 Minimum Absolute Difference in BST. This code demonstrates how to keep previous node in BST.

```

1  class Solution {
2  public:
3      TreeNode* prev;
4      int min = numeric_limits<int>::max();
5
6      void getM_re(TreeNode* root) {
7          if (root == nullptr)
8              return;
9
10         getM_re(root->left);
11         if (prev != nullptr) {
12             int temp = root->val - prev->val;
13             if (temp < min) {
14                 min = temp;
15             }
16         }
17         prev = root;
18         getM_re(root->right);
19     }
20     int getMinimumDifference(TreeNode* root) {
21         getM_re(root);
22         return min;
23     }
24 };

```

- 2331, no check leaf node, is it right? commonly, it's not right. only check if it's leaf code can't handle below tree. but for questions 2331, it states that it's full binary tree, so you can only check leaf code without handle base case.

```

1      1
2     /\
3    null 2

```

- 2932. Maximum Strong Pair XOR I. Learn how to build trie tree.

```

1      https://www.geeksforgeeks.org/maximum-xor-of-two-nodes-in-a-tree/

```

3.5.2.10 interval

- 2848, update tail correct. That is a typical interval questions.

```

1  int numberOfPoints(vector<vector<int>>& nums) {
2      ranges::sort(nums);

```

```

3  int result {};
4  vector<int> origin = nums[0];
5  int index = 1;
6  int n = nums.size();
7  int tail = origin[1];
8
9  for (; index < n; ++index) {
10     if (nums[index][0] <= tail) {
11         if (tail < nums[index][1]) {
12             tail = nums[index][1];
13         }
14     }
15     else {
16         result += tail - origin[0] + 1;
17         origin = nums[index];
18         tail = origin[1];
19     }
20 }
21
22 result += tail - origin[0] + 1;
23
24 return result;
25 }

```

3.5.2.11 bit

- trick

```

1  n>>1 // n/2
2  n&1  // n%2
3  n^1  // flip n
4
5  n&0xf //n %16.

```

- 405, how to modulus negative? common bit operation.

```

1  string toHex(int num) {
2      if (num == 0) return "0";
3      string s = "";
4      for (int i = 7; i >= 0; --i) {
5          int x = (num >> (4 * i)) & 0xf;
6          if (s.size() > 0 || x != 0) {
7              char c = x < 10 ? (char) (x + '0') : (char) (x - 10 + 'a');
8              s += c;
9          }
10     }
11     return s;
12 }

```

- 2595, sizeof(n) must multiply 8

```

1  // judge if a bit is 1? use if directly.
2  if (n & (1<<index)) {
3
4
5  vector<int> evenOddBit(int n) {
6      vector<int> ans(2);
7      for (int i = 0; n > 0; n >>= 1, i ^= 1) {
8          ans[i] += n & 1;
9      }

```

```

10  return ans;
11  }

```

- 2103, use bit to represent RGB color, should learn this trick.

```

1  vector<10, bitset<3>> > result;
2  ....

```

- 2220, how to understand $\hat{?}$ and how to know how many set bits in the binary represent.

```

1  bitset<sizeof(int)*8> b1(start);
2  bitset<sizeof(int)*8> b2(goal);
3  b1 ^= b2;
4  return b1.count();

```

- 3340, when to use stol? when not to use stol? stol to avoid overflow, how to flip flag.

```

1  bool isBalanced(string num) {
2      int f[2]{};
3      for (int i = 0; i < num.size(); ++i) {
4          f[i & 1] += num[i] - '0'; //pay attention to i & i.
5      }
6      return f[0] == f[1];
7  }

```

- 2315, how to flip the bit? You are given a string s, where every two consecutive vertical bars '|' are grouped into a pair. In other words, the 1st and 2nd '|' make a pair, the 3rd and 4th '|' make a pair, and so forth.

Return the number of '*' in s, excluding the '*' between each pair of '|'.

```

1  int countAsterisks(string s) {
2      int ans = 0, ok = 1;
3      for (char& c : s) {
4          if (c == '*') {
5              ans += ok;
6          } else if (c == '|') {
7              ok ^= 1;
8          }
9      }
10     return ans;
11 }

```

- 2032, map<int, bitset<3>> a new data structure. Because the bitset support count() function, it's very useful.

3.5.2.12 Good modeling

- 1854, so many detail. 1) priority_queue(smaller) 2) how to pop up priority_queue. That is how many meeting room questions.
- 1275, only last one decide who will win.
- 1886, how to rotate a array? map $i, j \rightarrow j, i, i, j \rightarrow n-j, n-i$.

0	i, j	i, j
90	i, j	$j, n-i$
180	i, j	$n-i, n-j$
270	i, j	$n-j, i$

second $\rightarrow$ first, first $\rightarrow$ n-first, that is all.

- 2873 We can use two variables `mx` and `mxdiff` to maintain the maximum prefix value and maximum difference, respectively. When traversing the array, we update these two variables, and the answer is the maximum value of all
- leetcode 1566, 2765, different with 2760. 2760 can use chunk, but 2765 can't use chunk. has to use iterate method. They all want to get subarray which satisfy certain condition. single iterate, but maintain previous condition, if condition ok, update the last result, if condition not ok, initial to the initial condition. These three questions you need to remember, 1) how to check pattern? 2) odd and even can be used splitted so use chunk 3) increase and decrease can't be split, so use double iterate. for example 2 3 4 3 (2, 3) is answer, but 3 4 3 is also answer, it overlaps on 3.
 1. 1566 Given an array of positive integers `arr`, find a pattern of length `m` that is repeated `k` or more times.
A pattern is a subarray (consecutive sub-sequence) that consists of one or more values, repeated multiple times consecutively without overlapping. A pattern is defined by its length and the number of repetitions.
Return true if there exists a pattern of length `m` that is repeated `k` or more times, otherwise return false.
 2. 2760 You are given a 0-indexed integer array `nums` and an integer threshold.
Find the length of the longest subarray of `nums` starting at index `l` and ending at index `r` ($0 \leq l \leq r < \text{nums.length}$) that satisfies the following conditions:
`nums[l] % 2 == 0` For all indices `i` in the range `[l, r - 1]`, `nums[i] % 2 != nums[i + 1] % 2` For all indices `i` in the range `[l, r]`, `nums[i] <= threshold` Return an integer denoting the length of the longest such subarray.
 - 3.
- leetcode 1184 A bus has `n` stops numbered from 0 to `n - 1` that form a circle. We know the distance between all pairs of neighboring stops where `distance[i]` is the distance between the stops number `i` and `(i + 1) % n`. The bus goes along both directions i.e. clockwise and counterclockwise. Return the shortest distance between the given start and destination stops.
 1. `(start+1)%n` to simulate the circular operation.
 2. only need to calculate `t`, `s-t` represents the other.

```

1  int distanceBetweenBusStops(vector<int>& distance, int start, int destination)
2  {
3      int s = accumulate(distance.begin(), distance.end(), 0);
4      int t = 0, n = distance.size();
5      while (start != destination) {
6          t += distance[start];
7          start = (start + 1) % n;
8      }
9      return min(t, s - t);

```

- 2515, how to deal with circular array. same with previous.
- 942, A permutation `perm` of `n + 1` integers of all the integers in the range `[0, n]` can be represented as a string `s` of length `n` where: Input: `s = "IDID"` Output: `[0,4,1,3,2]`
We can use two pointers `low` and `high` to represent the current minimum and maximum values, respectively. Then, we traverse the string `s`. If the current character is `I`, we add `low` to the result array, and increment `low` by 1; if the current character is `D`, we add `high` to the result array, and decrement `high` by 1. Finally, we add `low` to the result array and return the result array.
- 1037, how to decide if three points are in line, change divide to multiplication.
- 821, Given a string `s` and a character `c` that occurs in `s`, return an array of integers `answer` where `answer.length == s.length` and `answer[i]` is the distance from index `i` to the closest occurrence of

character c in s . The distance between two indices i and j is $\text{abs}(i - j)$, where abs is the absolute value function.

Next, we traverse the string s from left to right, recording the most recent position of the character c as pre . For position i , the answer is $i - pre$, i.e., $ans[i] = i - pre$. Then, we traverse the string s from right to left, recording the most recent position of the character c as suf . For position i , the answer is $suf - i$, i.e., $ans[i] = \min(ans[i], suf - i)$. Finally, we return the answer array ans . The time complexity is $O(n)$, where n is the length of the string s . Ignoring the space consumed by the answer array, the space complexity is $O(1)$.

- 696, Given a binary string s , return the number of non-empty substrings that have the same number of 0's and 1's, and all the 0's and all the 1's in these substrings are grouped consecutively. Substrings that occur multiple times are counted the number of times they occur.

```

1 int countBinarySubstrings(string s) {
2     int i = 0, n = s.size();
3     vector<int> t;
4     while (i < n) {
5         int cnt = 1;
6         while (i + 1 < n && s[i + 1] == s[i]) {
7             ++cnt;
8             ++i;
9         }
10        t.push_back(cnt);
11        ++i;
12    }
13
14
15    int ans = 0;
16    for (i = 1; i < t.size(); ++i)
17        ans += min(t[i - 1], t[i]);
18    return ans;
19    //return 0;
20 }

```

- leetcode 263, Ugly Number. An ugly number is a positive integer which does not have a prime factor other than 2, 3, and 5. Given an integer n , return true if n is an ugly number.
- leetcode 292, Nim Game. One use or, the other use and, it will satisfy the requirement.

```

1 bool NimRecursive(int n, bool flag){
2     if( flag == true&& n<=3){
3         return true;
4     }
5     if(flag == false && n<=3){
6         return false;
7     }
8     if(flag){
9         flag = flag ? false : true;
10        if(NimRecursive(n-1, flag ) || NimRecursive(n-2, flag ) || NimRecursive(n-3,
11            flag ) ){
12            return true;
13        }
14        return false;
15    }
16    else{
17        flag = flag ? false : true;
18        if(NimRecursive(n-1, flag ) && NimRecursive(n-2, flag ) && NimRecursive(n-3,
19            flag ) ){
20            return true;
21        }
22        return false;
23    }
24 }

```



```

21 }
22
23 }
24 bool canWinNim(int n) {
25     bool flag = true;
26     return NimRecursive(n, flag);
27 }

```

- 463. Island Perimeter, for each 1 block, only consider right and down, then -=2, it's easier than considering four sides.

```

1 int islandPerimeter(vector<vector<int>>& grid) {
2     int m = grid.size(), n = grid[0].size();
3     int ans = 0;
4     for (int i = 0; i < m; ++i) {
5         for (int j = 0; j < n; ++j) {
6             if (grid[i][j] == 1) {
7                 ans += 4;
8                 if (i < m - 1 && grid[i + 1][j] == 1) ans -= 2;
9                 if (j < n - 1 && grid[i][j + 1] == 1) ans -= 2;
10            }
11        }
12    }
13    return ans;
14 }

```

- 2908, keep left and right minimum, just like rain drop problem.
- 2073, judge first, then ++ index. that is the basic style.

```

1 int timeRequiredToBuy(vector<int>& tickets, int k) {
2     int ans = 0;
3     for (int i = 0; i < tickets.size(); ++i) {
4         ans += min(tickets[i], i <= k ? tickets[k] : tickets[k] - 1);
5     }
6     return ans;
7 }

```

- 2506. Count Pairs Of Similar Strings. **Map 26 letter to a inter. words[i] consist of only lowercase English letters.** is very important clue.

```

1 int similarPairs(vector<string>& words) {
2     int ans = 0;
3     unordered_map<int, int> cnt;
4     for (const auto& s : words) {
5         int x = 0;
6         for (auto& c : s) {
7             x |= 1 << (c - 'a'); //you need to remember this common snippet
8         }
9         ans += cnt[x]++;
10    }
11    return ans;
12 }

```

3.6 Project

- drop of knowledge C++, sixth edition. includes all modern C++ 17/20 features. Such as move semantic, smart pointer, concurrency, brace initialization and generic programming.
- A compiler project. Intermediate representation, Whirl to LLVM.

- A Trade Repository or Swap Data Repository is an entity that centrally collects and maintains the records of over-the-counter (OTC) derivatives. CFTC. Commodity Futures Trading Commission. Athena is a cross-asset platform transforming technology at JP Morgan. It delivers innovative and efficient applications to a wide range of the firm's businesses, including sales, trading, operations, risk and research. Athena combined the best of open source technologies
- There are two main types of RFID tags: battery-operated and passive.
- Tell me about a time you worked hard at something but the end result was unsuccessful. Why? What did you learn?

UH64 is a C++ compiler with its own intermediate representation (IR) called WHIRL. Clang, a highly successful commercial C++ compiler, uses LLVM as its IR. My role in the project was to convert WHIRL to LLVM. The goal was to extend LLVM's support for more CPUs while leveraging WHIRL's optimizations in certain areas, combining the strengths of both.

The main challenge was the large codebase, which consists of 4 million lines of code. To address this, I focused primarily on the IR domain. I also used a powerful IDE called Understand, which helps analyze source code quickly and efficiently.

- What was the most demanding or time-consuming project or initiative in your recent work experience?

We need to submit reports to the CFTC within 15 minutes, or we face a fine of \$15 per transaction. Initially, we used a single server for this task, but due to the volatility of the FX market, which is influenced by politics, we decided to deploy an additional server.

The reporting system is developed in Python, but since Python doesn't handle concurrency well, we implemented multiprocessing. To manage synchronization, we also used a database. Additionally, we developed algorithms to make the system scalable, similar to a simple Kubernetes setup.

The results were positive. Each year, the CFTC provides a statistical report on all major banks, and JPMorgan ranked number one, with 98% of transactions being reported within 15 minutes.

- Can you tell me how you discovered a hidden requirement or non-obvious outcome in a recent project?

move the bar to open the lid. But the bar

- Share an example of when you changed a design approach because of someone else's input/better idea?
- Can you describe a time when you were stuck on a project or solution?

Double-free errors are very challenging to debug, especially in embedded applications. I attempted to use tools like Valgrind, but it wasn't supported in this environment. I also tried implementing a custom new operator, but it wouldn't compile successfully.

After reviewing the source code, I discovered that both processes were using the same thread pool. I resolved the issue by switching to a shared_ptr, which fixed the memory problem. I was stuck on this for almost two weeks, but now I feel very confident in handling any memory-related issues, even in highly constrained environments.

- In the most recent case where a team member suggested a change that you disagreed with, what steps did you take to reach a resolution?

I will develop a test case to approve my stand point view. Stop talking, Show me the source code.

- Vizio was acquired by Walmart and is currently undergoing a transformation, outsourcing technical work to offshore teams while focusing on the advertising business. I've been moved to developing GUI apps and interfaces, which doesn't allow me to fully use my skills in low-level algorithms and C++. That's why I'm looking for a new position.
- My wife is a resident doctor undergoing training at a hospital in Dallas. This is a rare opportunity

and a family decision.

- I have expert-level knowledge of C/C++ and have published a book on C++ and you can purchase it from amazon website. I hold a Ph.D. and have a patent in algorithms, which enables me to handle complex algorithmic and machine learning challenges. I have experience working on a C++ compiler, giving me a strong understanding of optimization and machine code. I have many years of experience, including extensive teamwork and CI/CD expertise. I'm familiar with the entire software development process and tools, essentially making me a full-stack developer.